

**SISTEM PEMANTAUAN LIMBAH CAIR INDUSTRI
TERINTEGRASI *INTERNET OF THINGS* DAN
JAVASCRIPT *OBJECT NOTATION* WEB TOKEN
BERBASIS *WEBSITE* DAN ANDROID**



Disusun oleh:

Imam Zaenal Abidin 4.31.21.1.15

Muhammad Adriano Khairur Rizky Setyawan 4.31.21.1.18

**PROGRAM STUDI SARJANA TERAPAN TEKNIK
TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI SEMARANG**

2025

**SISTEM PEMANTAUAN LIMBAH CAIR INDUSTRI
TERINTEGRASI *INTERNET OF THINGS* DAN
JAVASCRIPT *OBJECT NOTATION WEB* TOKEN
BERBASIS *WEBSITE* DAN ANDROID**



Tugas akhir ini disusun untuk melengkapi sebagian persyaratan menjadi Sarjana
Terapan

Disusun oleh:

Imam Zaenal Abidin 4.31.21.1.15

Muhammad Adriano Khairur Rizky Setyawan 4.31.21.1.18

**PROGRAM STUDI SARJANA TERAPAN TEKNIK
TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI SEMARANG
2025**

PERNYATAAN KEASLIAN TUGAS AKHIR

Saya menyatakan dengan sesungguhnya bahwa tugas akhir dengan judul “**Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript Object Notation Web Token Berbasis Website dan Android**” yang dibuat untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan pada Program Studi Teknik Telekomunikasi, Jurusan Teknik Elektro Politeknik Negeri Semarang, sejauh yang saya ketahui bukan merupakan tiruan atau duplikasi dari tugas akhir yang sudah dipublikasikan dan atau pernah dipakai untuk mendapatkan gelar Sarjana Terapan di lingkungan Politeknik Negeri Semarang maupun di perguruan tinggi atau instansi manapun, kecuali bagian yang sumber informasinya dicantumkan sebagaimana mestinya.

Mahasiswa I

Imam Zaenal Abidin
4.31.21.1.15

Semarang, 11 Agustus 2025

Mahasiswa II

Muhammad Adriano K. R.S.
4.31.21.1.18

HALAMAN PERSETUJUAN

Tugas akhir dengan judul **“Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript *Object Notation Web Token* Berbasis *Website* dan *Android*”** dibuat untuk melengkapi sebagian persyaratan menjadi Sarjana Terapan pada Program Studi Teknik Telekomunikasi, Jurusan Elektro Politeknik Negeri Semarang dan disetujui untuk diajukan dalam sidang ujian tugas akhir.

Pembimbing I,

Semarang, 11 Agustus 2025

Pembimbing II,

Dr. Amin Suharjono, S.T., M.T.
NIP. 197210271999031002

Roni Apriantoro, S.Tr.T., M.Tr.T.
NIP. 199604092022031007

Mengetahui,

Ketua Program Studi Sarjana Terapan Teknik Telekomunikasi

Ari Sriyanto Nugroho, S.T, M.T., M.Sc
NIP. 197409042005011001

HALAMAN PENGESAHAN

Tugas akhir dengan judul **“Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript *Object Notation Web Token* Berbasis *Website* dan *Android*”** Telah dipertahankan dalam ujian wawancara dan diterima sebagai syarat untuk menjadi Sarjana Terapan pada Program Studi Teknik Telekomunikasi, Jurusan Teknik Elektro Politeknik Negeri Semarang pada tanggal 21 Agustus 2025.

Tim Penguji

Penguji I,

Penguji II,

Penguji III,

Sindung H. W. S., BSEE., M.Eng.Sc.

NIP. 196301251991031001

Drs. Arif Nursyahid., M.T.

NIP. 196107171986031001

Catur Budi Waluyo, S.T., M.T.

NIP. 198809142022031006

Ketua Penguji,

Sekretaris Penguji,

Dr. Amin Suharjono, S.T., M.T.

NIP. 197210271999031002

Khamami, S. Ag., M.M.

NIP. 197206102000031001

Mengesahkan,

Ketua Jurusan Teknik Elektro

Yusnan Badruzzaman, S.T., M.Eng.

NIP. 197503132006041001

PRAKATA

Puji syukur Penulis panjatkan kepada Tuhan Yang Maha Esa. Atas berkat dan rahmat-Nya, Penulis dapat menyelesaikan tugas akhir ini yang berjudul “**Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript Object Notation Web Token Berbasis *Website* dan *Android***” dengan baik. Tugas akhir ini disusun untuk memenuhi salah satu persyaratan kelulusan Program Studi Sarjana Terapan Teknik Telekomunikasi Jurusan Teknik Elektro Politeknik Negeri Semarang.

Selama proses penyusunan tugas akhir ini, penulis menghadapi berbagai hambatan, baik dari segi teknis maupun non-teknis. Namun, dengan bantuan dari berbagai pihak dan tekad yang kuat, hambatan-hambatan tersebut dapat diatasi dengan baik. Oleh karena itu, penulis ingin menyampaikan ucapan terima kasih kepada:

1. Bapak Dr. Garup Lambang Goro, S.T., M.T., selaku Direktur Politeknik Negeri Semarang,
2. Bapak Yusnan Badruzzaman, S.T., M.Eng., selaku Ketua Jurusan Teknik Elektro Politeknik Negeri Semarang,
3. Bapak Ari Sriyanto Nugroho, S.T., M.T., M.Sc, selaku Ketua Program Studi Sarjana Terapan Teknik Telekomunikasi Politeknik Negeri Semarang,
4. Bapak Dr. Amin Suharjono, S.T., M.T., selaku Dosen Pembimbing I,
5. Bapak Roni Aprianoro, S.Tr.T., M.Tr.T., selaku Dosen Pembimbing II,
6. Dosen - dosen Program Studi Sarjana Terapan Teknik Telekomunikasi yang telah memberikan bimbingan, arahan, dan ilmu selama menempuh studi.
7. Orang tua serta keluarga penulis yang telah ikut memberikan doa, semangat, serta dukungan lain yang tak terhingga jumlahnya.
8. Teman-teman kelas TE-4B yang senantiasa memberikan dukungan dan motivasi kepada penulis, baik berupa materi maupun spiritual.

Penulis menyadari bahwa tugas akhir ini masih memiliki banyak kekurangan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran dari para pembaca guna

penyempurnaan di masa mendatang. Penulis berharap tugas akhir ini dapat memberikan manfaat baik bagi penulis sendiri, maupun bagi para pembaca.

Semarang, 11 Agustus 2025

Penulis

ABSTRAK

Imam Zaenal Abidin dan Muhammad Adriano Khairur Rizky Setyawan. **“Sistem Pemantauan Limbah Cair Industri Terintegrasi Internet of Things dan Javascript Object Notation Web Token Berbasis Website dan Android”**, Tugas Akhir Sarjana Terapan Jurusan Teknik Elektro Politeknik Negeri Semarang, di bawah bimbingan Bapak Dr. Amin Suharjono, S.T., M.T. dan Bapak Roni Apriantoro, S.Tr.T., M.Tr.T., 2025, 112 halaman.

Peningkatan kebutuhan terhadap sistem pemantauan limbah cair industri merupakan tantangan strategis dalam memastikan kepatuhan terhadap regulasi lingkungan dan mencegah pencemaran. Sebagian besar sistem yang telah ada belum mampu mengintegrasikan komunikasi multi-protokol, mekanisme keamanan berlapis, serta fitur notifikasi yang responsif. Penelitian ini merancang dan mengimplementasikan sistem pemantauan limbah cair berbasis *Internet of Things* (IoT) dengan JSON *Web Token* (JWT), mendukung protokol HTTP dan MQTT, dilengkapi enkripsi AES-128, HTTPS, serta antarmuka berbasis web dan Android. Metodologi yang digunakan adalah Agile-Scrum, mencakup tahap perencanaan, perancangan, implementasi, serta pengujian meliputi *black box*, keamanan, performa, notifikasi, dan *User Acceptance Testing* (UAT). Hasil penelitian menunjukkan sistem mampu memantau parameter debit, kekeruhan, pH, NH₃-N, dan COD secara *real-time*, dengan tingkat keberhasilan 100% pada 16 skenario pengujian *black box*. Pengujian performa mencatat waktu DOMContentLoaded <250 ms, penggunaan CPU Android <10%, dan konsumsi memori 20–31 MB. Entropi kunci AES-128 setara ketahanan teoretis $\pm 10^{21}$ tahun terhadap serangan *brute force*. Notifikasi internal dan WhatsApp terbukti efektif memberikan peringatan saat parameter melampaui ambang batas. UAT menunjukkan tingkat penerimaan 93,25%, menandakan sistem memenuhi ekspektasi dari aspek antarmuka, dan pengalaman pengguna. Sistem direkomendasikan untuk industri berlimbah cair serta dapat diintegrasikan dengan *machine learning* pada penelitian mendatang guna prediksi kualitas limbah.

Kata kunci: IoT, Limbah cair industri, Aplikasi web, Aplikasi Android

ABSTRACT

Imam Zaenal Abidin and Muhammad Adriano Khairur Rizky Setyawan. “Industrial Wastewater Monitoring System Integrated with Internet of Things and JavaScript Object Notation Web Token Based on Website and Android”, Final Project for Applied Bachelor's Degree in Electrical Engineering, Semarang State Polytechnic, under the supervision of Dr. Amin Suharjono, S.T., M.T. and Roni Aprianoro, S.Tr.T., M.Tr.T., 2025, 112 pages

The increasing need for reliable and secure industrial wastewater monitoring systems represents a strategic challenge in ensuring environmental regulatory compliance and preventing pollution. Most existing systems have not been able to integrate multi-protocol communication, layered security mechanisms, and responsive notification features. This research designs and implements an Internet of Things (IoT)-based wastewater monitoring system with JSON Web Token (JWT), supporting HTTP and MQTT protocols, equipped with AES-128 encryption, HTTPS, and web-based and Android interfaces. The methodology used is Agile-Scrum, covering planning, design, implementation, and testing phases including black box, security, performance, notification, and User Acceptance Testing (UAT). The research results show that the system is capable of monitoring flow rate, turbidity, pH, NH₃-N, and COD parameters in real-time, with a 100% success rate in 16 black box testing scenarios. Performance testing recorded DOMContentLoaded time <250 ms, Android CPU usage <10%, and memory consumption 20–31 MB. AES-128 key entropy provides theoretical resistance of $\pm 10^{21}$ years against brute force attacks. Internal and WhatsApp notifications proved effective in providing alerts when parameters exceed threshold limits. UAT showed an acceptance rate of 93.25%, indicating the system meets expectations in terms of interface and user experience. The system is recommended for wastewater-producing industries and can be integrated with machine learning in future research for wastewater quality prediction.

Keywords: IoT, Industrial wastewater, Web application, Android application

DAFTAR ISI

PERNYATAAN KEASLIAN TUGAS AKHIR	ii
HALAMAN PERSETUJUAN	iii
HALAMAN PENGESAHAN	iv
PRAKATA	v
ABSTRAK	vii
<i>ABSTRACT</i>	viii
DAFTAR ISI	ix
DAFTAR GAMBAR	xii
DAFTAR TABEL	xiv
DAFTAR LAMPIRAN	xv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan	3
1.4 Manfaat	3
1.5 Batasan Masalah	4
1.6 Sistematika Penulisan	4
BAB II TINJAUAN PUSTAKA DAN DASAR TEORI	6
2.1 Tinjauan Pustaka	6
2.2 Landasan Teori	8
2.2.1 <i>Internet of Things (IoT)</i>	9
2.2.2 <i>Application Programming Interface (API)</i>	11
2.2.3 Protokol Komunikasi	11
2.2.4 Git	12
2.2.5 JavaScript	12
2.2.6 <i>JavaScript Object Notation (JSON)</i>	13
2.2.7 Next.js	13

2.2.8	<i>Database</i>	14
2.2.9	MySQL.....	14
2.2.10	<i>Message Broker</i>	15
2.2.11	<i>Black Box Testing</i>	15
2.2.12	JSON Web Token (JWT).....	15
2.2.13	Advanced Encryption Standard (AES)	17
2.2.14	<i>Entropy pada JWT Secret</i>	18
2.2.15	Serangan <i>Brute Force</i> pada <i>JWT Secret</i>	19
2.2.16	Klasifikasi Performa Pemuatan Website	20
2.2.17	<i>User Acceptance Testing (UAT)</i>	21
2.2.18	<i>Load Testing</i>	23
2.2.19	<i>Application Performance Index (APDEX)</i>	25
BAB III METODOLOGI PENELITIAN.....		27
3.1	Perencanaan.....	27
3.2	Perancangan	29
3.2.1	Arsitektur Sistem.....	29
3.2.2	Diagram <i>Use Case</i>	31
3.2.3	<i>Entity Relationship Diagram (ERD)</i>	31
3.2.4	Diagram Alur.....	33
3.2.5	Tampilan UI pada Aplikasi.....	35
3.3	Pembuatan	39
3.4	Pengujian.....	39
3.4.1	<i>Black Box Testing</i> (Pengujian fungsionalitas).....	39
3.4.2	Pengujian Sistem Keamanan.....	40
3.4.3	Pengujian Notifikasi.....	41
3.4.4	Pengujian Performa	42

3.5	<i>Load Testing</i>	42
3.6	<i>User Acceptance Testing (UAT)</i>	43
3.7	Evaluasi	46
BAB IV HASIL PENGUJIAN DAN ANALISIS		47
4.1	Hasil Pengujian Fungsionalitas Aplikasi (<i>Black Box Testing</i>)	47
4.2	Hasil Pengujian Sistem Keamanan	65
4.3	Hasil Pengujian Notifikasi	76
4.4	Hasil Pengujian Performa	77
4.5	<i>Load Testing</i>	84
4.6	<i>User Acceptance Testing (UAT)</i>	88
4.7	Evaluasi	90
BAB V PENUTUP.....		93
5.1	Kesimpulan	93
5.2	Saran.....	94
DAFTAR PUSTAKA.....		95
LAMPIRAN.....		100

DAFTAR GAMBAR

Gambar 2.1	Arsitektur IoT	9
Gambar 3.1	Metode Agile-Scrum	27
Gambar 3.2	Studi pustaka	28
Gambar 3.3	Arsitektur sistem secara keseluruhan	30
Gambar 3.4	Arsitektur sistem pada tugas akhir	31
Gambar 3.5	<i>Use Case Diagram</i>	31
Gambar 3.6	<i>Entity Relationship Diagram</i>	33
Gambar 3.7	Diagram alur autentikasi dan otorisasi pengguna	34
Gambar 3.8	Diagram alur penggunaan antarmuka aplikasi.....	34
Gambar 3.9	Diagram alur <i>node</i>	35
Gambar 4.1	Diagram hasil pengujian <i>Black Box</i>	47
Gambar 4.2	Daftar panduan pengguna baru	48
Gambar 4.3	<i>Login</i> gagal.....	49
Gambar 4.4	<i>Login</i> gagal menggunakan akun Google.....	50
Gambar 4.5	Indikator kolom kosong	50
Gambar 4.6	Halaman <i>dashboard</i> dengan <i>filter</i> berdasarkan jumlah data	51
Gambar 4.7	Halaman <i>dashboard</i> dengan <i>filter</i> berdasarkan waktu	51
Gambar 4.8	Halaman <i>devices</i>	52
Gambar 4.9	Formulir tambah <i>device</i>	53
Gambar 4.10	Formulir tambah <i>device</i> ketika memilih protokol MQTT.....	54
Gambar 4.11	Indikator berhasil menambahkan perangkat	54
Gambar 4.12	Formulir <i>edit device</i>	55
Gambar 4.13	Indikator berhasil mengubah data perangkat	55
Gambar 4.14	Peringatan persetujuan hapus <i>device</i>	56
Gambar 4.15	Indikator berhasil menghapus perangkat	56
Gambar 4.16	Halaman <i>datastreams</i>	57
Gambar 4.17	Formulir tambah <i>datastream</i>	57
Gambar 4.18	Indikator berhasil menambahkan <i>datastream</i>	58
Gambar 4.19	Formulir <i>edit datastream</i>	58
Gambar 4.20	Indikator berhasil mengubah <i>datastream</i>	58
Gambar 4.21	Peringatan persetujuan hapus <i>datastream</i>	59

Gambar 4.22	Indikator berhasil hapus <i>datastream</i>	59
Gambar 4.23	Mode <i>edit</i> halaman <i>dashboard</i>	60
Gambar 4.24	Formulir pengaturan grafik (<i>widget</i>)	60
Gambar 4.25	Indikator berhasil menyimpan <i>dashboard</i>	61
Gambar 4.26	Formulir <i>edit</i> grafik (<i>widget</i>)	61
Gambar 4.27	Formulir tambah alarm	63
Gambar 4.28	Indikator berhasil menambahkan alarm	63
Gambar 4.29	Indikator berhasil mengubah alarm	63
Gambar 4.30	Formulis <i>edit</i> alarm	64
Gambar 4.31	Peringatan persetujuan hapus alarm	64
Gambar 4.32	Indikator berhasil hapus alarm	64
Gambar 4.33	<i>Payload</i> HTTP pada Aplikasi Wireshark	68
Gambar 4.34	<i>Payload</i> HTTPS pada Aplikasi Wireshark	69
Gambar 4.35	Proses pemeriksaan <i>payload</i> HTTP	70
Gambar 4.36	Proses <i>Decode</i> Token JWT pada <i>Website</i> Jwt.io	70
Gambar 4.37	Proses pemeriksaan <i>payload</i> MQTT	71
Gambar 4.38	Tampilan Menu <i>Cookie</i> pada <i>Browser</i> Pengguna	73
Gambar 4.39	Hasil pengujian performa <i>website</i>	78
Gambar 4.40	Hasil dari “ <i>Network Monitor</i> ” halaman <i>login</i>	78
Gambar 4.41	Hasil dari “ <i>Network Monitor</i> ” halaman <i>dashboards</i>	79
Gambar 4.42	Hasil dari “ <i>Network Monitor</i> ” halaman <i>devices</i>	80
Gambar 4.43	Hasil dari “ <i>Network Monitor</i> ” halaman <i>datastreams</i>	81
Gambar 4.44	Hasil dari “ <i>Network Monitor</i> ” halaman <i>alarms</i>	82
Gambar 4.45	Hasil pengujian <i>Android Profiler</i> dari Android Studio	83
Gambar 4.46	Hasil APDEX terhadap 10 pengguna	85
Gambar 4.47	Hasil APDEX terhadap 25 pengguna	85
Gambar 4.48	Hasil APDEX terhadap 50 pengguna	86
Gambar 4.49	Hasil APDEX terhadap 100 pengguna	87
Gambar 4.50	Hasil APDEX terhadap 150 pengguna	87
Gambar 4.51	Hasil rata-rata tiap pernyataan	89

DAFTAR TABEL

Tabel 2.1	Ringkasan Tinjauan Pustaka	8
Tabel 2.2	Tingkat kekuatan untuk setiap rentang entropi	19
Tabel 2.3	Tabel Klasifikasi Performa Pemuatan <i>Website</i>	20
Tabel 2.4	Kategori skor skala Likert pada kuesioner UAT	22
Tabel 2.5	Interpretasi persentase tingkat penerimaan pengguna.....	23
Tabel 2.6	Kriteria interpretasi hasil load testing	25
Tabel 2.7	Kategori interpretasi skor APDEX.....	26
Tabel 3.1	Rancangan tampilan antarmuka aplikasi <i>website</i>	36
Tabel 3.2	Rancangan tampilan antarmuka aplikasi Android.....	37
Tabel 3.3	Rancangan pengujian fungsionalitas aplikasi <i>website</i> dan Android .	39
Tabel 3.4	Rancangan pengujian sistem keamanan	41
Tabel 3.5	Rancangan pengujian notifikasi	42
Tabel 3.6	Rancangan pengujian performa aplikasi <i>website</i>	42
Tabel 3.7	Rancangan pengujian <i>Load Testing</i>	43
Tabel 3.8	Daftar pernyataan kuesioner pengujian UAT	44
Tabel 4.1	Hasil pengujian sistem keamanan	65
Tabel 4.2	Hasil pengujian notifikasi	76
Tabel 4.3	Komparasi hasil penelitian	90

DAFTAR LAMPIRAN

Lampiran 1	Halaman <i>Landing Page</i> Aplikasi <i>Website</i>	100
Lampiran 2	Halaman <i>Login</i> Aplikasi <i>Website</i> dan Android	100
Lampiran 3	Pengaturan Akun Aplikasi <i>Website</i>	100
Lampiran 4	Riwayat Notifikasi Aplikasi <i>Website</i>	101
Lampiran 5	Fitur Ekspor.....	101
Lampiran 6	Hasil Ekspor CSV Data Perangkat.....	101
Lampiran 7	Hasil Ekspor PDF Data Perangkat	102
Lampiran 8	Hasil Ekspor CSV Riwayat Notifikasi	102
Lampiran 9	Hasil Ekspor PDF Riwayat Notifikasi	103
Lampiran 10	Hasil <i>Speedtest</i> Jaringan.....	103
Lampiran 11	Halaman dan Mode <i>Edit Dashboard</i> Aplikasi Android	104
Lampiran 12	Fitur Tambah dan <i>Filter Dashboard</i> Aplikasi Android	104
Lampiran 13	Halaman dan Fitur Tambah <i>Devices</i> Aplikasi Android.....	104
Lampiran 14	Halaman dan Fitur Tambah <i>Datastream</i> Aplikasi Android.....	105
Lampiran 15	Halaman dan Fitur Tambah <i>Alarm</i> Aplikasi Android.....	105
Lampiran 16	Hasil <i>Load Test</i> Aplikasi JMeter	105
Lampiran 17	Sensor PH <i>Meter Kit</i> V2 SKU SEN0161-V2.....	106
Lampiran 18	Datasheet Sensor PH <i>Meter Kit</i> V2 SKU SEN0161-V2.....	106
Lampiran 19	Sensor UV254 COD.....	107
Lampiran 20	Datasheet UV254 COD.....	107
Lampiran 21	Sensor DSN260 <i>Electronic Nitrate</i>	108
Lampiran 22	Datasheet Sensor DSN260 <i>Electronic Nitrate</i>	108
Lampiran 23	Sensor <i>Turbidity Meter</i> SKU SEN0189	109
Lampiran 24	Datasheet Sensor <i>Turbidity Meter</i> SKU SEN0189	109
Lampiran 25	Sensor OF05ZAT <i>Liquid Flow Sensor</i>	110
Lampiran 26	Datasheet Sensor OF05ZAT <i>Liquid Flow Sensor</i>	110
Lampiran 27	<i>Hardware</i> atau Alat Pengujian Sistem Pemantauan Limbah ...	111
Lampiran 28	Hasil Pembacaan Sensor	111
Lampiran 29	Hasil Pembacaan Sensor (Setelah penambahan HCL).....	111
Lampiran 30	<i>Source Code</i> Aplikasi	112

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemantauan adalah kegiatan pengumpulan data secara terstruktur dan terus-menerus untuk memantau suatu kondisi, parameter, atau proses tertentu (Gonzales dkk., 2021). Dalam konteks pengelolaan lingkungan, sistem pemantauan dirancang untuk memastikan kepatuhan terhadap regulasi, mendeteksi anomali, serta memberikan data yang akurat untuk pengambilan keputusan (Nurbaya, 2018). Sistem pemantauan terdiri dari perangkat keras, seperti sensor dan data *logger*, serta perangkat lunak yang mengolah data menjadi informasi yang dapat dianalisis. Salah satu contoh penerapan dari sistem pemantauan adalah pada pemantauan limbah cair industri.

Peraturan regulasi terkait pemantauan limbah cair industri telah ditetapkan oleh Kementerian Lingkungan Hidup dan Kehutanan (KLHK) dalam Peraturan Nomor P.80/MENLHK/SETJEN/KUM.1/10/2019 terkait pemantauan kualitas air limbah. Regulasi ini mewajibkan tindakan pemantauan kualitas air limbah secara berkala pada industri dengan menggunakan Sistem Pemantauan Kualitas Air Limbah secara Terus Menerus dan Dalam Jaringan (SPARING). SPARING merupakan instrumen pemantauan pencemaran lingkungan hidup dan pemenuhan baku mutu air limbah. SPARING terdiri dari alat pemantauan, data *logger*, dan pusat data yang mengolah informasi hasil pemantauan. Adapun parameter air limbah yang dipantau yaitu *Potential of Hydrogen* (pH), *Chemical Oxygen Demand* (COD), *Total Suspended Solid* (TSS), amonia nitrogen (NH₃-N), dan debit (Nurbaya, 2019). Untuk memastikan kepatuhan terhadap regulasi SPARING yang ditetapkan oleh KLHK, teknologi pemantauan terus berkembang seiring dengan kemajuan digital.

Di era Revolusi Industri 4.0, *Internet of Things* (IoT) kerap digunakan agar pemantauan menjadi lebih efektif. IoT memungkinkan perangkat pemantauan seperti sensor untuk berkomunikasi satu sama lain melalui jaringan internet, sehingga dapat mempermudah pengumpulan data secara otomatis (Sawitri, 2023).

Seiring meningkatnya jumlah perangkat IoT yang tersebar, tantangan baru muncul dalam pembuatan sistem pemantauan IoT yang efektif dan efisien (Sari, 2024).

Sebagian besar sistem pemantauan IoT yang sudah ada hanya menyediakan pemantauan pasif, tanpa adanya peringatan atau notifikasi otomatis saat parameter lingkungan melebihi ambang batas aman (Gonzales dkk., 2021). Selain itu, pemantauan hanya dapat dilakukan melalui *website*, sehingga menyulitkan pemantauan oleh pengguna dengan perangkat mobile (Akasiadis dkk., 2019). Kemudian dalam hal interoperabilitas, protokol komunikasi menjadi aspek yang krusial pada sebuah perangkat dalam sebuah sistem IoT. Hal ini mengakibatkan perangkat yang menggunakan protokol yang berbeda sulit untuk dikelola dalam satu ekosistem (Pramukantoro, 2020). Tantangan lain yang berhubungan erat dengan interoperabilitas adalah masalah keamanan. Dengan semakin banyaknya perangkat yang terhubung ke internet, keamanan data dalam sebuah sistem IoT akan semakin rentan terhadap resiko serangan siber, pencurian data, dan eksploitasi perangkat (Shakdher dkk., 2019).

Masalah keamanan pada sistem IoT merupakan masalah serius yang harus ditangani. Untuk mengatasi masalah tersebut, diperlukan sebuah mekanisme keamanan agar dapat melindungi integritas dan autentikasi data yang dikirim dan diterima oleh perangkat IoT. Salah satu mekanisme keamanan yang digunakan adalah JavaScript *Object Notation Web* Token (JWT) (Ahmed dkk., 2019). JWT merupakan standar token yang aman dan banyak digunakan dalam aplikasi web. Penerapan JWT memastikan verifikasi identitas pengguna atau perangkat lebih efisien dan aman, serta dapat mengurangi beban *server* sehingga meningkatkan skalabilitas sistem. Selain itu, JWT juga dapat memastikan otorisasi kepada pengguna atau perangkat untuk mengakses dan mengendalikan perangkat IoT (Darmawan dkk., 2023).

Hingga saat ini telah banyak penelitian yang merancang dan membuat sebuah alat IoT untuk memantau limbah cair, namun belum ada sistem pemantauan air limbah industri yang dapat memantau dan meningkatkan respons terhadap kondisi kritis

dari parameter sensor secara real-time, serta dapat mengatasi masalah terkait interoperabilitas dan keamanan. Oleh karena itu, tugas akhir ini bertujuan untuk merancang dan mengimplementasikan sistem pemantauan limbah cair industri yang mudah dan aman. Maka penulis merancang penelitian dengan judul “Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript *Object Notation Web Token* Berbasis *Website* dan Android”, yang dapat memberikan solusi efektif dalam mengatasi permasalahan pengelolaan IoT tersebut.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, dapat dirumuskan permasalahannya yaitu:

1. Keterbatasan sistem pemantauan limbah cair industri dalam hal aksesibilitas dan kemudahan penggunaan secara mandiri oleh pengguna.
2. Belum diterapkannya sistem pemantauan limbah cair industri yang dapat mendukung lebih dari satu protokol komunikasi.
3. Belum diterapkannya sistem keamanan pada sistem pemantauan limbah cair industri, yang berpotensi menyebabkan perubahan pada data secara ilegal.
4. Perlu adanya fitur notifikasi untuk memberikan peringatan kepada pengguna.

1.3 Tujuan

Tujuan dari pembuatan tugas akhir ini adalah:

1. Membuat antarmuka pemantauan berbasis *website* dan Android.
2. Menerapkan integrasi lebih dari satu protokol komunikasi.
3. Menerapkan sistem keamanan menggunakan JWT, AES dan HTTPS.
4. Menerapkan fitur notifikasi perangkat IoT untuk pengguna.

1.4 Manfaat

Manfaat setelah pembuatan tugas akhir adalah:

1. Akan mempermudah penyampaian informasi terhadap situasi darurat kepada tim teknis atau operator jika parameter limbah di luar batas aman, sehingga kondisi yang memerlukan tindakan cepat dapat segera terdeteksi, seperti halnya penyesuaian pada aliran limbah.

2. Akan mempermudah pengembangan sistem IoT secara berkelanjutan, seperti penambahan perangkat IoT baru tanpa merubah infrastruktur yang sudah ada.
3. Akan meningkatkan keamanan data, sehingga kerahasiaan data terkait parameter limbah industri maupun data pribadi pengguna akan terjaga.
4. Akan mendukung kepatuhan terhadap regulasi lingkungan terkait standar emisi yang sudah ditetapkan, sehingga dapat mengurangi risiko pelanggaran dan turut serta menjaga kelestarian lingkungan.

1.5 Batasan Masalah

Berdasarkan latar belakang, maka batasan masalahnya adalah:

1. Pembahasan akan berfokus pada pembuatan aplikasi pemantauan IoT.
2. Tidak membahas pembuatan alat pemantauan IoT.
3. Mikrokontroler yang akan digunakan adalah ESP32.
4. Parameter air yang akan diukur adalah pH, COD, kekeruhan, amonia dan debit atau aliran.
5. Protokol komunikasi yang akan digunakan adalah HTTP dan MQTT.
6. Fitur yang akan dibuat adalah autentikasi dan otorisasi pengguna, grafik visualisasi data, manajemen perangkat IoT, serta alarm notifikasi.

1.6 Sistematika Penulisan

Dalam penyusunan tugas akhir ini dilakukan pengelompokkan isi dalam beberapa bab. Bagian yang dapat berdiri sendiri dipisahkan dengan bagian yang lain dan ditempatkan dalam bab tersendiri dengan maksud mempermudah pemahaman. Adapun sistematika penulisannya adalah sebagai berikut:

BAB I PENDAHULUAN

Bab ini membahas mengenai latar belakang dari tugas akhir ini, perumusan masalah pada tugas akhir, tujuan dari tugas akhir, manfaat yang bisa dipetik dari tugas akhir, pembatasan masalah dari tugas akhir, dan sistematika penulisan pada tugas akhir.

BAB II TINJAUAN PUSTAKA DAN DASAR TEORI

Bab ini membahas mengenai tinjauan pustaka yang merupakan referensi dalam pembuatan tugas akhir ini dan dasar teori yang mendukung pembuatan tugas akhir.

BAB III METODOLOGI PENELITIAN

Bab ini membahas mengenai proses perencanaan, perancangan hingga pengujian dari pembuatan perangkat lunak (*Web* dan *Android*) untuk sistem pemantauan limbah cair industri.

BAB IV HASIL PENGUJIAN DAN ANALISIS

Bab ini membahas mengenai hasil pengujian sistem beserta analisi data hasil pengujian dari pembuatan perangkat lunak (*Web* dan *Android*) dari sistem pemantauan limbah cair industri.

BAB V PENUTUP

Bab ini berisi tentang kesimpulan dan saran yang diperoleh dari analisis yang telah dilakukan pada bab sebelumnya.

DAFTAR PUSTAKA

Daftar pustaka berisi sumber-sumber, jurnal, studi Pustaka yang penulis cantumkan dalam proses penyelesaian tugas akhir ini.

LAMPIRAN

Lampiran berisi data atau pelengkap atau hasil olahan yang menunjang penulisan laporan tugas akhir tetapi tidak dicantumkan di dalam isi tugas akhir, karena akan mengganggu kesinambungan penulisan.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Pada penyusunan tugas akhir ini, penulis mengkaji berbagai penelitian terdahulu yang berkaitan dengan sistem pemantauan berbasis IoT beserta permasalahan yang menyertainya. Salah satu isu utama yang sering diangkat adalah keamanan protokol komunikasi IoT. Shakhder dkk. (2019) dalam penelitiannya yang berjudul *“Security Vulnerabilities in Consumer IoT Applications”* menemukan bahwa beberapa perangkat IoT masih mengirimkan data dalam format teks biasa (*plaintext*), sehingga rawan terhadap serangan *Man In The Middle* (MITM). Dengan menggunakan aplikasi Wireshark untuk menganalisis lalu lintas data, peneliti menekankan pentingnya penambahan fitur keamanan seperti hashing untuk melindungi data dari serangan siber.

Menanggapi masalah tersebut, Shodiq dkk. (2021) melalui penelitian berjudul *“Secure MQTT Authentication and Message Exchange Methods for IoT Constrained Device”* mengusulkan mekanisme autentikasi berbasis JWT pada protokol MQTT. Tujuannya adalah untuk mencegah akses ilegal ke jaringan sekaligus meningkatkan keamanan data menggunakan enkripsi XXTEA. Hasil pengujian menunjukkan mekanisme ini efektif melawan serangan *brute force* dan MITM, serta tetap kompatibel dengan perangkat berdaya rendah. Secara kuantitatif, implementasi mekanisme JWT dengan panjang *secret* yang memiliki entropi sekitar 64 bit memberikan ruang kunci sebesar 2^{64} kemungkinan kombinasi. Dengan asumsi laju serangan *brute force* sebesar 10^6 tebakan per detik, maka dibutuhkan waktu kurang lebih 10^{10} tahun untuk menebak dengan benar. Nilai ini menunjukkan bahwa tingkat keamanan yang diperoleh tidak hanya bergantung pada panjang kunci, tetapi juga pada distribusi acak karakter penyusunnya.

Penelitian di bidang pemantauan kualitas air juga telah banyak dilakukan. Bogdan dkk. (2023) dalam penelitiannya *“Low-Cost Internet-of-Things Water-Quality Monitoring System for Rural Areas”* mengembangkan sistem berbasis Arduino UNO dengan sensor untuk mengukur suhu, pH, *Total Dissolved Solid* (TDS), dan

kekeruhan (*turbidity*). Data dikirimkan melalui konektivitas Bluetooth ke aplikasi Android untuk pemantauan *real-time*. Meskipun bermanfaat, sistem ini memiliki keterbatasan jarak komunikasi.

Raman & Martin (2024) melalui penelitian “*IoT-Enabled Water Pollution Detection for Real-Time Monitoring and Pollution Source Identification with MQTT Protocol*” membangun platform IoT berbasis protokol MQTT yang dapat mengirimkan data sensor secara *real-time*, seperti komposisi kimia, pH, *Dissolved Oxygen* (DO), dan NH₃-N. Sistem ini memudahkan identifikasi sumber pencemaran secara cepat sehingga tindakan pencegahan dapat dilakukan lebih efektif.

Di sisi lain, penelitian terkait pemantauan limbah cair juga berkembang pesat. Salem dkk. (2022) dalam penelitiannya “*An Industrial Cloud-Based IoT System for Real-Time Monitoring and Controlling of Wastewater*” menggunakan ESP8266 untuk memantau pH dan suhu limbah cair, dengan data dikirimkan ke platform ThingSpeak dan dilengkapi notifikasi SMS. Namun, metode ini belum menerapkan keamanan data berbasis *hashing*.

Febriana & Farros (2024) dalam penelitian “*Rancang Bangun Industrial Internet of Things Board Multiguna untuk Monitoring Limbah Cair*” merancang board IoT multiguna dengan konektivitas LoRa untuk memantau pH, COD, TSS, NH₃-N, dan debit air. Data disimpan pada kartu SD dan ditampilkan di Nxtion *Display*, serta dapat dipantau melalui ThingSpeak. Meski demikian, sistem ini belum memiliki platform pemantauan berbasis *mobile* dan layanan notifikasi untuk memberi tahu perubahan signifikan pada nilai sensor.

Berdasarkan penelitian-penelitian tersebut, terlihat bahwa sebagian besar sistem yang ada memiliki kemiripan dengan tugas akhir ini, tetapi masih menyisakan kelemahan seperti keterbatasan dukungan protokol komunikasi, kurangnya sistem keamanan data, dan ketiadaan fitur notifikasi. Pada penelitian ini, keunggulan yang ditawarkan adalah penerapan *message broker* untuk integrasi protokol MQTT, penggunaan enkripsi HTTPS, serta *hashing* JWT guna menjaga keamanan dan integritas data. Selain itu, sistem juga dilengkapi notifikasi melalui *browser* dan

WhatsApp agar pengguna dapat segera mengetahui adanya anomali pada nilai sensor. Tabel 2.1 merupakan tabel yang berisi ringkasan tinjauan pustaka yang telah dijelaskan di atas.

Tabel 2.1 Ringkasan Tinjauan Pustaka

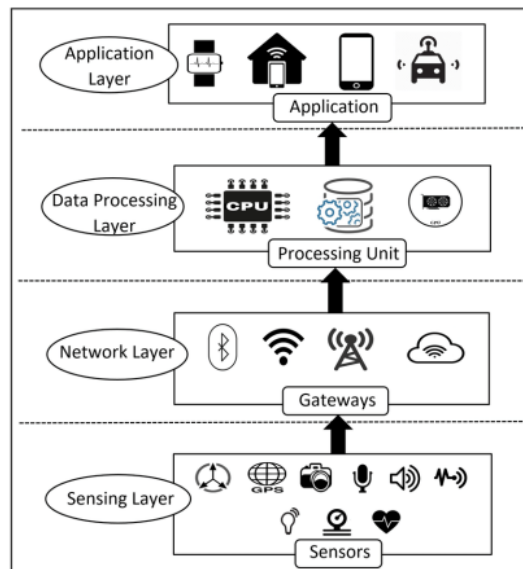
Judul Penelitian	Sensor	Antarmuka	Protokol	Notifikasi	Keamanan
(Febriana & Farros, 2024)	COD, pH, TSS, NH3-N, Debit air	<i>Website</i>	HTTP	-	-
(Raman & Martin, 2024)	NH3-N, DO, pH, COD	<i>Website</i>	MQTT	-	-
(Salem dkk., 2022)	Suhu air, pH	<i>Website</i>	HTTP	SMS	-
(Bogdan dkk., 2023)	Suhu air, pH, TDS, Kekeruhan	<i>Android</i>	HTTP	<i>Built-In</i> (Bawaan)	-
(Shakdher dkk., 2019)	-	<i>Website</i>	HTTP	-	-
(Shodiq dkk., 2021)	-	<i>Website</i>	MQTT	-	JWT, Enkripsi XXTEA
Usulan Tugas Akhir	COD, pH, Debit air, NH3-N, Kekeruhan	<i>Website, Android</i>	HTTP, MQTT	<i>Browser</i> (Bawaan), WhatsApp	JWT, Enkripsi AES, HTTPS

2.2 Landasan Teori

Pada bagian ini menjelaskan mengenai teori-teori yang digunakan dalam pembuatan tugas akhir yang berjudul "Sistem Pemantauan Limbah Cair Industri Terintegrasi *Internet of Things* dan JavaScript *Object Notation Web Token* Berbasis *Website* dan *Android*".

2.2.1 Internet of Things (IoT)

IoT adalah paradigma teknologi yang menghubungkan berbagai perangkat fisik, seperti sensor, aktuator, dan mikrokontroler ke dalam jaringan internet untuk memungkinkan komunikasi data secara otomatis dan *real-time* (Shakdher dkk., 2019). IoT memanfaatkan kombinasi teknologi perangkat keras (seperti sensor dan mikrokontroler), protokol komunikasi, serta sistem *cloud* atau *edge computing* guna menciptakan sistem yang cerdas, responsif, dan adaptif terhadap lingkungan sekitarnya. Dalam arsitektur sistem pemantauan limbah cair industri, IoT berperan sebagai fondasi utama yang memungkinkan pengawasan kualitas air secara berkelanjutan dengan tingkat akurasi tinggi. Sistem ini terdiri dari beberapa lapisan yang saling terintegrasi. Gambar 2.1 merupakan ilustrasi dari beberapa lapisan tersebut.



Gambar 2.1 Arsitektur IoT

(Sumber: Dokumentasi Pribadi)

Setiap lapisan memiliki tanggung jawab spesifik dalam proses akuisisi data, transmisi informasi, hingga penyajian layanan kepada pengguna akhir. Untuk memperjelas fungsi dan peran masing-masing komponen dalam sistem, penjelasan berikut disusun berdasarkan pembagian umum lapisan IoT.

1. *Sensing Layer*

Lapisan ini merupakan fondasi dari sistem IoT yang terdiri dari berbagai sensor dan aktuator yang mengumpulkan data dari lingkungan fisik. Lapisan ini terdiri

atas sensor-sensor fisis seperti sensor pH, suhu, cahaya, tekanan, suara, dan sebagainya. Sensor ini dikendalikan oleh mikrokontroler dan bertugas mengakuisisi data secara periodik. Data yang diperoleh bersifat mentah dan perlu dikirimkan ke lapisan berikutnya untuk diproses lebih lanjut.

2. *Network Layer*

Lapisan ini bertanggung jawab atas transmisi data dari *sensing layer* ke *data processing layer*. Lapisan ini terdiri dari berbagai macam perangkat jaringan yang saling terhubung agar dapat menunjang komunikasi data antar jaringan lokal maupun internet.

3. *Data Processing Layer*

Pada lapisan ini, data yang diterima dari perangkat IoT akan diproses, disimpan, dan dianalisis. Komponen utama dalam layer ini adalah *server*, yang dapat berupa *cloud-based server* atau *on-premise server*. Teknologi seperti *edge computing*, *machine learning*, atau sistem basis data digunakan untuk menghasilkan informasi yang lebih bermakna dari data mentah yang dikumpulkan sensor.

4. *Application Layer*

Lapisan ini merupakan antarmuka pengguna berupa aplikasi berbasis *website*, *mobile*, atau perangkat pintar yang menampilkan data hasil pemrosesan secara *real-time* dan historis. Melalui lapisan ini, pengguna dapat berinteraksi secara langsung dengan sistem.

Karakteristik kunci dari implementasi IoT dalam sistem pemantauan industri meliputi skalabilitas, keandalan, interoperabilitas, dan keamanan (Mirani dkk., 2022). Skalabilitas memungkinkan sistem untuk dikembangkan dengan menambahkan sensor atau perangkat baru tanpa perlu merombak keseluruhan infrastruktur yang telah ada, menjadikannya fleksibel untuk kebutuhan jangka panjang. Keandalan sangat penting agar data dari sensor dapat dikirim dan diterima secara konsisten tanpa kehilangan informasi. Interoperabilitas memungkinkan integrasi berbagai perangkat dan protokol komunikasi dari *vendor* yang berbeda dalam satu sistem yang terkoordinasi. Aspek keamanan menjadi sangat krusial untuk mencegah akses tidak sah dan manipulasi data, yang dicapai melalui mekanisme enkripsi dan autentikasi yang kuat, seperti penerapan JWT dalam proses

komunikasi data antara *klien* dan *server*. JWT memastikan bahwa hanya perangkat dan pengguna yang sah yang dapat mengakses atau mengirimkan data ke server. Token ini dihasilkan saat autentikasi dan disertakan dalam setiap permintaan API atau publikasi data dari perangkat (Shodiq dkk., 2021).

2.2.2 Application Programming Interface (API)

API merupakan antarmuka yang menyediakan seperangkat fungsi dan prosedur yang memungkinkan suatu perangkat lunak berinteraksi dengan perangkat lunak lainnya secara terstruktur. API berfungsi sebagai perantara komunikasi antara program aplikasi dengan sistem lain, seperti sistem operasi, sistem manajemen basis data (DBMS), atau layanan berbasis protokol komunikasi tertentu. Implementasi API dilakukan melalui pemanggilan fungsi-fungsi tertentu dalam kode program untuk menjalankan tugas-tugas spesifik. Dalam konteks pengembangan sistem terdistribusi, API memegang peranan penting karena memungkinkan pertukaran data yang efisien dan terstandar melalui protokol komunikasi seperti HTTP, dengan metode umum seperti GET, POST, PUT, dan DELETE (Mudassir & Mushtaq, 2024). Dalam tugas akhir ini, API digunakan untuk menjembatani komunikasi antara perangkat IoT dengan platform pemantauan. Melalui API, data dari sensor dapat dikirim ke *server* secara *real-time* dan disajikan kepada pengguna dalam bentuk yang informatif dan mudah diakses. (Kim dkk., 2021).

2.2.3 Protokol Komunikasi

Protokol komunikasi adalah aturan dan standar yang digunakan oleh berbagai macam perangkat pada jaringan lokal untuk menukar informasi lewat internet dengan efisien. Dalam tugas akhir ini, terdapat dua protokol yang akan digunakan diantaranya:

1. *Hypertext Transfer Protocol* (HTTP): protokol komunikasi berbasis teks yang dirancang untuk mentransfer data antara perangkat IoT dan server. HTTP umumnya digunakan untuk mengakses dan mengirimkan data ke server melalui *Representational State Transfer* (REST) API (Ahmad dkk., 2022). HTTP bekerja dengan pendekatan *request-response*, di mana *client*

mengirimkan permintaan atau *request* dan *server* merespons dengan informasi yang diminta (Shakdher dkk., 2019).

2. *Message Queuing Telemetry Transport* (MQTT): Protokol ini digunakan untuk mengirimkan pesan berbasis *publisher/subscriber* (Pub/Sub) antara perangkat IoT dan *server*. Protokol ini sangat cocok untuk digunakan dalam sistem IoT yang memiliki *bandwidth* yang terbatas dan koneksi yang tidak stabil (Raman & Martin, 2024).
3. *Hypertext Transfer Protocol Secure* (HTTPS): merupakan versi aman dari protokol HTTP yang digunakan untuk mengirimkan data antara *client* dan *server* melalui *internet*. HTTPS mengintegrasikan lapisan keamanan *Transport Layer Security* (TLS) atau *Secure Sockets Layer* (SSL) untuk memberikan tiga elemen utama keamanan, yaitu enkripsi data, integritas, dan autentikasi. Enkripsi memastikan bahwa data yang dikirimkan tidak dapat diakses oleh pihak yang tidak berwenang selama data dikirimkan. Integritas menjamin bahwa data tidak diubah selama transmisi, sementara autentikasi memungkinkan verifikasi identitas *server* sehingga *client* dapat memastikan bahwa mereka terhubung ke sumber yang sah (Shah & Correia, 2021).

2.2.4 Git

Git adalah sebuah perangkat lunak yang berfungsi untuk melacak perubahan dari suatu proyek. Git dirancang untuk mendukung kolaborasi tim dalam mengembangkan proyek secara bersamaan tanpa konflik menggunakan GitHub, yang merupakan layanan Git berbasis *cloud* (Loeliger & McCullough, 2012).

2.2.5 JavaScript

JavaScript adalah bahasa pemrograman yang awalnya dirancang untuk membuat halaman web menjadi lebih interaktif. Namun, dengan perkembangan teknologi, JavaScript kini digunakan untuk berbagai aplikasi, termasuk pengembangan *back-end*, aplikasi *desktop*, hingga aplikasi *mobile*. JavaScript memiliki keunggulan berupa sintaks yang sederhana, fleksibilitas tinggi, dan dukungan komunitas yang luas. Ekosistem JavaScript mencakup pustaka dan framework seperti React JS, Angular, dan Vue JS untuk pengembangan *front-end*, Node JS untuk pengembangan *back-end*, serta React Native, serta Next.js untuk pengembangan *web modern*.

dengan fitur *server-side rendering* (Vercel, 2023). Fitur seperti *event-driven programming* dan asinkronisasi membuat JavaScript sangat efisien untuk menangani operasi *real-time*, seperti *streaming* data atau aplikasi berbasis chat. Bahasa ini juga kompatibel dengan berbagai *browser* dan perangkat, menjadikannya salah satu teknologi paling relevan dalam pengembangan aplikasi modern (Loring dkk., 2015).

2.2.6 JavaScript Object Notation (JSON)

JSON adalah format data ringan yang digunakan untuk pertukaran data antara *server* dan klien (Nurseitov dkk., 2021). JSON menggunakan struktur berbasis objek dengan pasangan *key-value*, membuatnya mudah dibaca oleh manusia dan diproses oleh mesin. JSON banyak digunakan dalam aplikasi *web* karena kompatibilitasnya yang luas dengan berbagai bahasa pemrograman dan efisiensinya dalam serialisasi data (json.org, 2006).

Format JSON digunakan baik dalam komunikasi API maupun dalam penyusunan *payload* pada JWT (Bourhis dkk., 2020). JSON mendukung efisiensi dan kecepatan pertukaran data, yang sangat penting untuk sistem pemantauan *real-time* seperti sistem pemantauan limbah cair. Dalam tugas akhir ini, JSON memfasilitasi integrasi antara mikrokontroler ESP32 sebagai pengirim data dan *server* sebagai penerima data.

2.2.7 Next.js

Next.js adalah *framework open-source* berbasis React yang dirancang untuk membangun aplikasi *web* modern dengan dukungan *server-side rendering* (Vercel, 2023). Next.js menyediakan serangkaian fitur seperti sistem *routing* otomatis, optimasi performa, pemisahan kode, serta dukungan API bawaan yang memudahkan pengembangan *front-end* dan *back-end* dalam satu proyek (nextjs.org, 2024).

Dalam tugas akhir ini, Next.js digunakan sebagai *framework* pembuatan antarmuka *web* untuk sistem pemantauan. *Framework* ini dipilih karena kemampuannya dalam membangun aplikasi yang ringan, responsif, dan optimal untuk kebutuhan pemantauan data secara *real-time*. *Framework* ini juga kompatibel dengan cukup

banyak *library*, salah satunya adalah next-pwa. *Library* next-pwa berfungsi untuk men-*compile server web* Next.js menjadi aplikasi Android berbasis *WebView*, sehingga pengguna dapat memasang versi Android dari sistem pemantauan dengan mudah. (Brahmbhatt & Vora, 2022).

2.2.8 Database

Database atau basis data adalah kumpulan data yang disimpan secara sistematis dan dapat diakses serta dikelola secara elektronik melalui sistem komputer. Tujuan utama dari basis data adalah untuk menyimpan informasi secara terorganisir agar dapat dengan mudah dicari, ditambahkan, diubah, atau dihapus. Terdapat berbagai jenis model *database*, namun yang paling umum digunakan adalah model relasional. Dalam model ini, data disimpan dalam bentuk tabel (relasi) yang terdiri dari baris dan kolom. Masing-masing baris mewakili suatu entitas data, sementara kolom mewakili atribut dari entitas tersebut (Pavlović dkk., 2021).

Structured Query Language (SQL) merupakan bahasa pemrograman standar yang digunakan untuk mengelola *database* relasional. SQL memungkinkan pengguna untuk melakukan berbagai operasi seperti pencarian, penyisipan, pembaruan, dan penghapusan data secara efisien (Lee, 2022). Selain itu, SQL juga mendukung pengelompokan data, pengurutan, dan penggabungan antar tabel (Pavlović dkk., 2021).

2.2.9 MySQL

MySQL adalah salah satu contoh sistem pengelolaan basis data yang dirancang untuk menyimpan dan mengelola data dengan menggunakan SQL. MySQL banyak digunakan dalam pengembangan aplikasi web dan sistem informasi karena kinerjanya yang cepat, stabilitas tinggi, dan kemudahan integrasi dengan berbagai bahasa pemrograman. Dalam pengembangan aplikasi, MySQL sering digunakan untuk menyimpan data pengguna, *log* aktivitas, dan informasi transaksi secara efisien (Pavlović dkk., 2021).

Sebagai sistem manajemen basis data relasional yang bersifat *open-source*, MySQL menyediakan fitur yang mendukung kebutuhan sistem skala kecil hingga besar. Fitur-fitur tersebut meliputi dukungan transaksi dengan prinsip *Atomicity*,

Consistency, Isolation, Durability (ACID) indexing untuk mempercepat pencarian data, serta replikasi dan *clustering* untuk menjaga ketersediaan dan keandalan data. MySQL juga kompatibel dengan berbagai sistem operasi dan memiliki komunitas pengguna serta dokumentasi yang sangat luas (Sarwar & Khan, 2021).

2.2.10 Message Broker

Message broker adalah komponen perangkat lunak yang berfungsi sebagai perantara komunikasi data antar sistem atau perangkat untuk bertukar pesan secara asinkron, tanpa harus terhubung secara langsung satu sama lain secara *real-time*. *Message broker* bekerja dengan cara menerima pesan dari pengirim (*publisher*), kemudian menyimpannya sementara, dan mendistribusikannya ke penerima (*subscriber*) yang terdaftar pada topik atau saluran tertentu. Proses ini dikenal sebagai model *publish-subscribe*. Dengan demikian, sistem menjadi lebih fleksibel karena setiap komponen dapat beroperasi secara independen. Contoh *message broker* yang umum digunakan dalam sistem IoT adalah RabbitMQ, Mosquitto *broker*, dan Apache Kafka. Dalam tugas akhir ini, penulis menggunakan Mosquitto *broker* sebagai *message broker* yang akan dijalankan pada *server*.

2.2.11 Black Box Testing

Black Box Testing adalah metode pengujian perangkat lunak yang berfokus pada evaluasi fungsionalitas aplikasi tanpa melihat struktur internal atau implementasi kode. Pengujian ini dilakukan dengan memberikan *input* tertentu dan memeriksa apakah *output* yang dihasilkan sesuai dengan spesifikasi yang diharapkan. Pengujian ini dilakukan di akhir pembuatan perangkat lunak untuk mengetahui apakah perangkat lunak dapat berfungsi dengan baik. Pendekatan ini sangat efektif untuk mengidentifikasi bug terkait fungsionalitas, kompatibilitas, atau kinerja dari perspektif pengguna, memastikan bahwa aplikasi memenuhi kebutuhan bisnis dan berfungsi sebagaimana mestinya (Ayuningtyas dkk., 2023; Myers dkk., 2011).

2.2.12 JSON Web Token (JWT)

JWT merupakan standar terbuka yang digunakan untuk pertukaran informasi secara aman dalam format objek JSON, terutama dalam konteks autentikasi dan otorisasi pada aplikasi berbasis *web* (Darmawan dkk., 2023). Token JWT terdiri atas tiga komponen utama:

1. *Header* : berisi informasi tipe token (*typ*) dan algoritma yang digunakan untuk menandatangani token (*alg*), misalnya HS256 atau RS256.
2. *Payload* : memuat klaim (*claims*) yang berisi informasi pengguna atau perangkat, seperti *issuer* (iss), *subject* (sub), *audience* (aud), dan *expiration time* (exp).
3. *Signature* : digunakan untuk memverifikasi keaslian token. *Signature* dihasilkan dengan menggabungkan *header* dan *payload* yang telah di-*encode* dalam format Base64URL, kemudian ditandatangani menggunakan algoritma yang sesuai dan kunci rahasia (*secret key*).

Proses verifikasi JWT di sisi server dilakukan dengan cara menghitung kembali nilai *signature* menggunakan *header* dan *payload* yang diterima, lalu membandingkannya dengan *signature* pada token. Token dinyatakan valid jika hasil perhitungan sama persis dengan *signature* yang dikirimkan (Hosenkhan & Pattanayak, 2021). Secara umum, proses pembentukan JWT dapat ditulis dalam bentuk Persamaan (2.1) (Jwt.io, 2024).

$$JWT = HMACSHA256(base64Url(H) + "." + base64Url(P), K) \quad (2.1)$$

dengan:

$HMACSHA256$ = algoritma *hashing*

$Base64Url$ = algoritma *encoding*

H = *header*

P = *payload*

K = kunci *secret*

Pada tugas akhir ini, JWT digunakan untuk mengamankan akses antara pengguna maupun perangkat IoT terhadap sistem. Setiap permintaan ke *server* memerlukan token JWT yang telah ditandatangani dengan *secret key* tertentu. Token juga

memiliki masa berlaku (*expiration time*) untuk membatasi risiko penyalahgunaan jika token jatuh ke pihak yang tidak berwenang.

2.2.13 Advanced Encryption Standard (AES)

AES merupakan salah satu algoritma kriptografi simetris yang banyak digunakan untuk melindungi data sensitif karena tingkat keamanannya yang tinggi dan efisiensinya dalam proses enkripsi maupun dekripsi. Algoritma AES bekerja dengan membagi data menjadi blok-blok dengan ukuran tertentu dan melakukan serangkaian transformasi yang melibatkan operasi aritmetika berbasis Galois Field (2^8) (Shah & Correia, 2021). Penggunaan AES memberikan tingkat keamanan yang memadai apabila nilai dari kunci dipilih secara acak dan tidak dapat diprediksi. Namun, keamanan ini sangat bergantung pada kerahasiaan kunci. Oleh karena itu, pemilihan panjang kunci yang tepat, serta penyimpanan yang aman menjadi aspek penting dalam penerapan algoritma ini pada sistem.

Pada tugas akhir ini digunakan AES dengan panjang kunci 128-bit (*AES-128*) dan mode operasi *Cipher Block Chaining* (CBC). Proses dekripsi AES-128-CBC dapat dijelaskan sebagai kebalikan dari proses enkripsi, di mana setiap blok *ciphertext* (C_i) diubah kembali menjadi *plaintext* (P_i) menggunakan kunci *secret* (K) dan *Initialization Vector* (IV). Secara umum, proses dekripsi satu blok pada mode CBC dapat dihitung menggunakan Persamaan (2.2).

$$P_i = D(C_i, K) \oplus C_{i-1} \quad \dots\dots\dots (2.2)$$

dengan:

P_i = blok *plaintext* ke-i

C_i = blok *ciphertext* ke-i

D = fungsi dekripsi AES

K = kunci *secret*

$\oplus$ = operasi XOR (*exclusive OR*)

C_{i-1} = blok *ciphertext* sebelumnya (untuk blok pertama digantikan oleh IV)

Pada implementasi tugas akhir ini, data terenkripsi disimpan dalam format Base64, kemudian diubah menjadi *buffer* biner. Enkripsi dilakukan dengan menyisipkan IV sepanjang 16 *byte* pada bagian awal *ciphertext*, sehingga proses dekripsi dapat dilakukan tanpa harus menyimpan IV secara terpisah. *Secret key* diberikan dalam bentuk *hexadecimal* dan diambil 16 *byte* pertama untuk memenuhi ukuran kunci AES-128 (128 bit = 16 *byte*). Hasil dari dekripsi yang masih berbentuk *hexadecimal*, kemudian dikonversi ke format *American Standard Code for Information Interchange* (ASCII) agar dapat dibaca oleh manusia.

2.2.14 Entropy pada JWT Secret

Entropy dalam kriptografi adalah ukuran ketidakpastian atau kerandoman dalam sebuah *string* karakter yang digunakan untuk membentuk suatu kata (Grassi dkk., 2017). *Secret key* dalam JWT berfungsi untuk menghasilkan tanda tangan digital (*signature*) yang menjamin integritas dan autentikasi *payload*. Semakin tinggi nilai *entropy*, semakin sulit untuk menebak atau menduplikasi kunci tersebut, sehingga sistem menjadi lebih aman dari serangan. *Entropy* diukur dalam satuan bit dan dapat dihitung menggunakan Persamaan (2.3).

$$H = L \times \log_2 N \quad \dots\dots\dots (2.3)$$

di mana:

H = *entropy* dalam bit

L = panjang *secret*

N = jumlah kemungkinan karakter unik dalam himpunan karakter

Sebagai contoh, untuk kata dengan panjang 16 karakter dan jumlah kemungkinan karakter sebanyak 94, maka nilai entropinya adalah sekitar 104.8 bit. Nilai ini menunjukkan seberapa besar jumlah kombinasi kemungkinan yang harus ditebak oleh pihak yang mencoba membobol sistem secara *brute force*. Menurut Grassi dkk. (2017) dan Barker (2020), standar minimum yang direkomendasikan untuk sistem autentikasi adalah 112 bit, sedangkan tingkat keamanan 128 bit atau lebih dianggap memadai untuk jangka panjang. Temuan ini juga diperkuat oleh penelitian Saputra, Sugiarti, Junianto, & Suhartono (2025) yang menyatakan bahwa nilai *entropy* sangat berperan dalam menentukan waktu yang dibutuhkan dalam serangan *brute*

force, serta secara langsung memengaruhi kekuatan suatu sandi atau kunci dalam sistem keamanan. Sebagai tambahan, klasifikasi tingkat keamanan berdasarkan rentang *entropy* dapat dilihat pada Tabel 2.2 yang diadaptasi dari Vainer (2023), di mana nilai *entropy* ≥ 128 bit dikategorikan sebagai Sangat Kuat, sedangkan nilai *entropy* di bawah 28 bit digolongkan Sangat Lemah.

Tabel 2.2 Tingkat kekuatan untuk setiap rentang entropi

Rentang Entropi	Level Kekuatan
$E < 28$	Sangat Lemah
$28 \leq E \leq 35$	Lemah
$36 \leq E \leq 59$	Sedang
$60 \leq E \leq 127$	Kuat
$E \geq 128$	Sangat Kuat

2.2.15 Serangan *Brute Force* pada JWT *Secret*

Brute force merupakan salah satu metode serangan kriptografi yang dilakukan dengan mencoba seluruh kemungkinan kombinasi dari sebuah kunci atau secret hingga ditemukan yang sesuai (Saputra, dkk., 2025). Dalam konteks JWT, *brute force* digunakan untuk menebak nilai *secret* yang digunakan dalam proses penandatanganan token, dengan mencocokkan hasil tanda tangan terhadap token yang valid. Jumlah kemungkinan kombinasi kata dihitung dengan Persamaan (2.4) dan estimasi waktu yang dibutuhkan untuk menyelesaikan proses *brute force* juga dapat dihitung menggunakan Persamaan (2.5).

$$C = N^L \quad \dots\dots\dots (2.4)$$

dengan:

C = jumlah total kemungkinan kombinasi

L = panjang *secret*

N = jumlah kemungkinan karakter unik dalam himpunan karakter

$$T = \frac{C}{R} = \frac{N^L}{R} \quad \dots\dots\dots (2.5)$$

dengan:

R = jumlah tebakan yang dapat dilakukan per detik

Sebagai contoh, apabila sebuah *secret* terdiri dari 6 karakter huruf kecil ($N = 26$), maka jumlah kemungkinan kombinasi adalah sekitar 308 juta. Jika proses *brute force* dilakukan dengan kecepatan satu juta tebakan per detik, maka waktu yang diperlukan untuk menjangkau seluruh kemungkinan adalah sekitar 309 detik atau sekitar 5 menit. Penelitian yang dilakukan oleh Saputra, dkk. (2025) menunjukkan bahwa semakin rendah kompleksitas suatu kunci atau *secret*, maka semakin cepat proses *brute force* dapat dilakukan. Oleh karena itu, pemilihan *secret* dengan panjang karakter yang cukup dan keragaman karakter yang tinggi menjadi langkah penting untuk memperkuat keamanan sistem terhadap serangan semacam ini.

2.2.16 Klasifikasi Performa Pemuatan Website

Waktu pemuatan halaman (*page load time*) merupakan salah satu metrik utama dalam mengevaluasi performa sebuah situs *web*. Metrik ini menggambarkan lamanya waktu yang dibutuhkan oleh *browser* untuk menampilkan seluruh konten halaman secara utuh hingga siap digunakan oleh pengguna akhir. Kinerja waktu pemuatan yang baik berkontribusi langsung terhadap peningkatan pengalaman pengguna (*user experience*) serta kepuasan pengunjung (Google, 2023). Guna memastikan pengalaman akses yang efisien dan responsif, sejumlah pedoman telah dikembangkan oleh Google untuk menilai kualitas waktu pemuatan halaman secara objektif. Adapun kategori tersebut disusun berdasarkan total waktu hingga halaman selesai dimuat, yang dibagi menjadi empat tingkatan, sebagaimana disajikan pada Tabel 2.3.

Tabel 2.3 Tabel Klasifikasi Performa Pemuatan *Website*

Kategori	Waktu Pemuatan (<i>Finish</i>)	Keterangan
Sangat Baik	< 1 detik	Hampir tidak terasa oleh pengguna. Memberikan pengalaman terbaik.
Baik	1 – 2,5 detik	Masih dalam batas optimal dan responsif.
Cukup	2,5 – 4 detik	Performa mulai menurun, pengguna mulai menyadari keterlambatan.
Buruk	> 4 detik	Keterlambatan akan sangat dirasakan oleh pengguna.

Google juga memperkenalkan beberapa terminologi penting terkait proses pemuatan halaman, antara lain:

1. *DOMContentLoaded* : Waktu ketika dokumen HTML telah sepenuhnya di-*parse* dan struktur DOM tersedia, namun belum termasuk *file* eksternal seperti CSS, JavaScript, dan gambar.
2. *Load* : Menunjukkan bahwa seluruh elemen eksternal telah dimuat secara lengkap.
3. *Finish* : Mencerminkan total waktu hingga tidak ada lagi permintaan jaringan aktif, dan halaman sepenuhnya tenang secara trafik.

Pemahaman terhadap metrik-metrik tersebut memungkinkan pengembang untuk mengidentifikasi titik-titik kritis dalam siklus pemuatan halaman dan menentukan strategi optimalisasi yang tepat. Kinerja pemuatan yang buruk dapat meningkatkan potensi pengguna meninggalkan situs sebelum halaman selesai ditampilkan, sebuah fenomena yang dikenal sebagai *bounce* (Google, 2023). Dengan demikian, waktu pemuatan bukan hanya persoalan teknis, tetapi juga elemen strategis yang memengaruhi efektivitas interaksi pengguna terhadap sistem pemantauan (Aladwani, 2020).

2.2.17 *User Acceptance Testing (UAT)*

UAT merupakan tahapan akhir dalam proses pengembangan sistem informasi sebelum sistem tersebut diterapkan secara penuh oleh pengguna akhir. UAT bertujuan untuk memastikan bahwa sistem telah memenuhi kebutuhan, ekspektasi, dan spesifikasi yang telah disepakati oleh pengguna (Marnewick, 2016). Pada tahap ini, pengguna yang mewakili kelompok target dari sistem melakukan serangkaian pengujian untuk menilai kesesuaian fungsionalitas sistem terhadap skenario penggunaan riil.

UAT sangat penting dalam siklus hidup pengembangan perangkat lunak karena dapat mengungkapkan ketidaksesuaian antara desain sistem dan kebutuhan aktual pengguna, yang tidak selalu terdeteksi pada tahap pengujian teknis seperti *unit*

testing atau integration testing (Sabherwal dkk., 2019). Selain itu, hasil UAT juga menjadi dasar evaluasi terhadap kesiapan sistem untuk diluncurkan secara operasional. Metode yang umum digunakan dalam pelaksanaan UAT adalah penyebaran kuesioner kepada pengguna untuk memperoleh umpan balik terhadap pengalaman pengguna atau *user experience (UX)* dan tingkat kepuasan terhadap sistem. Salah satu pendekatan yang sering digunakan dalam evaluasi kuesioner adalah Skala Likert, yang memungkinkan responden memberikan penilaian berdasarkan tingkat persetujuan mereka terhadap sejumlah pernyataan (Joshi dkk., 2015). Skala ini biasanya terdiri dari 4 hingga 7 poin, yang mencerminkan rentang dari sangat tidak setuju hingga sangat setuju. Adapun skala yang digunakan dalam penelitian ini terdiri dari empat tingkat penilaian, seperti ditunjukkan pada Tabel 2.4.

Tabel 2.4 Kategori skor skala Likert pada kuesioner UAT

Skor	Kategori	Interpretasi
1	Sangat Tidak Setuju	Responden sangat tidak menyetujui pernyataan yang diberikan
2	Tidak Setuju	Responden tidak menyetujui pernyataan yang diberikan
3	Setuju	Responden menyetujui pernyataan yang diberikan
4	Sangat Setuju	Responden sangat menyetujui pernyataan yang diberikan

Untuk mengetahui rata-rata penerimaan pengguna, dilakukan pengolahan data kuesioner dengan perhitungan nilai rata-rata setiap item pernyataan menggunakan Persamaan (2.6) (Yaguana, 2023).

$$\bar{X} = \frac{\sum_{i=1}^n x_i}{n} \dots\dots\dots (2.6)$$

dengan:

$\bar{X}$ = nilai rata-rata

x_i = skor masing-masing responden

n = jumlah responden

Tingkat penerimaan pengguna terhadap sistem juga dapat dihitung dengan mengonversi nilai rata-rata menjadi persentase, menggunakan rumus:

$$\text{Tingkat Penerimaan (\%)} = \frac{\bar{X}}{\text{Skor Maksimal}} \times 100 \quad \dots\dots\dots (2.7)$$

Skor maksimal pada skala *Likert* empat poin adalah 4, maka nilai persentase yang mendekati 100% menunjukkan tingkat penerimaan yang tinggi. Untuk memberikan acuan dalam menafsirkan tingkat penerimaan pengguna berdasarkan nilai persentase tersebut, berikut disajikan Tabel 2.5 yang menunjukkan kategori interpretasi tingkat penerimaan sistem berdasarkan rentang persentase.

Tabel 2.5 Interpretasi persentase tingkat penerimaan pengguna

Persentase	Interpretasi
81% – 100%	Sangat Baik (Diterima)
61% – 80%	Baik
41% – 60%	Cukup
21% – 40%	Kurang
0% – 20%	Tidak Layak

2.2.18 Load Testing

Load testing merupakan metode pengujian kinerja perangkat lunak yang bertujuan mengevaluasi kemampuan sistem dalam menangani beban kerja tertentu secara representatif, baik dari jumlah pengguna, volume transaksi, maupun pola interaksi. Menurut standar ISO/IEC 25010:2023, *load testing* berhubungan langsung dengan karakteristik *performance efficiency*, yaitu kemampuan produk menjalankan fungsi sesuai batas waktu tanggap (*response time*), *throughput*, dan efisiensi sumber daya yang ditentukan (ISO/IEC, 2023). Pengujian ini berbeda dengan *stress testing* yang berfokus pada titik kegagalan, maupun *soak testing* yang mengevaluasi stabilitas dalam jangka panjang, karena *load testing* menitikberatkan pada performa dalam rentang beban yang realistis (Bolanowski dkk., 2024).

Beberapa metrik utama yang digunakan dalam *load testing* meliputi:

1. Jumlah Sampel : Metrik ini menggambarkan total permintaan yang dikirimkan selama pengujian. Semakin banyak jumlah sampel, semakin tinggi tingkat kepercayaan statistik terhadap hasil pengujian (Xia dkk., 2024).

2. *Error Rate* : Persentase permintaan yang gagal diproses oleh sistem. *Error rate* yang rendah (mendekati 0%) menunjukkan kestabilan sistem. Hal ini sejalan dengan standar ISO/IEC 25023:2016 yang mendefinisikan indikator kualitas perangkat lunak melalui pengukuran tingkat kegagalan (ISO/IEC, 2016).
3. *Response Time* : Meliputi nilai rata-rata, median, dan *percentile* (ke-90, ke-95, ke-99). Penggunaan *percentile* dianggap lebih representatif dalam mengevaluasi kinerja karena mampu menangkap *outlier* yang tidak tercermin dalam nilai rata-rata (Gortázar dkk., 2022).
4. *Throughput* : Mengukur jumlah transaksi yang berhasil diproses per satuan waktu. *Throughput* yang tinggi mengindikasikan kapasitas sistem yang baik dalam menangani permintaan (Camilli dkk., 2022).
5. *Network Utilization* : Mencakup volume data yang diterima maupun dikirim (KB/s). Metrik ini digunakan untuk mengevaluasi apakah keterbatasan bandwidth berpengaruh terhadap performa sistem, terutama pada arsitektur berbasis cloud (Eismann dkk., 2022).

Secara keseluruhan, interpretasi hasil *load testing* harus mempertimbangkan semua indikator secara bersamaan. Misalnya, sebuah sistem dengan waktu tanggap rata-rata 628 ms, *percentile* ke-95 sebesar 885 ms, *throughput* 68 transaksi/detik, serta *error rate* 0% dapat dikategorikan memiliki performa yang stabil pada beban yang diuji. Namun, kesimpulan akhir sangat bergantung pada perbandingan hasil pengujian dengan standar kinerja yang telah ditetapkan dalam *Service Level Agreement* (SLA). Oleh karena itu, validitas *load testing* sangat ditentukan oleh desain skenario uji, keterwakilan pola beban, serta kondisi lingkungan pengujian yang terdokumentasi secara baik (Xia dkk., 2024; Bolanowski dkk., 2024).

Dalam tugas akhir ini, pengujian *load testing* dilakukan untuk mengevaluasi stabilitas dan kinerja sistem pada kondisi beban tertentu. Fokus pengamatan dibatasi pada tiga parameter utama, yaitu *error rate*, *response time*, dan *throughput*, karena ketiga parameter ini secara langsung berhubungan dengan kualitas layanan sistem yang dirasakan pengguna. Untuk menilai hasil pengujian, digunakan acuan standar internasional ISO/IEC 25010:2011 dan ISO/IEC 25023:2016, yang merumuskan kriteria penilaian kinerja perangkat lunak. Kriteria tersebut dapat dilihat pada Tabel 2.6.

Tabel 2.6 Kriteria interpretasi hasil load testing

Parameter	Kategori	Ambang/Standar Umum
<i>Error Rate</i>	Tinggi (buruk)	> 1 %
	Dapat diterima (baik)	≤ 1 %
<i>Response Time</i>	Lambat (risiko UX negatif)	> 500 ms
	Cukup responsif	≤ 500 ms
<i>Throughput</i>	Rendah (potensi <i>lagging</i>)	< 1 000 TPS
	Memadai / Optimal	≥ 1 000 TPS

2.2.19 Application Performance Index (APDEX)

APDEX merupakan standar terbuka untuk mengukur kepuasan pengguna terhadap kinerja aplikasi, khususnya waktu respons. APDEX menyajikan hasil pengukuran dalam bentuk indeks tunggal dengan rentang nilai 0 hingga 1, di mana nilai 0 menunjukkan seluruh pengguna tidak puas, sementara nilai 1 menunjukkan seluruh pengguna puas (Apdex Alliance, 2007).

Perhitungan APDEX dilakukan dengan membagi *response time* (RT) ke dalam tiga kategori, yaitu *Satisfied* ($RT \leq T$), *Tolerating* ($T < RT \leq 4T$), dan *Frustrated* ($RT > 4T$). Rumus perhitungan APDEX dapat dilihat pada Persamaan (2.8).

$$APDEX = \frac{(S + 0,5 \times T)}{N} \dots\dots\dots (2.8)$$

dengan:

S = jumlah sampel yang masuk kategori *Satisfied*

T = jumlah sampel yang masuk kategori *Tolerating*

N = jumlah sampel pengukuran

Indeks APDEX selanjutnya diinterpretasikan dalam kategori kualitas performa aplikasi seperti ditunjukkan pada Tabel 2.7.

Tabel 2.7 Kategori interpretasi skor APDEX

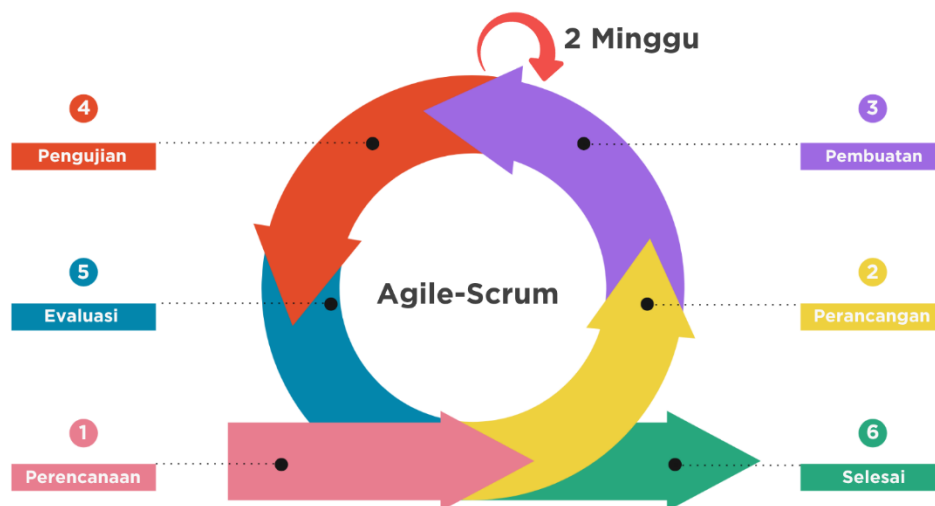
Rentang APDEX	Kategori	Deskripsi
0,94 – 1,00	<i>Excellent</i>	Performa sangat baik
0,85 – 0,93	<i>Good</i>	Sedikit toleransi ada
0,70 – 0,84	<i>Fair</i>	Sebagian pengguna mulai terganggu
0,50 – 0,69	<i>Poor</i>	Performa perlu perbaikan
< 0,50	<i>Unacceptable</i>	Performa buruk

Penerapan APDEX sangat bermanfaat dalam konteks pengujian performa dan pemantauan aplikasi, karena mampu menyederhanakan pelaporan metrik teknis menjadi skor yang mudah dipahami oleh pemangku kepentingan non-teknis. Selain itu, APDEX juga banyak digunakan sebagai acuan dalam menetapkan *Service Level Objective* (SLO) dan *Service Level Agreement* (SLA), misalnya menjaga skor APDEX di atas 0,90 pada transaksi penting untuk memastikan kepuasan pengguna tetap terjaga (Apdex Alliance, 2007).

BAB III

METODOLOGI PENELITIAN

Metode penelitian yang digunakan dalam penyusunan tugas akhir “Sistem Pemantauan Limbah Cair Industri Terintegrasi IoT dan JWT Berbasis *Website* dan *Android*” yaitu menggunakan metode Agile-Scrum. *Agile* adalah sebuah metode pengembangan perangkat lunak yang berbasis pada pengembangan iteratif, di mana persyaratan dan solusi berkembang melalui kolaborasi antar tim yang terorganisir. Salah satu jenis metode *Agile* yang paling populer adalah *Scrum*. *Scrum* mengorganisir pengembangan software ke dalam siklus waktu terbatas yang disebut *sprint*. Dalam setiap *sprint*, tim melakukan perencanaan, analisis, desain, implementasi, pengujian, dan peninjauan (Avancha dkk., 2024). Tahapan-tahapan metode penelitian dalam tugas akhir ini dapat dilihat pada Gambar 3.1.

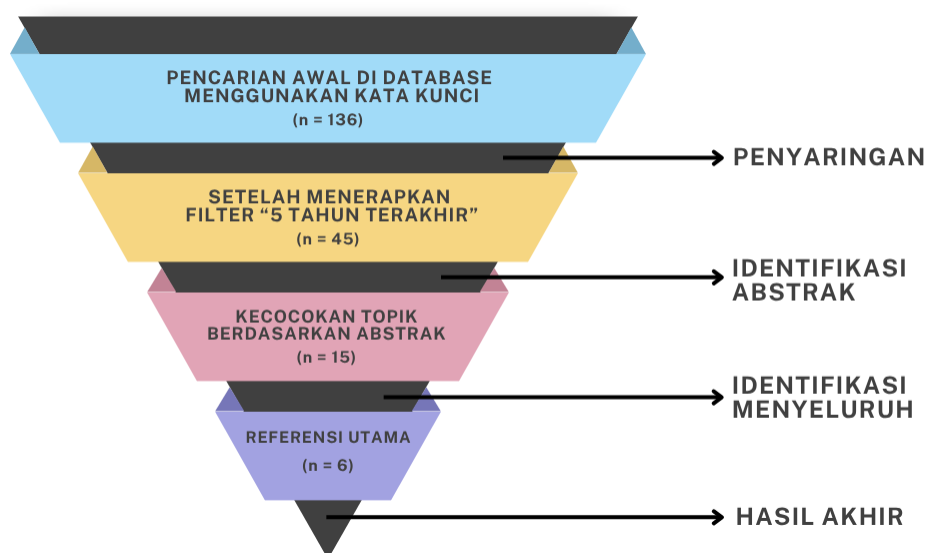


Gambar 3.1 Metode Agile-Scrum
(Sumber: Dokumentasi Pribadi)

3.1 Perencanaan

Pada tahap ini dilakukan pengumpulan data sebelum tugas akhir dilaksanakan, yaitu dengan mencari literasi studi pustaka, serta mengidentifikasi kebutuhan. Studi Pustaka dilaksanakan untuk memahami teori penunjang sistem yang akan digunakan pada tahap perancangan dan pembuatan (Avancha dkk., 2024). Literasi dalam penelitian ini adalah yang berkaitan dengan tugas akhir. Studi dilakukan secara sistematis dengan menelusuri berbagai sumber seperti tugas akhir tahun-

tahun sebelumnya, dan referensi digital dari jurnal ilmiah nasional maupun internasional yang relevan pada *website* Google Scholar, IEEE Xplore, serta MDPI. Hasil studi pustaka dapat dilihat pada Gambar 3.2.



Gambar 3.2 Studi pustaka
(Sumber: Dokumentasi pribadi)

Proses pencarian difokuskan pada artikel yang dipublikasikan dalam rentang waktu 2020 hingga 2024 dengan menggunakan beberapa kata kunci seperti “*IoT Platform*”, “*Wastewater Monitoring*”, dan “*JSON Web Token*”. Dari hasil pencarian, terkumpul sebanyak 136 artikel maupun jurnal ilmiah yang relevan. Selanjutnya Penulis melakukan proses seleksi sesuai dengan topik dan ruang lingkup tugas akhir melalui abstrak karya ilmiah, didapatkanlah sebanyak 6 karya ilmiah yang dijadikan referensi utama. Masing-masing karya ilmiah yang terpilih memiliki metode yang berbeda dalam membuat sistem pemantauan limbah cair industri yang efektif. Namun, masih terdapat kekurangan dalam hal skalabilitas, keandalan, interoperabilitas, atau keamanan.

Selanjutnya dilaksanakan identifikasi kebutuhan untuk memastikan tahapan pembuatan dapat berjalan dengan lancar. Hasil identifikasi yaitu berupa kebutuhan perangkat keras dan perangkat lunak yang akan digunakan. Perangkat keras dapat mendukung dalam pembuatan sistem, sedangkan perangkat lunak dapat membantu dalam membuat program atau aplikasi. Berikut adalah hasil identifikasi kebutuhan:

1. Perangkat Keras yang Digunakan (*Hardware*)

Hardware yang diperlukan ialah seperangkat alat atau elemen elektronik yang dapat membantu atau mendukung dalam kinerja aplikasi ini, sehingga aplikasi yang diusulkan dapat bekerja dengan baik. *Hardware* minimal yang digunakan oleh *server* adalah sebagai berikut:

- a. Komputer dengan *processor 1 Core*.
- b. Memori internal berukuran 1 GB.
- c. Penyimpanan internal berukuran 60 GB.
- d. Koneksi internet.

Adapun *Hardware* minimal yang digunakan oleh *client* (pengguna) adalah sebagai berikut:

- a. Komputer dengan sistem operasi Windows, Linux, atau Mac.
- b. *Smartphone* dengan sistem operasi Android, atau iOS.
- c. Mempunyai koneksi jaringan seluler, atau Wi-Fi.

2. Perangkat Lunak yang Digunakan (*Software*)

Software yang diperlukan ialah yang dapat mendukung dari sistem operasi dan aplikasi *database*, adapun *software* yang digunakan adalah sebagai berikut:

- a. Visual Studio Code.
- b. Android Studio.
- c. *Browser*.
- d. Figma.
- e. Wireshark.
- f. JMeter.

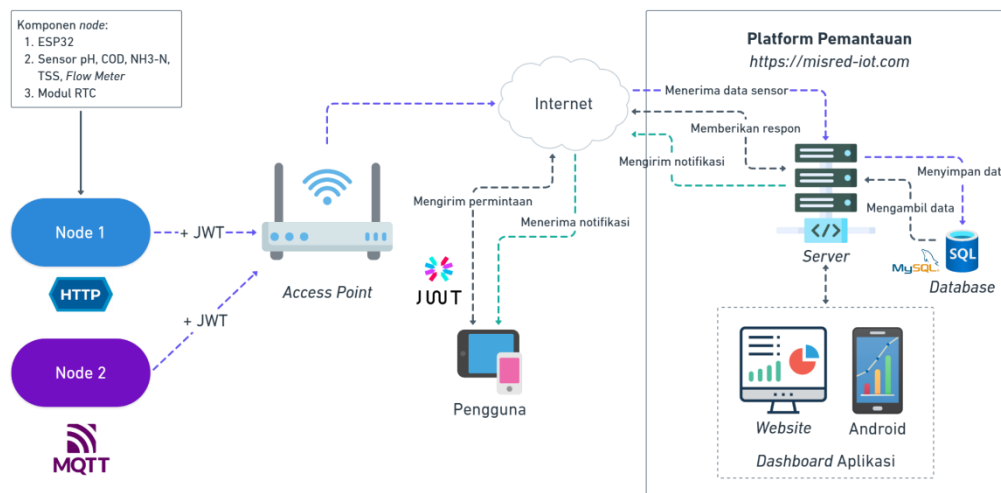
3.2 Perancangan

Pada tahap ini dilakukan pembuatan rancangan kerja untuk sistem pemantauan. Tahap ini meliputi pembuatan rancangan arsitektur dari sistem, rancangan diagram *Use Case*, rancangan ERD, rancangan diagram alur, dan rancangan tampilan antarmuka pengguna atau *user interface* (UI) dari aplikasi.

3.2.1 Arsitektur Sistem

Rancangan dari arsitektur sistem terdiri dari diagram blok sistem secara keseluruhan, dan diagram blok sistem pada tugas akhir. Pada tugas akhir ini,

terdapat dua buah *node* yang akan digunakan untuk berkomunikasi dengan *server*. *Node* merupakan perangkat IoT yang bertugas mengumpulkan data (Rana dkk., 2023). *Node* terdiri dari mikrokontroler ESP32 dan beberapa sensor, yaitu pH, COD, kekeruhan, NH3-N, debit air, serta modul *Real-Time Clock* (RTC). *Node* pertama akan menggunakan protokol HTTP dan *node* kedua akan menggunakan protokol MQTT. Pengiriman data yang dilakukan oleh pengguna dan *node* akan di-*hashing* dengan JWT. *Platform* pemantauan yang berfungsi sebagai *server*, akan menerima data sensor yang dikirim oleh *node*, memvalidasi token JWT milik *node*, dan menyimpannya pada basis data MySQL. *Platform* pemantauan memiliki dua antarmuka yang dapat diakses oleh pengguna, yaitu *website* dan Android. *Platform* berfungsi untuk menampilkan visualisasi data dari sensor untuk dipantau secara *real-time*. Gambaran umum dari arsitektur sistem secara keseluruhan dapat dilihat pada Gambar 3.3

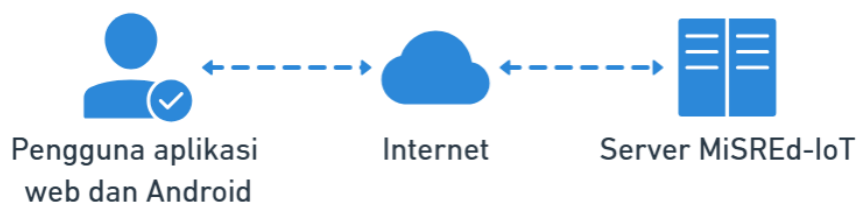


Gambar 3.3 Arsitektur sistem secara keseluruhan

(Sumber: Dokumentasi Pribadi)

Tugas akhir ini akan menghasilkan sebuah aplikasi *web* dan Android. Aplikasi akan berinteraksi dengan sistem melalui *server* ketika terhubung dengan internet. Gambar 3.4 menunjukkan arsitektur dari sistem yang akan dihasilkan pada tugas akhir ini. *Server* yang digunakan berada pada domain <https://misred-iot.com>. Aplikasi *web* dan Android akan berkomunikasi dengan *server* menggunakan protokol HTTP melalui REST API. REST API akan digunakan untuk proses

pemantauan parameter limbah cair, dan proses autentikasi serta otorisasi pengguna aplikasi.

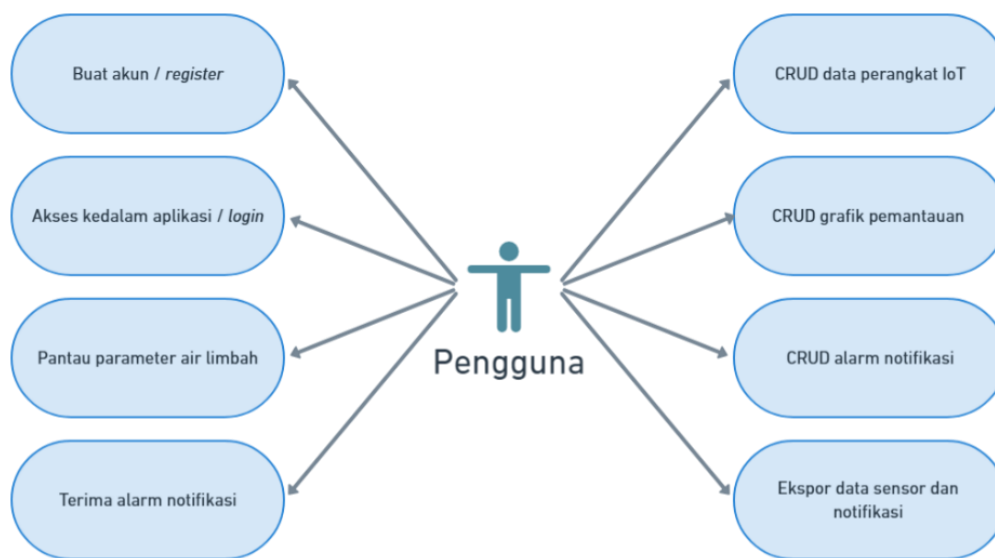


Gambar 3.4 Arsitektur sistem pada tugas akhir

(Sumber: Dokumentasi Pribadi)

3.2.2 Diagram Use Case

Berdasarkan arsitektur sistem dari tugas akhir, dibuatlah diagram *use case* untuk desain skenario awal dari pengguna. Pengguna dapat melakukan *register* atau *login* pada platform pemantauan, melakukan proses *Create, Read, Update, Delete* (CRUD) pada perangkat IoT, grafik pemantauan parameter air limbah, serta alarm notifikasi. Pengguna juga dapat menerima notifikasi pada *browser* atau nomor WhatsApp saat ada alarm yang terpicu. Selain itu, pengguna juga dapat mengekspor data sensor dan notifikasi. Gambar 3.5 menunjukkan tentang diagram ini.



Gambar 3.5 Use Case Diagram

(Sumber: Dokumentasi Pribadi)

3.2.3 Entity Relationship Diagram (ERD)

Pada tugas akhir ini, digunakan sembilan tabel basis data yang dirancang untuk mendukung sistem pemantauan IoT yang komprehensif. Tabel *users* berfungsi

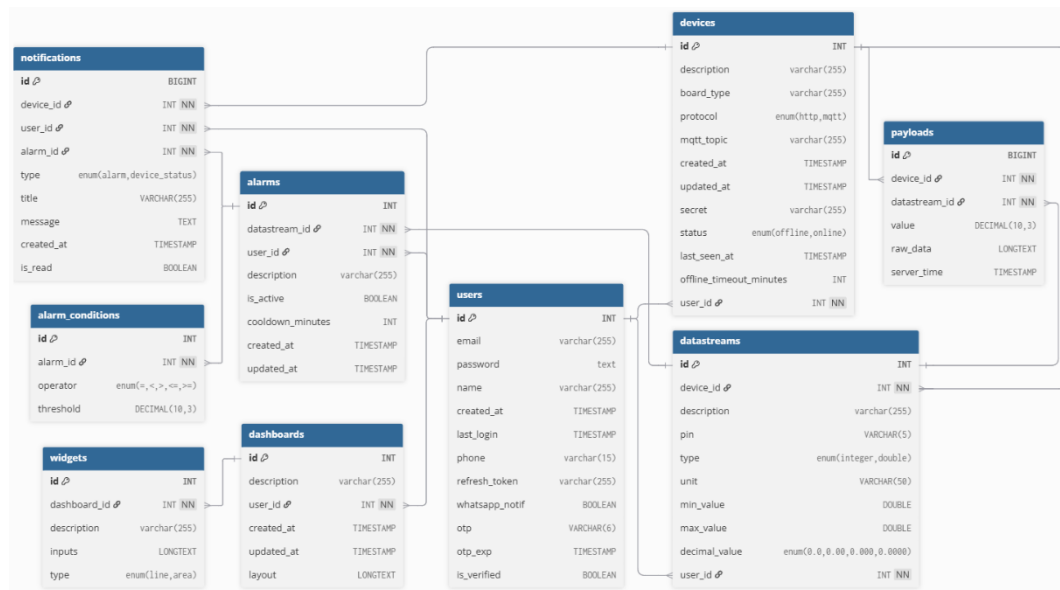
menyimpan kredensial pengguna aplikasi termasuk *email*, *password*, nama, OTP, dan token JWT untuk sistem autentikasi yang aman. Tabel *dashboards* menyimpan konfigurasi dashboard yang dibuat pengguna, termasuk tata letak grafik, nama *dashboard*, dan pengaturan tampilan untuk visualisasi data yang optimal.

Tabel *devices* menyimpan informasi lengkap perangkat IoT yang terdaftar pada *server*, mencakup deskripsi perangkat, jenis *board* (ESP32/Arduino), protokol komunikasi (MQTT/HTTP), status aktif, dan JWT *secret* unik untuk autentikasi komunikasi *node-server*. Tabel *datastreams* berfungsi sebagai konfigurasi *virtual pin* yang mendefinisikan aliran data dari setiap perangkat, menyimpan tipe data sensor, unit pengukuran, nilai maksimum-minimum, dan deskripsi *datastream* untuk fleksibilitas pemetaan data sensor. Tabel *payloads* merupakan tabel penyimpanan utama untuk seluruh data sensor yang dikirimkan perangkat IoT, menyimpan nilai sensor, *timestamp* pengiriman, dan referensi *datastream* untuk analisis historis dan pemantauan *real-time*.

Tabel *widgets* menyimpan konfigurasi visualisasi *dashboard* pengguna, termasuk informasi *dashboard*, *datastream* yang digunakan, tipe grafik, dan tata letak grafik pada *dashboard* untuk tampilan pemantauan. Tabel *alarms* digunakan untuk menyimpan pengaturan sistem peringatan yang dibuat pengguna, mencakup deskripsi alarm, status aktif, referensi *datastream* yang dipantau, dan kondisi pemicu untuk integrasi dengan sistem notifikasi *real-time*. Tabel *alarms_conditions* berfungsi sebagai tabel yang menyimpan kondisi spesifik setiap alarm, termasuk operator perbandingan ($>$, $<$, $=$, $>=$, $<=$), nilai ambang batas, dan logika kombinasi antar kondisi untuk pembuatan alarm yang fleksibel. Tabel *notifications* menyimpan riwayat lengkap notifikasi yang dikirimkan kepada pengguna, mencakup pesan notifikasi *browser* dan WhatsApp, status pembacaan, waktu pengiriman, dan referensi ke perangkat serta pengguna terkait.

Relasi antar tabel menunjukkan struktur hierarkis dimana setiap pengguna (*users*) dapat memiliki satu atau banyak perangkat (*devices*) dengan relasi *one-to-many*. Setiap perangkat kemudian dapat memiliki banyak *datastreams*, *widgets*, *payloads*, *alarms*, *notifications*, dan *dashboards*, membentuk struktur yang memungkinkan skalabilitas sistem. Tabel *alarms_conditions* memiliki relasi *many-to-one* dengan

tabel *alarms*, memungkinkan satu alarm memiliki beberapa kondisi pemicu sekaligus untuk logika yang kompleks. Struktur relasi ini dirancang untuk memberikan fleksibilitas maksimal dalam pengelolaan data IoT, mulai dari pengumpulan data sensor, visualisasi *dashboard*, hingga sistem notifikasi otomatis yang dapat disesuaikan dengan kebutuhan spesifik setiap pengguna. Gambar 3.6 menunjukkan ERD dari tugas akhir ini.

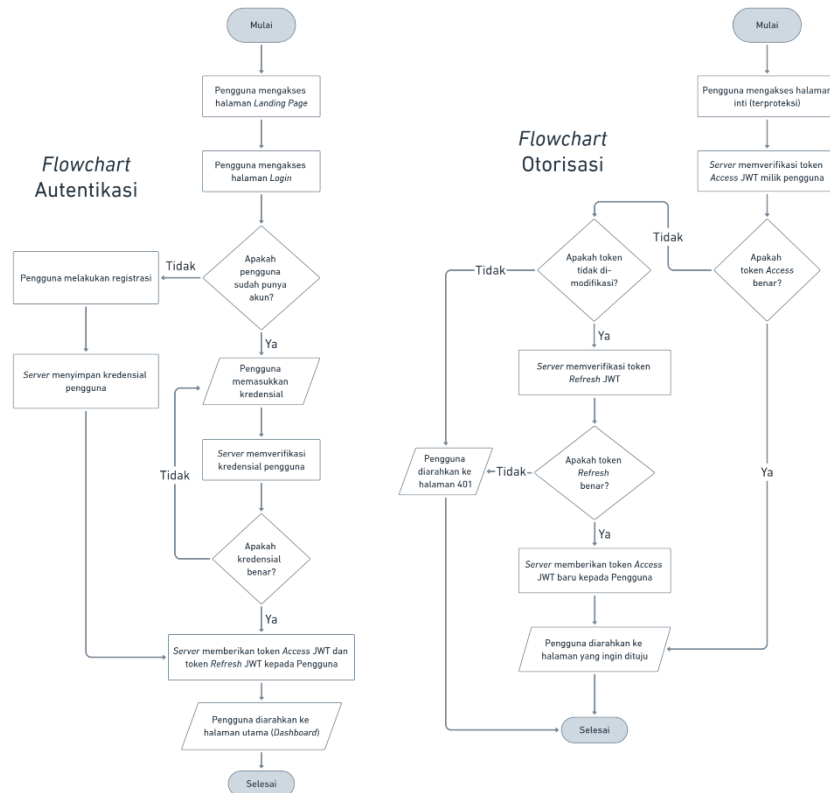


Gambar 3.6 Entity Relationship Diagram

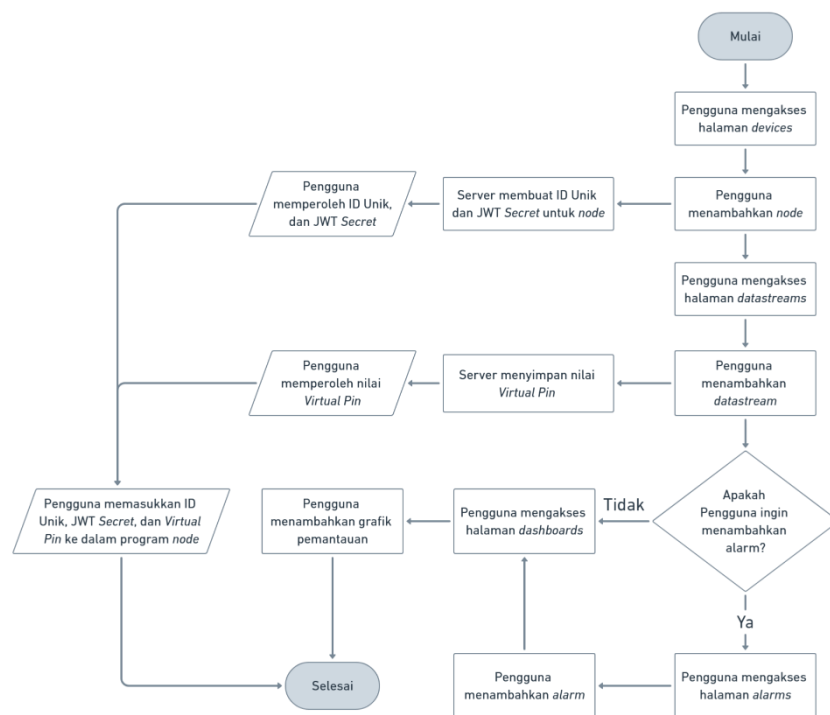
(Sumber: Dokumentasi Pribadi)

3.2.4 Diagram Alur

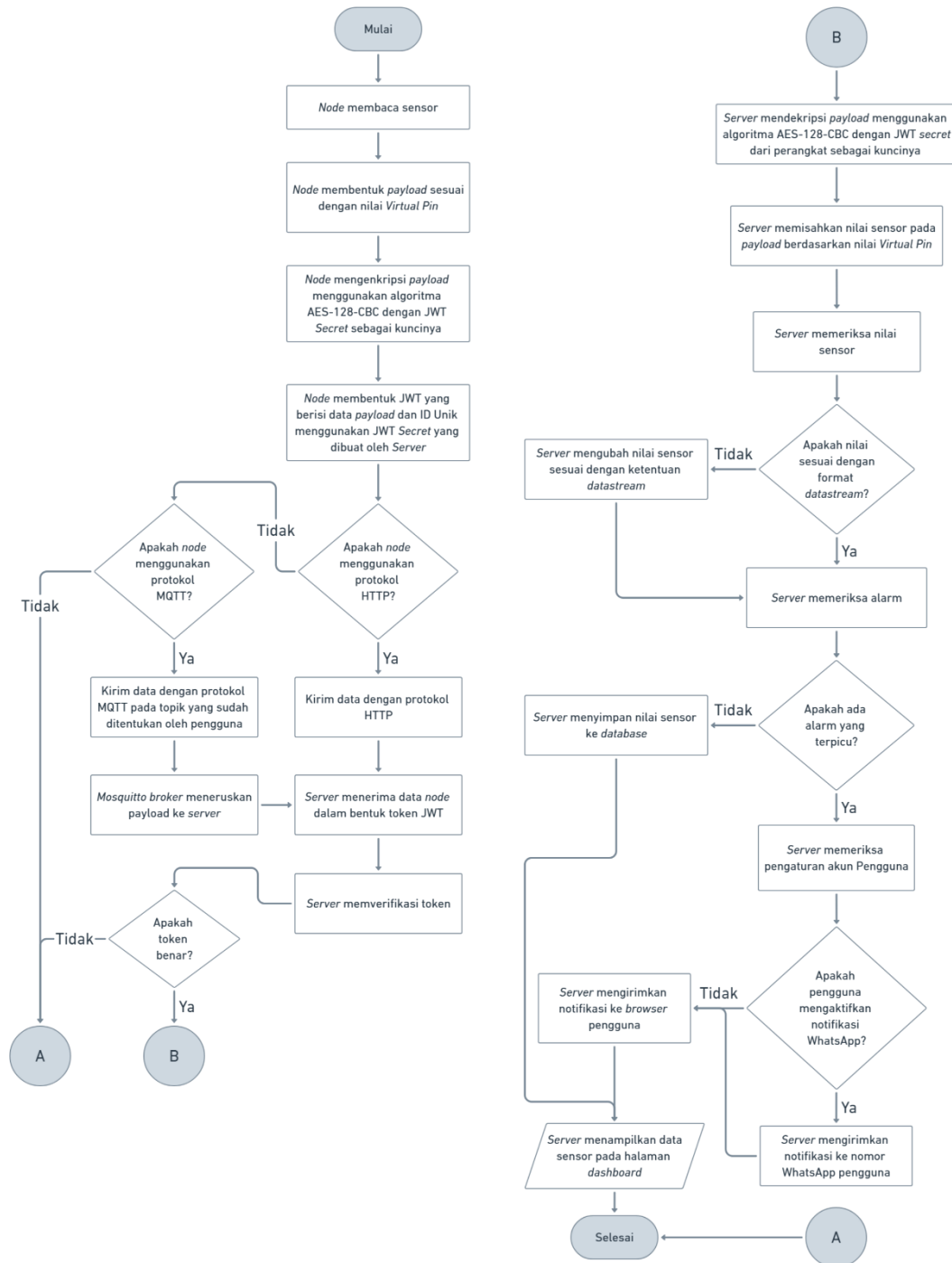
Berdasarkan arsitektur sistem dari tugas akhir, dibuatlah diagram alur untuk desain cara kerja dari sistem pemantauan. Adapun diagram alur dari sistem terbagi menjadi tiga, yaitu diagram alur autentikasi dan otorisasi pengguna, diagram alur penggunaan antarmuka aplikasi, dan diagram alur *node*. Diagram alur autentikasi dan otorisasi pengguna menggambarkan proses keamanan akses sistem, diagram ini dapat dilihat pada Gambar 3.7. Diagram alur penggunaan antarmuka aplikasi menunjukkan navigasi dan fitur sistem pemantauan, diagram ini dapat dilihat pada Gambar 3.8. Diagram alur *node* menjelaskan proses pengumpulan dan pengiriman data sensor dari *node* ke *server*, diagram ini dapat dilihat pada Gambar 3.9. Ketiga diagram alur ini memberikan gambaran komprehensif tentang alur kerja sistem pemantauan IoT mulai dari level pengguna hingga level perangkat keras.



Gambar 3.7 Diagram alur autentikasi dan otorisasi pengguna
(Sumber: Dokumentasi Pribadi)



Gambar 3.8 Diagram alur penggunaan antarmuka aplikasi
(Sumber: Dokumentasi pribadi)

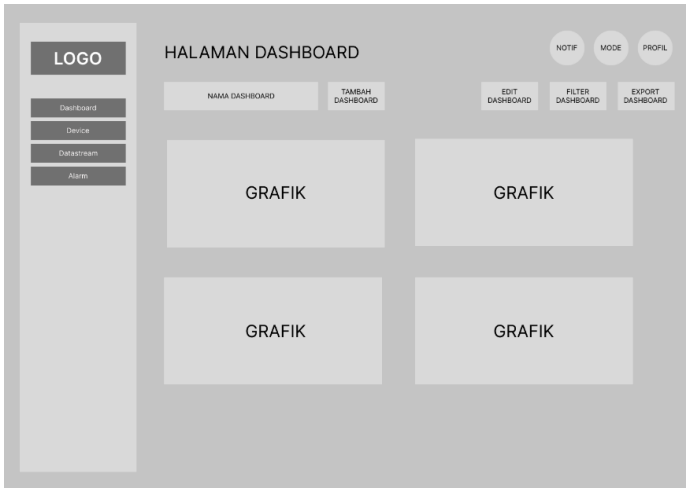
Gambar 3.9 Diagram alur *node*

(Sumber: Dokumentasi pribadi)

3.2.5 Tampilan UI pada Aplikasi

Rancangan UI dari aplikasi *website* dan Android dibuat menggunakan *software* Figma. Tabel 3.1 menunjukkan tampilan antarmuka pada aplikasi *website*, sedangkan Tabel 3.2 menunjukkan tampilan antarmuka pada aplikasi Android.

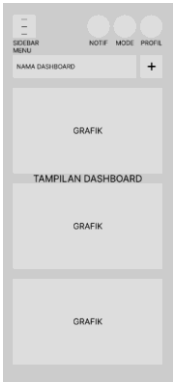
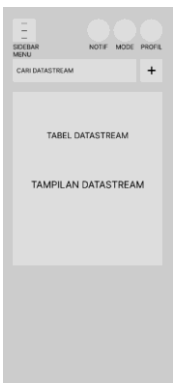
Tabel 3.1 Rancangan tampilan antarmuka aplikasi *website*

No	Nama Halaman	Gambar
1.	Login	
2.	Dashboard	
3.	Devices	

No	Nama Halaman	Gambar
4.	Datastreams	
5.	Alarms	

Tabel 3.2 Rancangan tampilan antarmuka aplikasi Android

No	Nama Halaman	Gambar
1.	Login	

No	Nama Halaman	Gambar
2.	Dashboard	
3.	Devices	
4.	Datastreams	
5.	Alarms	

3.3 Pembuatan

Pada tahap ini dilakukan pengerjaan aplikasi *web* dan Android dengan *software* Visual Studio Code. Tahap ini meliputi berbagai langkah untuk membangun sistem pemantauan limbah cair industri berbasis *web* dan Android. Langkah-langkah tersebut adalah:

1. Membuat tabel data dalam basis data pada *server* sesuai rancangan basis data pada Gambar 3.6.
2. Membuat kode untuk membangun API antara aplikasi *web* dan Android dengan basis data *server* dan *web server*. Pembuatan kode dilakukan menggunakan *framework* Node JS.
3. Membuat kode untuk menjalankan tampilan aplikasi sesuai dengan rancangan UI yang sudah dibuat. Pembuatan kode dilakukan menggunakan *framework* Next JS dan bahasa pemrograman CSS.

3.4 Pengujian

Pada tahap ini dilakukan pengujian sistem yang telah dibuat. Dalam tugas akhir ini terdapat lima pengujian, yaitu:

3.4.1 Black Box Testing (Pengujian fungsionalitas)

Pengujian yang dilakukan pertama kali adalah *Black Box Testing*. Pengujian ini dilakukan untuk menguji apakah fungsionalitas aplikasi sudah benar dan sesuai dengan rancangan. Tabel 3.3 menunjukkan rancangan tabel pengujian fungsionalitas untuk aplikasi *website* dan Android.

Tabel 3.3 Rancangan pengujian fungsionalitas aplikasi *website* dan Android

No	Deskripsi pengujian	Parameter yang diharapkan	Kesesuaian hasil pengujian
1.	Melakukan <i>login</i> dengan data <i>username</i> dan <i>password</i> benar	Pengguna berhasil <i>login</i>	
2.	Melakukan <i>login</i> dengan data <i>username</i> dan <i>password</i> salah	Pengguna gagal <i>login</i>	
3.	Mengakses halaman <i>dashboard</i>	Pengguna akan diarahkan menuju halaman <i>dashboard</i> yang berisi <i>widget</i> dari tiap parameter lengkap dengan waktu terakhirnya	

No	Deskripsi pengujian	Parameter yang diharapkan	Kesesuaian hasil pengujian
4.	Mengakses halaman <i>devices</i>	Pengguna dapat menampilkan daftar perangkat IoT	
5.	Menambahkan perangkat IoT	Pengguna dapat menambahkan perangkat IoT pada halaman <i>devices</i> .	
6.	Mengubah data perangkat IoT	Pengguna dapat mengubah data perangkat IoT pada halaman <i>devices</i>	
7.	Menghapus perangkat IoT	Pengguna dapat menghapus perangkat IoT pada halaman <i>devices</i>	
8.	Menambahkan <i>datastream</i>	Pengguna dapat menambahkan data <i>datastream</i> dari perangkat IoT pada halaman <i>datastreams</i>	
9.	Mengubah <i>datastream</i>	Pengguna dapat mengubah data dari <i>datastream</i> halaman <i>datastreams</i>	
10.	Menghapus <i>datastream</i>	Pengguna dapat menghapus <i>datastream</i> pada halaman <i>datastreams</i>	
11.	Menambahkan grafik pemantauan	Pengguna dapat menambahkan grafik pemantauan pada halaman <i>dashboard</i> .	
12.	Mengubah grafik pemantauan	Pengguna dapat mengubah grafik pemantauan pada halaman <i>dashboard</i> .	
13.	Menghapus grafik pemantauan	Pengguna dapat menghapus grafik pemantauan pada halaman <i>dashboard</i> .	
14.	Menambahkan alarm notifikasi	Pengguna dapat menambahkan alarm pada menu halaman <i>dashboard</i> .	
15.	Mengubah alarm notifikasi	Pengguna dapat mengubah alarm pada menu halaman <i>dashboard</i> .	
16.	Menghapus alarm notifikasi	Pengguna dapat menghapus alarm pada menu halaman <i>dashboard</i> .	

3.4.2 Pengujian Sistem Keamanan

Pengujian selanjutnya adalah pengujian sistem keamanan. Pengujian ini dilakukan untuk dapat mengetahui isi dari *payload* yang dikirimkan dan menguji integritas *payload* tersebut. Tabel 3.4 menunjukkan rancangan pengujian ini.

Tabel 3.4 Rancangan pengujian sistem keamanan

No.	Deskripsi Pengujian	Gambar
1.	Mengakses halaman aplikasi menggunakan token JWT yang benar.	
2.	Mengakses halaman aplikasi menggunakan token JWT yang dimodifikasi.	
3.	Mengakses halaman aplikasi tanpa disertai <i>refresh token</i> JWT	
4.	<i>Node</i> mengirim data sensor menggunakan token yang benar	
5.	<i>Node</i> mengirim data sensor menggunakan token yang kadaluarsa	
6.	<i>Node</i> mengirim data sensor menggunakan token yang dimodifikasi	

Selain pengujian-pengujian tersebut, dilakukan pula pengujian tambahan yang bertujuan untuk mengevaluasi kekuatan *secret key* yang digunakan dalam proses penandatanganan token JWT. Aspek yang diuji meliputi analisis *entropy* dari *secret* JWT untuk menilai tingkat keunikan serta kompleksitasnya secara matematis, dan simulasi serangan *brute force* terhadap token JWT untuk mengukur ketahanan *secret* terhadap eksploitasi berbasis enumerasi kunci. Pengujian ini memberikan pemahaman kuantitatif dan praktis mengenai seberapa aman *secret* yang digunakan terhadap upaya pemalsuan token. Pengukuran *entropy* dilakukan menggunakan Persamaan (2.3). Sementara itu, jumlah kemungkinan kombinasi karakter dalam *brute force* dihitung dengan Persamaan (2.4) dan estimasi waktu *brute force* dihitung menggunakan Persamaan (2.5).

3.4.3 Pengujian Notifikasi

Pengujian selanjutnya adalah pengujian notifikasi. Pengujian ini dilakukan untuk menguji keberhasilan aplikasi dalam memunculkan notifikasi pada *browser* dan WhatsApp. Tabel 3.5 menunjukkan rancangan pengujian ini.

Tabel 3.5 Rancangan pengujian notifikasi

No.	Deskripsi Pengujian	Gambar
1.	Menampilkan pesan notifikasi pada nomor <i>browser</i> saat ada alarm yang terpicu atau ada peringatan perangkat yang <i>offline</i>	
2.	Menampilkan pesan notifikasi pada nomor WhatsApp saat ada alarm yang terpicu atau ada peringatan perangkat yang <i>offline</i>	

3.4.4 Pengujian Performa

Pengujian selanjutnya adalah pengujian performa. Pengujian ini dilakukan untuk dapat mengetahui penggunaan memori, *processor* (CPU), dan jaringan. Dalam pengujian performa aplikasi *website*, digunakan fitur ‘*Network Monitor*’ pada *browser* Google Chrome. Fitur ini digunakan untuk memperoleh waktu yang dibutuhkan untuk memuat halaman. Waktu yang diambil adalah waktu dimuatnya struktur HTML halaman saja (waktu *DOMContentLoaded*), waktu dimuatnya halaman tanpa ikon (waktu *load*), dan waktu dimuatnya keseluruhan halaman (waktu *finish*). Tabel 3.6 menunjukkan rancangan pengujian ini.

Tabel 3.6 Rancangan pengujian performa aplikasi *website*

No.	Halaman	Waktu memuat (ms)		
		<i>DOMContentLoaded</i>	<i>Load</i>	<i>Finish</i>
1.	<i>Login</i>			
2.	<i>Dashboards</i>			
3.	<i>Devices</i>			
4.	<i>Datastreams</i>			
5.	<i>Alarms</i>			

Sementara pada pengujian performa aplikasi Android, digunakan fitur “*Android Profiler*” pada *software* Android Studio. Fitur ini dapat menyediakan informasi mengenai bagaimana kondisi CPU dan memori saat aplikasi dijalankan.

3.5 Load Testing

Pada bab ini akan dijelaskan rancangan pengujian *load testing* yang bertujuan untuk mengevaluasi performa sistem *web* dalam menangani beban akses dari sejumlah

pengguna secara bersamaan. Pengujian dilakukan pada halaman-halaman utama sistem, yaitu *landing page* (halaman awal), *login*, *dashboards*, *datastreams*, *devices*, dan *alarms*, karena halaman-halaman tersebut merupakan komponen inti yang paling sering digunakan oleh pengguna. Parameter yang digunakan dalam pengujian meliputi *error rate*, *response time*, dan *throughput*. Ketiga parameter ini dipilih karena berhubungan langsung dengan stabilitas, kecepatan, dan kapasitas sistem dalam memberikan layanan. Selain itu, hasil pengujian juga akan ditambahkan dengan perhitungan *Application Performance Index* (APDEX) sebagai metrik tambahan untuk mengukur tingkat kepuasan pengguna berdasarkan waktu *respons* yang diterima.

Pengujian dilakukan dengan variasi jumlah pengguna secara bertahap, mulai dari 10, 25, 50, 100, hingga 150 pengguna, untuk mengetahui batas kemampuan sistem dalam menangani beban kerja yang meningkat. Rancangan tabel pengujian yang akan digunakan dapat dilihat pada Tabel 3.7

Tabel 3.7 Rancangan pengujian *Load Testing*

Jumlah Pengguna	Response Time (ms)	Error Rate (%)	Throughput (Transaction/s)	APDEX
10				
25				
50				
100				
150				

3.6 User Acceptance Testing (UAT)

UAT dilakukan untuk memperoleh umpan balik langsung dari pengguna terhadap kualitas antarmuka dan pengalaman pengguna (UI/UX) dari sistem berbasis *web* yang telah dikembangkan. Dalam pengujian ini, penyusun menggunakan pendekatan kualitatif dengan menyebarkan kuesioner yang terdiri dari beberapa pernyataan yang relevan dengan aspek UI/UX kepada sejumlah responden terpilih. Setiap responden memberikan tanggapan terhadap setiap pernyataan menggunakan Skala Likert empat poin, yaitu: sangat tidak setuju, tidak setuju, setuju, dan sangat setuju.

Data yang diperoleh dari hasil kuesioner kemudian diolah untuk memperoleh nilai rata-rata dari setiap pernyataan menggunakan Persamaan (2.1). Selanjutnya, nilai rata-rata tersebut dikonversi ke dalam bentuk persentase penerimaan dengan menggunakan Persamaan (2.2). Dengan menggunakan dua rumus tersebut, hasil dari kuisisioner dapat dianalisis untuk mengetahui sejauh mana sistem diterima oleh pengguna dari sisi kenyamanan visual, kemudahan navigasi, kecepatan akses, kejelasan informasi, dan elemen-elemen UX lainnya.

Pernyataan-pernyataan yang digunakan dalam kuesioner disusun berdasarkan prinsip evaluasi antarmuka dan pengalaman pengguna yang mencakup aspek visual, navigasi, kecepatan akses, serta keterbacaan informasi. Daftar lengkap pernyataan yang diajukan kepada responden disajikan pada Tabel 3.7.

Tabel 3.8 Daftar pernyataan kuesioner pengujian UAT

No.	Pernyataan	Aspek
1.	Data grafik yang ditampilkan secara visual mudah dipahami.	Visualisasi dan Antarmuka
2.	Penggunaan warna pada grafik (seperti merah dan hijau) sudah cukup jelas untuk membedakan perangkat sensor.	Visualisasi dan Antarmuka
3.	Keterangan pada grafik (judul, label sumbu, dan legenda) mudah dimengerti oleh pengguna.	Visualisasi dan Antarmuka
4.	Antarmuka halaman <i>Devices</i> terlihat rapi dan mudah dipahami.	Visualisasi dan Antarmuka
5.	Antarmuka halaman <i>Datastreams</i> terlihat rapi dan mudah dipahami.	Visualisasi dan Antarmuka
6.	Antarmuka halaman <i>Alarms</i> terlihat rapi dan mudah dipahami.	Visualisasi dan Antarmuka
7.	Riwayat notifikasi dapat diakses dengan mudah oleh pengguna.	Navigasi dan Akses Informasi
8.	Pengguna dapat dengan mudah beralih antara mode gelap dan terang.	Visualisasi dan Antarmuka
9.	Mode gelap memiliki kontras warna yang baik dan nyaman untuk dilihat.	Visualisasi dan Antarmuka
10.	Teks dan ikon tetap jelas terlihat dalam kedua mode (gelap maupun terang).	Visualisasi dan Antarmuka
11.	Fitur <i>filter</i> atau penyaringan data bekerja dengan baik dan mudah digunakan.	Fungsionalitas dan Interaksi
12.	Pengaturan <i>widget</i> saat menambahkan grafik baru (seperti mengisi nama dan memilih datastream) mudah dipahami.	Fungsionalitas dan Interaksi
13.	Perubahan tata letak dan ukuran grafik terasa fleksibel dan responsif.	Fungsionalitas dan Interaksi

No.	Pernyataan	Aspek
14.	Proses ekspor berjalan lancar tanpa hambatan teknis.	Fungsionalitas dan Interaksi
15.	Opsi “Sesuai Filter” dan “Semua Data” membantu dalam menyesuaikan kebutuhan ekspor.	Fungsionalitas dan Interaksi
16.	Tombol “Edit” dan “Tambah <i>Dashboard</i> ” mudah ditemukan pada halaman.	Navigasi dan Akses Informasi
17.	Fungsi pencarian <i>device</i> mudah digunakan dan efektif.	Navigasi dan Akses Informasi
18.	Pengguna dapat memahami daftar <i>device</i> tanpa perlu penjelasan tambahan.	Navigasi dan Akses Informasi
19.	Fungsi pencarian <i>datastream</i> mudah digunakan dan efektif.	Navigasi dan Akses Informasi
20.	Pengguna dapat memahami daftar <i>datastream</i> tanpa perlu penjelasan tambahan.	Navigasi dan Akses Informasi
21.	Fungsi pencarian <i>alarm</i> mudah digunakan dan efektif	Navigasi dan Akses Informasi
22.	Pengguna dapat memahami daftar <i>alarm</i> tanpa perlu penjelasan tambahan.	Navigasi dan Akses Informasi
23.	Sistem memberikan umpan balik yang cukup setelah perubahan disimpan.	Umpan Balik Sistem
24.	Sistem memberikan umpan balik yang cukup setelah <i>device</i> ditambahkan.	Umpan Balik Sistem
25.	Sistem memberikan umpan balik yang cukup saat <i>device</i> akan dihapus.	Umpan Balik Sistem
26.	Sistem memberikan umpan balik yang cukup setelah <i>datastream</i> ditambahkan.	Umpan Balik Sistem
27.	Sistem memberikan umpan balik yang cukup saat <i>datastream</i> akan dihapus.	Umpan Balik Sistem
28.	Sistem memberikan umpan balik yang cukup setelah <i>alarm</i> ditambahkan.	Umpan Balik Sistem
29.	Sistem memberikan umpan balik yang cukup saat <i>alarm</i> akan dihapus.	Umpan Balik Sistem
30.	Formulir penambahan <i>device</i> memiliki label dan instruksi yang jelas.	Formulir dan Kejelasan Instruksi
31.	Formulir penambahan <i>datastream</i> memiliki label dan instruksi yang jelas.	Formulir dan Kejelasan Instruksi
32.	Formulir penambahan <i>alarm</i> memiliki label dan instruksi yang jelas.	Formulir dan Kejelasan Instruksi
33.	Notifikasi muncul dengan jelas saat alarm terpicu.	Notifikasi dan Respons Sistem
34.	Sistem memberikan informasi notifikasi yang mudah dipahami.	Notifikasi dan Respons Sistem

Tabel 3.7 menunjukkan bahwa setiap pernyataan telah dirancang untuk menggambarkan persepsi pengguna terhadap elemen-elemen utama dari UI/UX

sistem. Dengan menggunakan Skala Likert, responden dapat mengungkapkan tingkat kesetujuan mereka terhadap setiap pernyataan tersebut.

3.7 Evaluasi

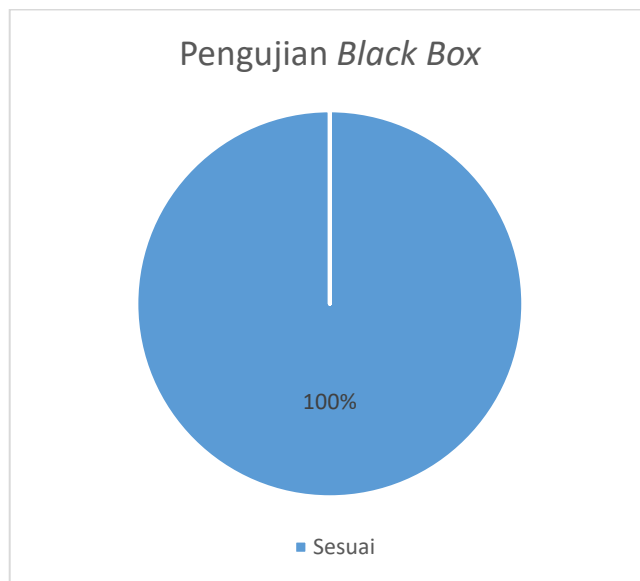
Pada tahap ini dilakukan proses *debugging* dan perilisan, kemudian dilanjutkan dengan pembahasan hasil penelitian. *Debugging* dilakukan untuk mengatasi ketidakselarasan kode. Ketidakselarasan kode ini biasanya sebuah galat yang tidak terdeteksi saat pembuatan hingga proses *compiling* dan hanya akan terlihat setelah program dijalankan. Selain itu, *debugging* juga digunakan sebagai peninjau apakah aplikasi sudah sesuai dan dapat dengan mudah dipahami oleh pengguna. Untuk aplikasi *web*, akan dirilis pada *server* Misred-IoT dengan *domain* <https://misred-iot.com/>. Adapun aplikasi Android dapat dipasang pada perangkat Android melalui implementasi Android WebView, dengan cara menekan menu "Tambahkan ke Layar Utama" pada *browser* perangkat Android.

BAB IV

HASIL PENGUJIAN DAN ANALISIS

4.1 Hasil Pengujian Fungsionalitas Aplikasi (*Black Box Testing*)

Hasil dari *black box testing* adalah berupa sukses atau gagalnya aplikasi berjalan sesuai skenario. Pada pengujian ini hanya berfokus pada fungsionalitas dan keluaran aplikasi. Hasil pengujian fungsionalitas dapat dilihat dengan grafik pada Gambar 4.1.



Gambar 4.1 Diagram hasil pengujian *Black Box*
(Sumber: Dokumentasi pribadi)

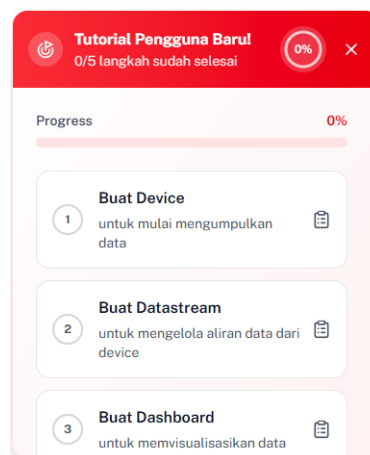
Berdasarkan hasil pengujian *black box* terhadap sistem, telah dilakukan 16 skenario pengujian yang mencakup seluruh fitur utama pada aplikasi, antara lain proses *login*, manajemen perangkat IoT, pengelolaan *datastream*, visualisasi grafik pemantauan, serta pengaturan alarm notifikasi. Hasil pengujian disajikan dalam bentuk diagram lingkaran yang menggambarkan distribusi status pengujian terhadap masing-masing fungsi. Dari keseluruhan pengujian, seluruh skenario menunjukkan status “Sesuai” (100%), yang berarti semua fungsi berjalan sesuai dengan parameter yang diharapkan. Hasil ini menunjukkan bahwa dari sisi fungsionalitas, sistem telah memenuhi kebutuhan dasar pengguna sebagaimana dirancang pada tahap perancangan sistem. Setiap *input* yang diberikan

menghasilkan *output* yang sesuai, serta tidak ditemukan kesalahan dalam alur proses maupun tampilan antar muka pengguna pada saat pengujian dilakukan.

Data hasil pengujian fungsionalitas aplikasi terbagi ke dalam 16 bagian, mencakup platform *website* maupun Android, di antaranya:

1. *Login* Berhasil

Proses *login* dilakukan dengan mengisi alamat email dan kata sandi pengguna, kemudian menekan tombol masuk. Alternatifnya, pengguna juga dapat melakukan login melalui akun Google dengan memilih opsi “*Login* dengan Gmail”. Apabila proses *login* berhasil, sistem akan langsung mengarahkan pengguna ke halaman *dashboard*. Pada halaman ini, ditampilkan daftar panduan penggunaan awal yang ditujukan bagi pengguna baru agar dapat memahami dan menggunakan aplikasi secara optimal. Daftar panduan terbaru dapat terlihat seperti Gambar 4.2.



Gambar 4.2 Daftar panduan pengguna baru

(Sumber: Dokumentasi pribadi)

Pengguna baru dapat mengikuti panduan tersebut dengan menekan daftar langkah yang tersedia. Terdapat lima langkah utama dalam panduan ini, yaitu:

- 1) *Buat Device* : Langkah pertama adalah membuat atau menambahkan *device* (perangkat IoT) yang akan digunakan. Langkah ini bertujuan untuk memulai proses pengumpulan data dari perangkat yang terhubung.

- 2) Buat *Datastream* : Langkah kedua adalah membuat atau menambahkan *datastream*, yang digunakan untuk mengelola aliran data dari perangkat IoT yang telah ditambahkan.
- 3) Buat *Dashboard* : Langkah ketiga adalah membuat *dashboard* baru yang memungkinkan pengguna untuk memvisualisasikan data serta menambahkan *widget* sesuai kebutuhan.
- 4) Buat *Widget* : Langkah keempat adalah menambahkan *widget* atau grafik dengan menekan tombol *edit* pada halaman *dashboards*.
- 5) Buat *Alarm* : Langkah kelima adalah membuat alarm yang berfungsi untuk menetapkan ambang batas pada parameter tertentu, sehingga sistem dapat memberikan notifikasi peringatan apabila terjadi pelampauan nilai.

Setiap langkah yang diselesaikan akan meningkatkan presentase panduan pengguna hingga mencapai 100%, yang menandakan bahwa seluruh panduan telah berhasil dilaksanakan.

2. *Login* Gagal

Proses *login* dapat dinyatakan gagal apabila pengguna memasukkan alamat email atau kata sandi yang tidak sesuai. Dalam kondisi tersebut, sistem akan menampilkan indikator berupa pesan peringatan “*Login* gagal!” disertai dengan keterangan “Kredensial tidak valid”, sebagaimana ditunjukkan pada Gambar 4.3.

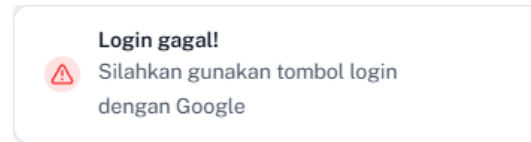


Gambar 4.3 *Login* gagal

(Sumber: Dokumentasi pribadi)

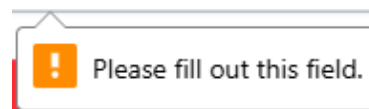
Indikator ini muncul khususnya ketika pengguna telah mendaftarkan akun secara manual melalui halaman registrasi. Namun, apabila pengguna mencoba

login menggunakan akun yang didaftarkan melalui integrasi akun Google tanpa menggunakan tombol “*Login dengan Gmail*”, sistem akan menampilkan indikator “*Login gagal!*” dengan keterangan “Silakan gunakan tombol *login dengan Google*”, seperti diperlihatkan pada Gambar 4.4.



Gambar 4.4 *Login gagal menggunakan akun Google*
(Sumber: Dokumentasi pribadi)

Selain itu, apabila pengguna menekan tombol *login* tanpa mengisi kolom *email* maupun kata sandi, sistem akan menampilkan indikator kesalahan pada masing-masing kolom yang belum diisi, sebagaimana ditunjukkan pada Gambar 4.5.



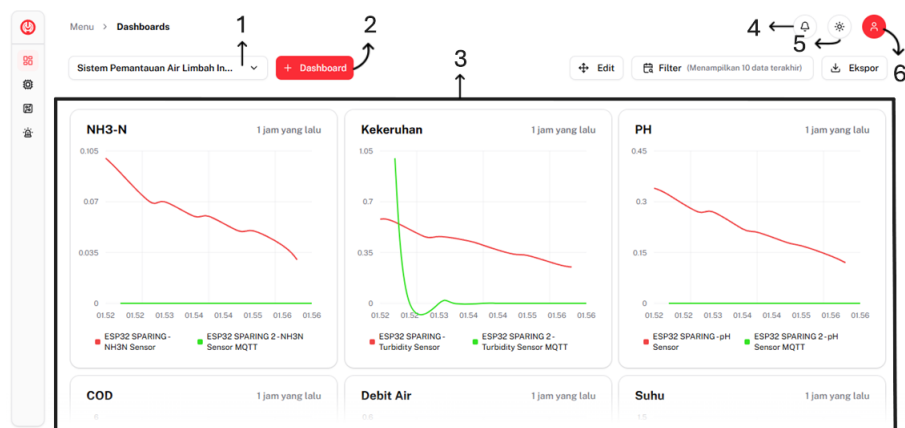
Gambar 4.5 Indikator kolom kosong
(Sumber: Dokumentasi pribadi)

3. Mengakses Halaman *Dashboard*

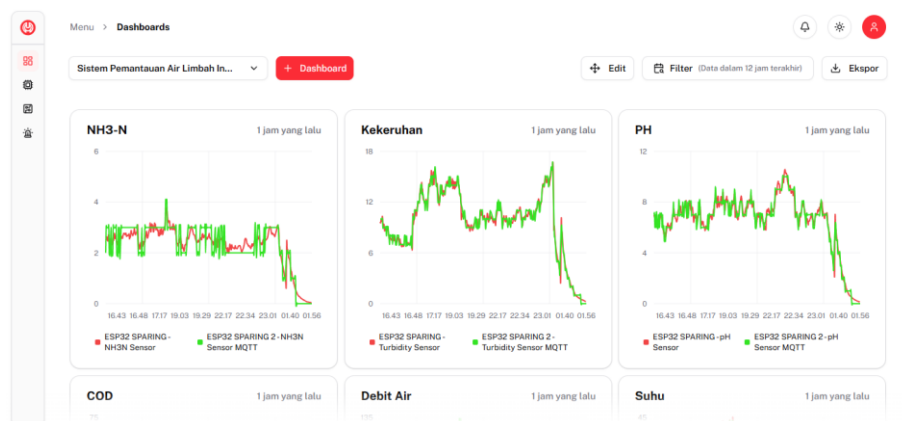
Pada halaman *Dashboard*, pengguna dapat melihat grafik yang telah ditambahkan sebelumnya. Data yang ditampilkan pada grafik dapat disaring berdasarkan rentang waktu tertentu dengan menekan tombol *Filter*, sebagaimana ditunjukkan pada Gambar 4.6 dengan *filter* berdasarkan jumlah data dan Gambar 4.7 dengan *filter* berdasarkan waktu. Untuk menambahkan grafik baru, pengguna harus terlebih dahulu memasuki mode *Edit*, dengan ketentuan bahwa *dashboard* telah dibuat sebelumnya. Selain itu, pada halaman ini, pengguna juga memiliki kemampuan untuk membuat lebih dari satu *dashboard* sesuai dengan kebutuhan visualisasi data. Pada halaman *dashboard* terdapat beberapa komponen seperti pada Gambar 4.6, yaitu:

- 1) Nama *Dashboard* : Kolom ini menampilkan nama *dashboard* yang telah ditambahkan oleh pengguna.

- 2) Tombol : Tombol ini digunakan untuk menambahkan *dashboard*.
- 3) Kanvas *Dashboard* : Ini merupakan bagian yang menampilkan grafik yang telah ditambahkan.
- 4) Ikon Notifikasi : Ikon ini untuk menampilkan dan melihat daftar riwayat notifikasi.
- 5) Ikon Tema : Ikon untuk merubah tema latar menjadi gelap atau terang.
- 6) Ikon Profil : Ikon untuk melihat profil pengguna dan terdapat tombol keluar.



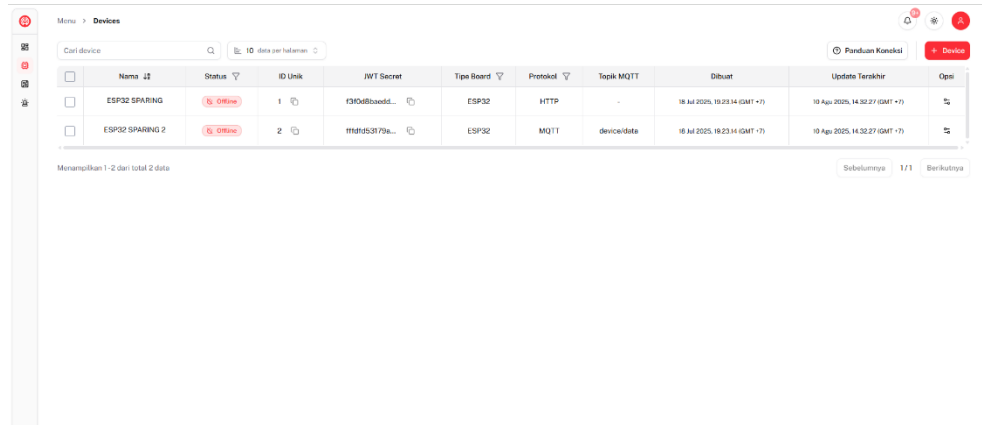
Gambar 4.6 Halaman *dashboard* dengan *filter* berdasarkan jumlah data
(Sumber: Dokumentasi pribadi)



Gambar 4.7 Halaman *dashboard* dengan *filter* berdasarkan waktu
(Sumber: Dokumentasi pribadi)

4. Mengakses Halaman *Devices*

Pada halaman *Devices*, pengguna dapat melihat daftar perangkat atau perangkat IoT yang telah ditambahkan sebelumnya. Tampilan halaman ini disusun dalam bentuk tabel yang memuat informasi terkait perangkat yang tersedia, sebagaimana ditunjukkan pada Gambar 4.8.



	Nama	Status	ID Unik	JWT Secret	Tipe Board	Protokol	Topik MQTT	Dibuat	Update Terakhir	Ops
☐	ESP32 SPARING	Offline	1	ESP32Board...	ESP32	HTTP	-	18 Jul 2025, 18:23:14 (GMT +7)	10 Apr 2025, 14:32:27 (GMT +7)	
☐	ESP32 SPARING 2	Offline	2	ESP32Board...	ESP32	MQTT	device/data	18 Jul 2025, 18:23:14 (GMT +7)	10 Apr 2025, 14:32:27 (GMT +7)	

Menampilkan 1-2 dari total 2 data

Sebelumnya 1/1 Berikutnya

Gambar 4.8 Halaman *devices*

(Sumber: Dokumentasi pribadi)

Adapun isi dari tabel tersebut meliputi:

- 1) Nama : Kolom ini menampilkan nama perangkat IoT yang telah ditambahkan oleh pengguna.
- 2) *Status* : Kolom ini menampilkan status konektivitas perangkat. Perangkat akan berstatus *Offline* apabila tidak terhubung dengan *server*, dan *Online* apabila perangkat berhasil terhubung dengan *server*.
- 3) *JWT Secret* : Kolom ini memuat token JWT yang diperlukan oleh perangkat untuk dapat terhubung dengan *server*.
- 4) *Tipe Board* : Kolom ini mencantumkan jenis *board* yang digunakan oleh perangkat.
- 5) Protokol : Kolom ini menunjukkan jenis protokol komunikasi yang digunakan oleh perangkat.
- 6) Topik MQTT : Kolom ini berisi topik yang digunakan apabila perangkat menggunakan protokol MQTT untuk berkomunikasi dengan *server*.

- 7) *Dibuat* : Kolom ini menunjukkan waktu saat data perangkat pertama kali dibuat.
- 8) *Update Terakhir*: Kolom ini menampilkan waktu terakhir data perangkat diperbarui atau diubah.
- 9) *Opsi* : Kolom ini menyediakan tombol untuk mengubah maupun menghapus data perangkat yang telah ditambahkan.

5. Menambahkan Perangkat IoT

Penambahan perangkat IoT dapat dilakukan dengan menekan tombol “+ *Device*” yang terletak di pojok kanan atas tabel pada halaman *Devices*. Setelah tombol tersebut ditekan, akan muncul formulir “Tambah *Device*” sebagaimana ditunjukkan pada Gambar 4.9.

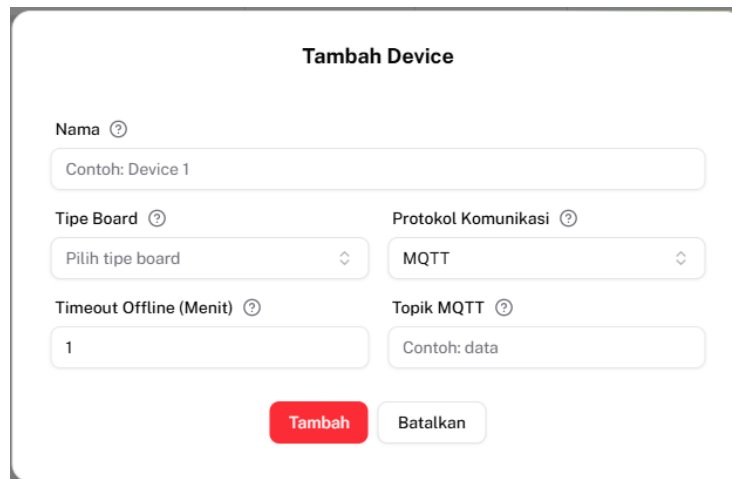
Gambar 4.9 Formulir tambah *device*

(Sumber: Dokumentasi pribadi)

Formulir ini terdiri atas beberapa isian, antara lain:

- 1) *Nama* : Kolom isian untuk memberikan nama pada perangkat (*device*) yang akan ditambahkan.
- 2) *Tipe Board* : Kolom berupa pilihan jenis *board* yang akan digunakan, seperti ESP32, ESP8266, Arduino, Raspberry Pi, atau opsi lainnya.
- 3) *Protokol Komunikasi* : Kolom berupa pilihan jenis protokol komunikasi yang digunakan oleh perangkat, yaitu HTTP atau MQTT.

- 4) **Topik MQTT** : Kolom isian untuk menentukan *topic* yang akan digunakan oleh perangkat jika protokol yang dipilih adalah MQTT. Kolom ini hanya akan ditampilkan apabila pengguna memilih protokol MQTT, sebagaimana diperlihatkan pada Gambar 4.10.
- 5) *Timeout Offline* Kolom berupa isian untuk menentukan waktu jeda pengiriman data. Jika perangkat tidak mengirimkan data apapun ke *server* dalam rentang waktu ini, maka *server* akan menganggap perangkat tersebut tidak aktif atau *offline*.



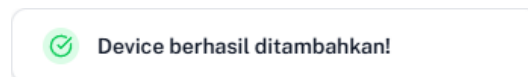
The image shows a web form titled "Tambah Device". It contains the following fields:

- Nama**: A text input field with a placeholder "Contoh: Device 1".
- Tipe Board**: A dropdown menu with the text "Pilih tipe board".
- Protokol Komunikasi**: A dropdown menu with the text "MQTT".
- Timeout Offline (Menit)**: A text input field with the value "1".
- Topik MQTT**: A text input field with a placeholder "Contoh: data".

At the bottom of the form are two buttons: a red "Tambah" button and a grey "Batal" button.

Gambar 4.10 Formulir tambah *device* ketika memilih protokol MQTT
(Sumber: Dokumentasi pribadi)

Setelah perangkat (*device*) berhasil ditambahkan, akan ditampilkan indikator keberhasilan sebagaimana ditunjukkan pada Gambar 4.11.

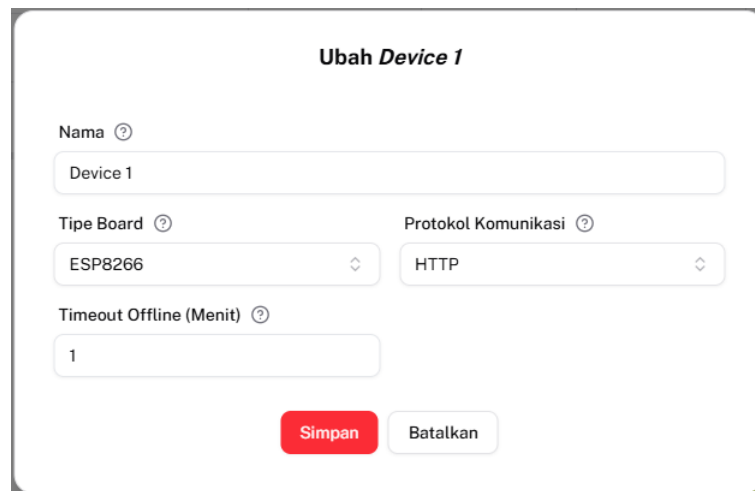


Gambar 4.11 Indikator berhasil menambahkan perangkat
(Sumber: Dokumentasi Pribadi)

6. Mengubah Data Perangkat IoT

Pengubahan data perangkat dapat dilakukan dengan menekan tombol *edit* pada kolom Opsi. Setelah tombol ditekan, sistem akan menampilkan formulir

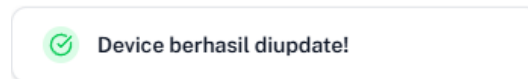
bertajuk “*Edit [Nama Device]*” yang memuat informasi perangkat yang telah ditambahkan sebelumnya, sebagaimana ditunjukkan pada Gambar 4.12.



Gambar 4.12 Formulir *edit device*

(Sumber: Dokumentasi pribadi)

Setelah proses penyimpanan data berhasil dilakukan, sistem akan menampilkan indikator sebagai konfirmasi keberhasilan, seperti diperlihatkan pada Gambar 4.13.

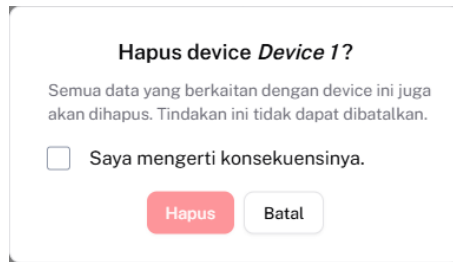


Gambar 4.13 Indikator berhasil mengubah data perangkat

(Sumber: Dokumentasi pribadi)

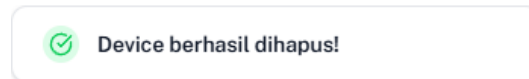
7. Menghapus Perangkat IoT

Pengguna dapat menghapus data perangkat IoT dengan menekan tombol “Hapus” yang tersedia pada kolom Opsi. Setelah tombol tersebut ditekan, sistem akan menampilkan dialog konfirmasi penghapusan yang berisi peringatan mengenai konsekuensi dari tindakan tersebut, yaitu seluruh data yang terkait dengan perangkat akan turut terhapus dan tindakan ini bersifat permanen serta tidak dapat dibatalkan, sebagaimana ditunjukkan pada Gambar 4.14.



Gambar 4.14 Peringatan persetujuan hapus *device*
(Sumber: Dokumentasi pribadi)

Setelah pengguna menyetujui konsekuensi dengan mencentang pernyataan persetujuan dan menekan tombol “Hapus”, sistem akan menampilkan indikator bahwa proses penghapusan telah berhasil, sebagaimana diperlihatkan pada Gambar 4.15.



Gambar 4.15 Indikator berhasil menghapus perangkat
(Sumber: Dokumentasi pribadi)

8. Menampilkan Halaman *Datastreams*

Pada halaman *Datastreams*, pengguna dapat melihat daftar *datastream* yang berasal dari perangkat IoT yang telah ditambahkan sebelumnya. Tampilan halaman ini disusun dalam bentuk tabel yang menyajikan informasi detail mengenai masing-masing *datastream* dari perangkat yang tersedia. Adapun isi dari tabel tersebut meliputi:

- 1) Nama : Kolom ini menampilkan nama *datastream* atau parameter yang telah ditambahkan oleh pengguna.
- 2) *Device* : Kolom ini menunjukkan nama perangkat (*device*) yang terhubung dengan *datastream* tersebut.
- 3) *Virtual Pin* : Kolom ini menampilkan nomor *virtual pin* yang digunakan oleh *datastream* untuk keperluan visualisasi pada halaman *dashboard*.
- 4) Tipe Data : Kolom ini menunjukkan jenis data dari *datastream* atau parameter yang digunakan, yaitu *Integer* atau *Double*.

- 5) Satuan : Kolom ini menampilkan satuan pengukuran yang digunakan untuk data dari *datastream* atau parameter.
- 6) *Min* : Kolom ini menunjukkan nilai minimum yang diperbolehkan untuk *datastream* atau parameter tersebut.
- 7) *Max* : Kolom ini menunjukkan nilai maksimum yang diperbolehkan untuk *datastream* atau parameter tersebut.
- 8) Opsi : Kolom ini menyediakan tombol untuk mengedit maupun menghapus data *datastream* yang telah ditambahkan.

Tampilan halaman *Datastreams* dapat dilihat pada Gambar 4.16.

	Nama	Device	Virtual Pin	Tipe Data	Satuan	Min	Max	Opsi
☐	Suhu	Device 1	V0	Integer	°C	0	35	
☐	Suhu	Device 2	V0	Integer	°C	0	1	

Gambar 4.16 Halaman *datastreams*

(Sumber: Dokumentasi pribadi)

9. Menambahkan *Datastream*

Penambahan *datastream* dapat dilakukan dengan menekan tombol “+ *Datastream*” yang terletak di pojok kanan atas tabel pada halaman *Datastreams*. Setelah tombol tersebut ditekan, akan muncul formulir “Tambah *Datastream*” sebagaimana ditunjukkan pada Gambar 4.17.

Tambah Datastream

Nama
Contoh: Sensor suhu ruangan

Device: Virtual Pin: Tipe Data:

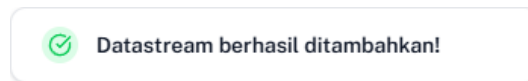
Satuan:

Minimal: Maksimal:

Gambar 4.17 Formulir tambah *datastream*

(Sumber: Dokumentasi pribadi)

Setelah *datastream* berhasil ditambahkan, akan ditampilkan indikator keberhasilan sebagaimana ditunjukkan pada Gambar 4.18.



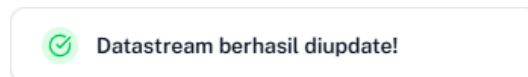
Gambar 4.18 Indikator berhasil menambahkan *datastream*
(Sumber: Dokumentasi pribadi)

10. Mengubah *Datastream*

Pengubahan data perangkat dapat dilakukan dengan menekan tombol *edit* pada kolom Opsi. Setelah tombol ditekan, sistem akan menampilkan formulir berjudul “*Edit [Nama Datastream]*” yang memuat informasi perangkat yang telah ditambahkan sebelumnya, sebagaimana ditunjukkan pada Gambar 4.19.

Gambar 4.19 Formulir *edit datastream*
(Sumber: Dokumentasi pribadi)

Setelah proses penyimpanan data berhasil dilakukan, sistem akan menampilkan indikator sebagai konfirmasi keberhasilan, seperti diperlihatkan pada Gambar 4.20.

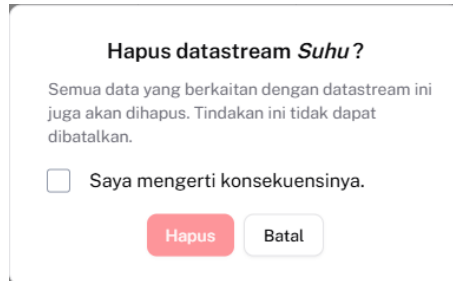


Gambar 4.20 Indikator berhasil mengubah *datastream*
(Sumber: Dokumentasi pribadi)

11. Menghapus *Datastream*

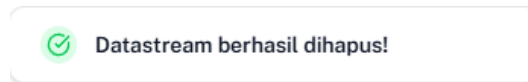
Pengguna dapat menghapus data *datastream* dengan menekan tombol “Hapus” yang tersedia pada kolom Opsi. Setelah tombol tersebut ditekan, sistem akan menampilkan dialog konfirmasi penghapusan yang berisi peringatan mengenai

konsekuensi dari tindakan tersebut, yaitu seluruh data yang terkait dengan perangkat akan turut terhapus dan tindakan ini bersifat permanen serta tidak dapat dibatalkan, sebagaimana ditunjukkan pada Gambar 4.21.



Gambar 4.21 Peringatan persetujuan hapus *datastream*
(Sumber: Dokumentasi pribadi)

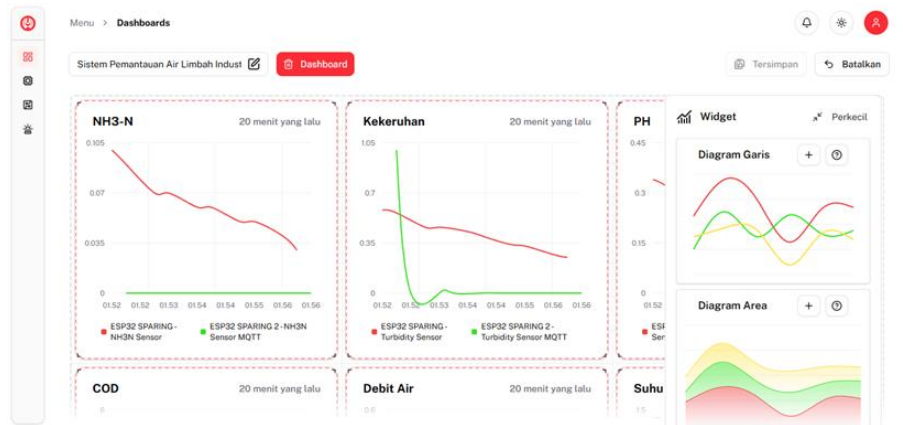
Setelah pengguna menyetujui konsekuensi dengan mencentang pernyataan persetujuan dan menekan tombol “Hapus”, sistem akan menampilkan indikator bahwa proses penghapusan telah berhasil, sebagaimana diperlihatkan pada Gambar 4.22.



Gambar 4.22 Indikator berhasil hapus *datastream*
(Sumber: Dokumentasi pribadi)

12. Menambahkan Grafik Pemantauan

Penambahan grafik dapat dilakukan setelah pengguna terlebih dahulu membuat data perangkat dan *datastream* yang terkait. Proses ini dilakukan melalui halaman *Dashboard*, dengan langkah awal membuat *dashboard* baru dengan menekan tombol “+ *Dashboard*”. Setelah *dashboard* berhasil dibuat, pengguna dapat mengaktifkan mode pengeditan dengan menekan tombol *Edit* yang terletak di pojok kanan atas halaman. Pada mode ini, pengguna dapat menambahkan jenis grafik seperti diagram garis atau diagram area, sebagaimana ditunjukkan pada Gambar 4.23.



Gambar 4.23 Mode *edit* halaman *dashboard*

(Sumber: Dokumentasi pribadi)

Grafik dapat ditambahkan dengan dua cara, yaitu menekan tombol “+” atau menarik jenis grafik yang diinginkan ke dalam kanvas *dashboard*. Setelah grafik diletakkan pada kanvas, sistem akan menampilkan formulir konfigurasi grafik seperti diperlihatkan pada Gambar 4.24. Formulir ini terdiri dari beberapa komponen isian, yaitu:

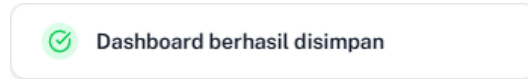
- 1) Nama : Kolom isian untuk memberikan nama pada grafik (*widget*) yang akan ditambahkan.
- 2) *Device & Datastream* : Kolom pilihan untuk menentukan dari perangkat (*device*) mana yang ingin ditambahkan beserta *datastream*-nya.

Setelah mengisi seluruh komponen pada formulir konfigurasi, pengguna dapat menyesuaikan posisi dan ukuran grafik sesuai kebutuhan.

Gambar 4.24 Formulir pengaturan grafik (*widget*)

(Sumber: Dokumentasi pribadi)

Jika telah selesai, pengguna dapat menekan tombol “Simpan” yang terletak di pojok kanan atas halaman. Setelah proses penyimpanan berhasil, sistem akan menampilkan indikator keberhasilan, sebagaimana ditunjukkan pada Gambar 4.25.



Gambar 4.25 Indikator berhasil menyimpan *dashboard*

(Sumber: Dokumentasi pribadi)

13. Mengubah Grafik Pemantauan

Pengubahan grafik dapat dilakukan dengan memasuki mode *edit* pada halaman *Dashboard*. Dalam mode ini, pengguna dapat melakukan penyesuaian terhadap ukuran, posisi, serta data yang terkait oleh grafik, termasuk data perangkat (*device*) maupun parameter (*datastream*) yang digunakan. Untuk mengubah data yang ditampilkan pada grafik, pengguna dapat menekan tombol *Opsi* yang terletak di pojok kanan atas grafik saat berada dalam mode *edit*. Setelah tombol tersebut ditekan, sistem akan menampilkan formulir pengaturan seperti yang ditunjukkan pada Gambar 4.26. Setelah proses penyimpanan berhasil, sistem akan menampilkan indikator keberhasilan, sebagaimana ditunjukkan pada Gambar 4.25.

Gambar 4.26 Formulir *edit* grafik (*widget*)

(Sumber: Dokumentasi pribadi)

14. Menghapus Grafik Pemantauan

Penghapusan grafik dapat dilakukan oleh pengguna dengan menekan tombol “Hapus” yang terletak di pojok kanan atas grafik saat berada dalam mode *Edit* pada halaman *Dashboard*. Setelah grafik dihapus dari kanvas *dashboard* dan pengguna menekan tombol “Simpan”, sistem akan menghapus grafik tersebut dari tampilan *dashboard* secara permanen dan menampilkan indikator keberhasilan, sebagaimana ditunjukkan pada Gambar 4.25.

15. Menambahkan Alarm Notifikasi

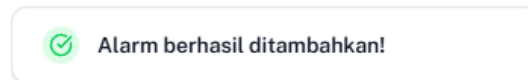
Penambahan alarm dapat dilakukan dengan menekan tombol “Tambah Alarm” pada halaman *Alarms*. Setelah tombol tersebut ditekan, sistem akan menampilkan formulir konfigurasi seperti yang ditunjukkan pada Gambar 4.27. Formulir ini terdiri atas beberapa komponen isian, antara lain:

- 1) Deskripsi : Kolom isian untuk memberikan deskripsi pada alarm yang akan ditambahkan.
- 2) *Device* : Kolom pilihan untuk menentukan perangkat yang akan dikaitkan dengan *alarm*.
- 3) *Datastream* : Kolom pilihan untuk memilih *datastream* dari perangkat yang telah ditentukan sebelumnya.
- 4) Tunggu : Kolom isian untuk menetapkan waktu jeda pengiriman ulang notifikasi *alarm* dalam satuan menit.
- 5) Kondisi : Kolom isian untuk menetapkan ambang batas nilai suatu parameter yang akan memicu *alarm*.
- 6) Status : Tombol pengaturan untuk menentukan apakah *alarm* dalam kondisi aktif atau tidak aktif guna menerima notifikasi.

Gambar 4.27 Formulir tambah alarm

(Sumber: Dokumentasi pribadi)

Setelah *alarm* berhasil ditambahkan, akan ditampilkan indikator keberhasilan, sebagaimana ditunjukkan pada Gambar 4.28.

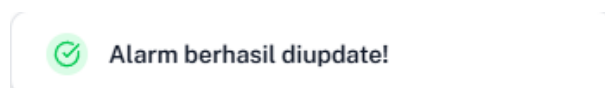


Gambar 4.28 Indikator berhasil menambahkan alarm

(Sumber: Dokumentasi pribadi)

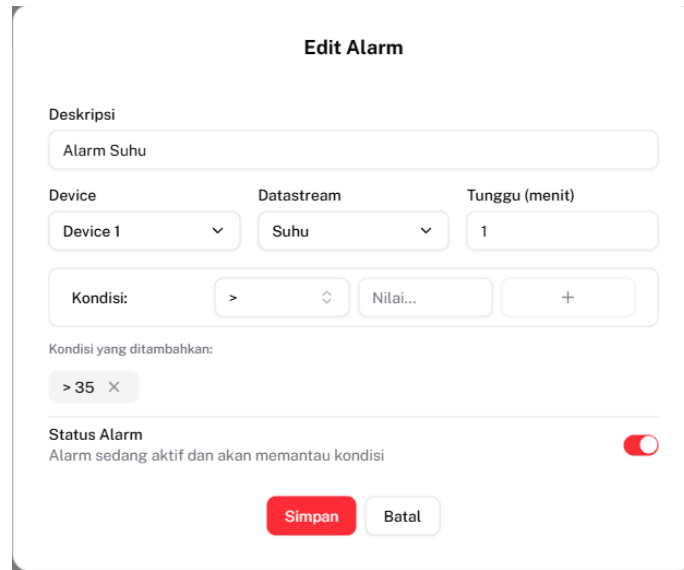
16. Mengubah Alarm Notifikasi

Pengubahan data alarm dapat dilakukan dengan menekan tombol *Edit* pada kolom Opsi. Setelah tombol ditekan, sistem akan menampilkan formulir bertajuk “*Edit Alarm*” yang memuat informasi *alarm* yang telah ditambahkan sebelumnya, sebagaimana ditunjukkan pada Gambar 4.30. Setelah proses penyimpanan data berhasil dilakukan, sistem akan menampilkan indikator sebagai konfirmasi keberhasilan, seperti diperlihatkan pada Gambar 4.29.



Gambar 4.29 Indikator berhasil mengubah alarm

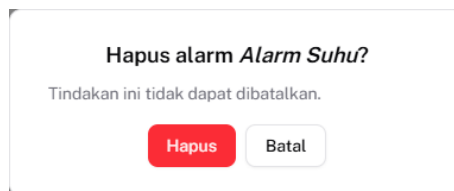
(Sumber: Dokumentasi pribadi)


Gambar 4.30 Formulir *edit* alarm

(Sumber: Dokumentasi pribadi)

17. Menghapus Alarm Notifikasi

Pengguna dapat menghapus data alarm dengan menekan tombol “Hapus” yang tersedia pada kolom Opsi. Setelah tombol tersebut ditekan, sistem akan menampilkan dialog konfirmasi penghapusan yang berisi peringatan mengenai tindakan ini tidak dapat dibatalkan, sebagaimana ditunjukkan pada Gambar 4.31.



Gambar 4.31 Peringatan persetujuan hapus alarm

(Sumber: Dokumentasi pribadi)

Setelah pengguna menyetujui dengan menekan tombol “Hapus”, sistem akan menampilkan indikator bahwa proses penghapusan telah berhasil, sebagaimana diperlihatkan pada Gambar 4.32.



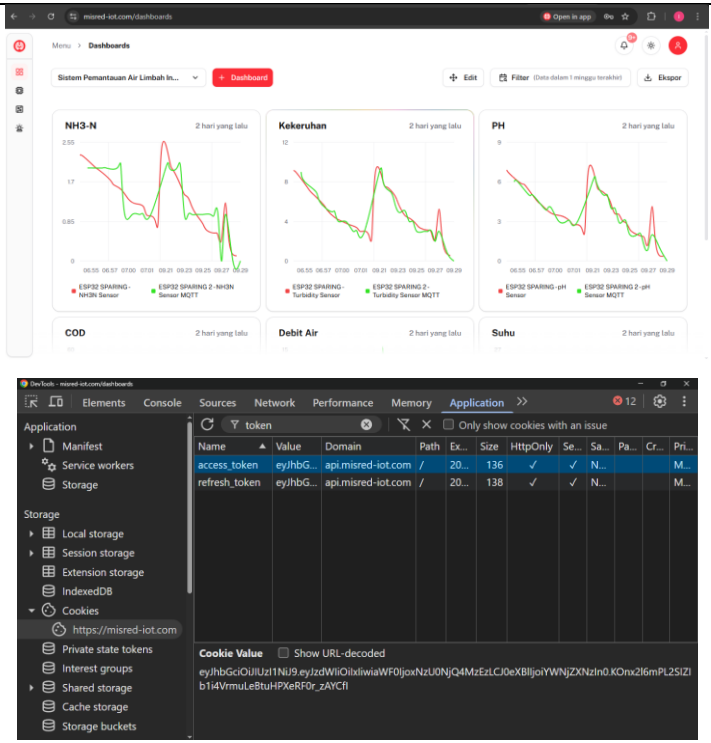
Gambar 4.32 Indikator berhasil hapus alarm

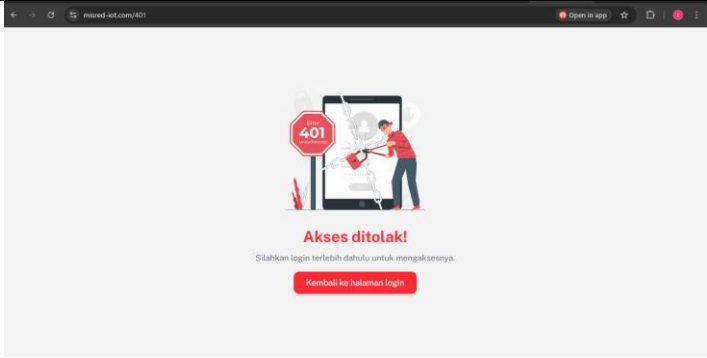
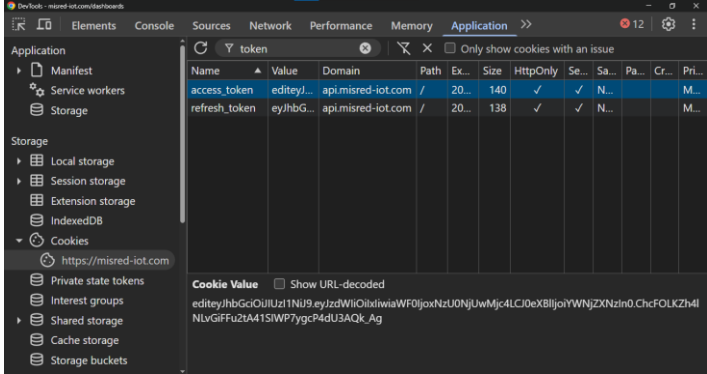
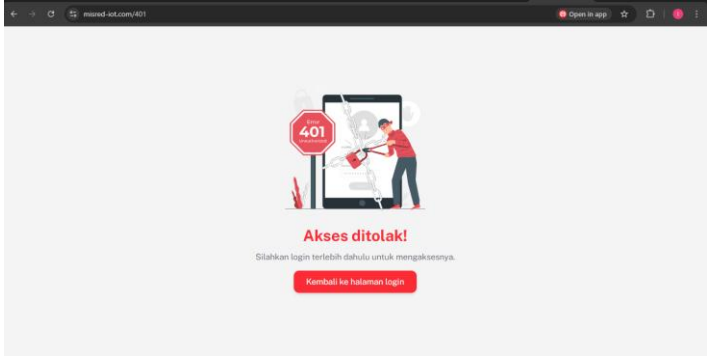
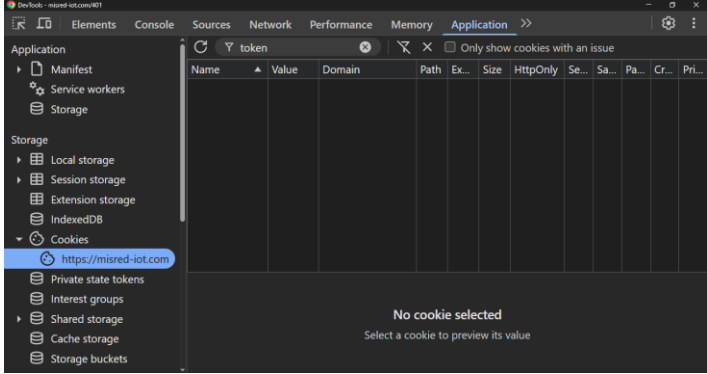
(Sumber: Dokumentasi pribadi)

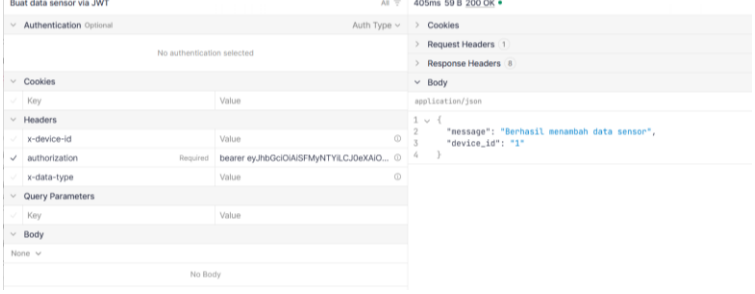
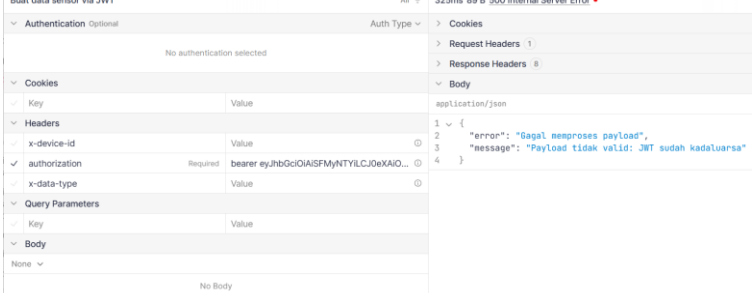
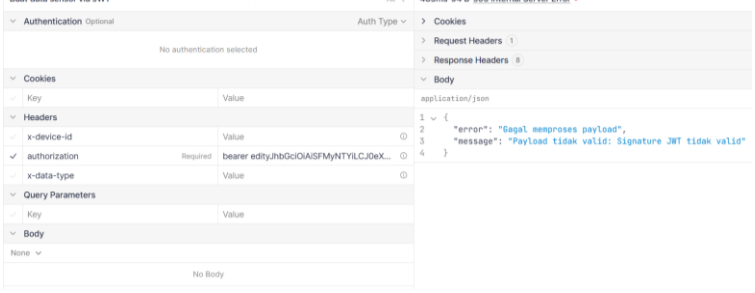
4.2 Hasil Pengujian Sistem Keamanan

Bagian ini menyajikan hasil dari serangkaian pengujian yang dilakukan untuk mengevaluasi aspek keamanan sistem pemantauan limbah cair industri yang telah dikembangkan. Pengujian difokuskan pada validasi mekanisme perlindungan data, khususnya pada proses transmisi dan otentikasi, guna memastikan bahwa informasi yang dikirim antara klien dan *server* tetap aman, utuh, dan hanya dapat diakses oleh entitas yang berwenang. Tujuan utama dari pengujian ini adalah untuk mengukur sejauh mana sistem mampu menangani berbagai skenario autentikasi menggunakan JWT, serta menjamin bahwa *payload* tidak dapat dimodifikasi atau diakses secara tidak sah selama proses komunikasi berlangsung. Hasil pengujian dirangkum dalam Tabel 4.1.

Tabel 4.1 Hasil pengujian sistem keamanan

No.	Deskripsi Pengujian	Gambar
1.	Pengguna mengakses halaman aplikasi menggunakan token JWT yang benar.	 The image shows two screenshots. The top screenshot is a web dashboard titled 'Sistem Pemantauan Air Limbah Industri' with several line charts for NH3-N, Kekeruhan, PH, COD, Debit Air, and Suhu. The bottom screenshot is a Chrome DevTools 'Application' tab showing the 'token' object in the console, which contains 'access_token' and 'refresh_token' values, confirming successful authentication.

No.	Deskripsi Pengujian	Gambar																																				
2.	Pengguna mengakses halaman aplikasi menggunakan token JWT yang dimodifikasi.	 <table><thead><tr><th>Name</th><th>Value</th><th>Domain</th><th>Path</th><th>Ex...</th><th>Size</th><th>HttpOnly</th><th>Se...</th><th>Sa...</th><th>Pa...</th><th>Cr...</th><th>Pri...</th></tr></thead><tbody><tr><td>access_token</td><td>editeyl...</td><td>api.misred-iot.com</td><td>/</td><td>20...</td><td>140</td><td>✓</td><td>✓</td><td>N...</td><td></td><td></td><td>M...</td></tr><tr><td>refresh_token</td><td>eyJhbG...</td><td>api.misred-iot.com</td><td>/</td><td>20...</td><td>138</td><td>✓</td><td>✓</td><td>N...</td><td></td><td></td><td>M...</td></tr></tbody></table>	Name	Value	Domain	Path	Ex...	Size	HttpOnly	Se...	Sa...	Pa...	Cr...	Pri...	access_token	editeyl...	api.misred-iot.com	/	20...	140	✓	✓	N...			M...	refresh_token	eyJhbG...	api.misred-iot.com	/	20...	138	✓	✓	N...			M...
Name	Value	Domain	Path	Ex...	Size	HttpOnly	Se...	Sa...	Pa...	Cr...	Pri...																											
access_token	editeyl...	api.misred-iot.com	/	20...	140	✓	✓	N...			M...																											
refresh_token	eyJhbG...	api.misred-iot.com	/	20...	138	✓	✓	N...			M...																											
3.	Pengguna mengakses halaman aplikasi tanpa disertai <i>refresh token</i> JWT	 																																				

No.	Deskripsi Pengujian	Gambar
4.	<i>Node</i> mengirim data sensor menggunakan token yang benar	
5.	<i>Node</i> mengirim data sensor menggunakan token yang kadaluarsa	
6.	<i>Node</i> mengirim data sensor menggunakan token yang dimodifikasi	

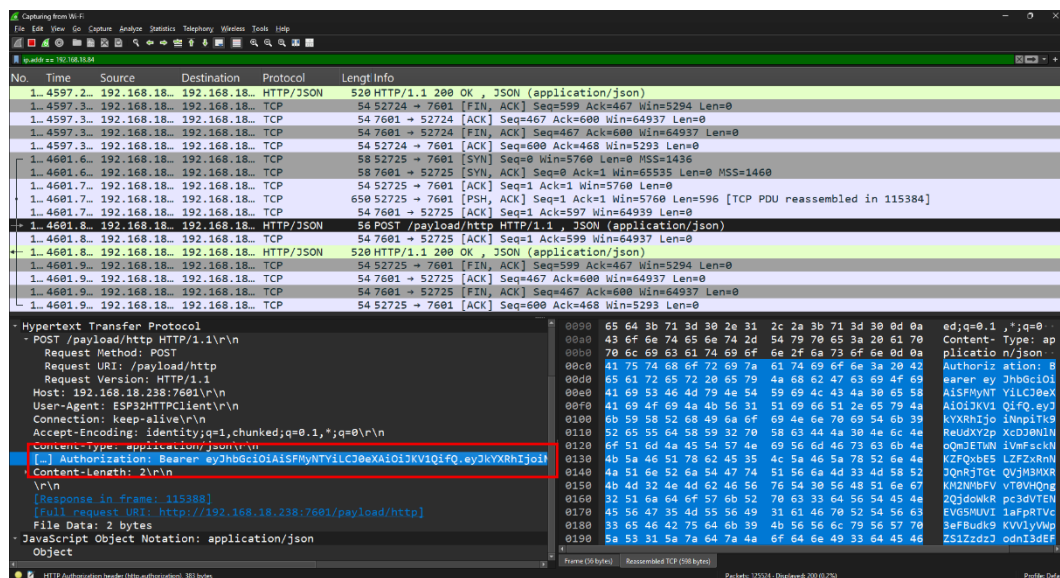
Pada pengujian pertama, pengguna berhasil mengakses halaman aplikasi dengan menggunakan token JWT yang benar. Sistem menampilkan *dashboard* utama dengan grafik pemantauan limbah cair. Hal ini membuktikan bahwa sistem autentikasi berfungsi dengan baik ketika token yang digunakan masih aktif dan belum kadaluarsa. Komunikasi antara klien dan *server* berjalan lancar tanpa ada pesan error atau penolakan akses.

Pengujian kedua menunjukkan hasil yang berbeda ketika token JWT dimodifikasi secara manual. Sistem langsung mendeteksi bahwa token telah diubah dan menolak akses dengan menampilkan pesan "Akses ditolak!" disertai ikon keamanan yang menandakan adanya pelanggaran autentikasi. Respons sistem ini mengonfirmasi bahwa mekanisme verifikasi integritas token berfungsi dengan baik, di mana setiap

perubahan pada struktur atau isi token akan langsung terdeteksi melalui proses validasi *signature*.

Pada pengujian ketiga, ketika pengguna mencoba mengakses aplikasi tanpa menyertakan token JWT dalam request, sistem kembali menampilkan pesan "Akses ditolak!". Hal ini menunjukkan bahwa sistem telah mengimplementasikan mekanisme autentikasi yang tepat, di mana setiap permintaan yang masuk akan diperiksa tokennya sebelum diizinkan mengakses halaman yang dilindungi.

Pengujian nomor 4, 5, dan 6 menunjukkan berbagai skenario respons dari sisi *server* saat menerima data sensor dari *node*. Terlihat bahwa *server* dapat membedakan antara token yang benar, token yang kadaluwarsa, dan token yang dimodifikasi. Setiap kondisi menghasilkan respons yang sesuai, baik berupa pengiriman data sensor untuk token yang valid maupun penolakan akses untuk token yang bermasalah.

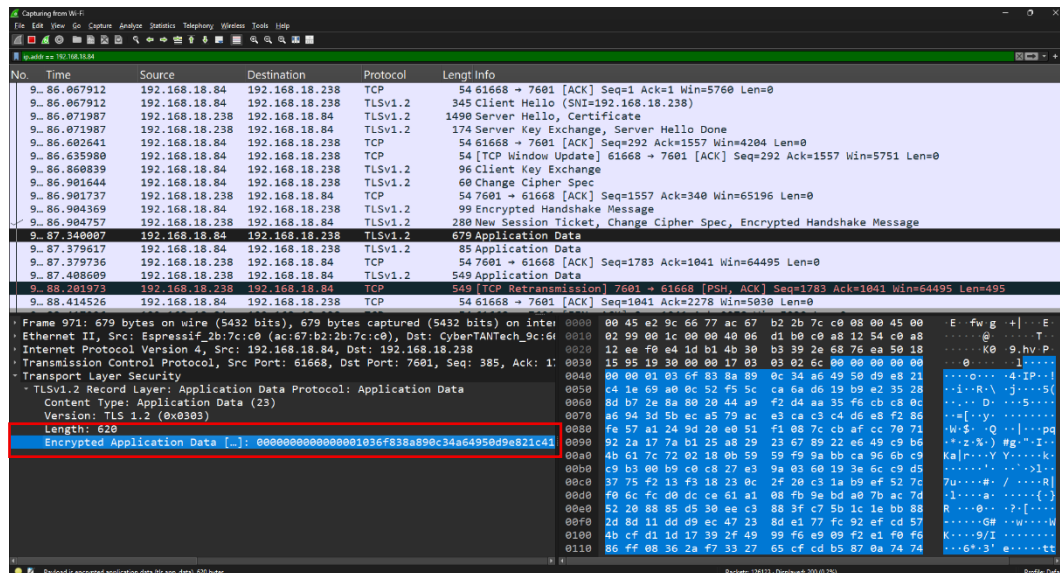


Gambar 4.33 *Payload* HTTP pada Aplikasi Wireshark

(Sumber: Dokumentasi pribadi)

Pengiriman data sensor pada pengujian-pengujian ini dilakukan masing-masing dua kali menggunakan protokol yang berbeda, yaitu HTTP dan HTTPS. Gambar 4.33 menunjukkan hasil *capture* paket menggunakan aplikasi Wireshark ketika data dikirim melalui protokol HTTP biasa. Terlihat jelas bahwa seluruh *payload* HTTP, termasuk token JWT yang berisi informasi sensitif, dapat dibaca dengan mudah

dalam bentuk *plain text*. Hal ini membuat token JWT rentan terhadap serangan MITM dan *packet sniffing*, di mana penyerang dapat dengan mudah mengintip dan mencuri token yang sedang ditransmisikan.



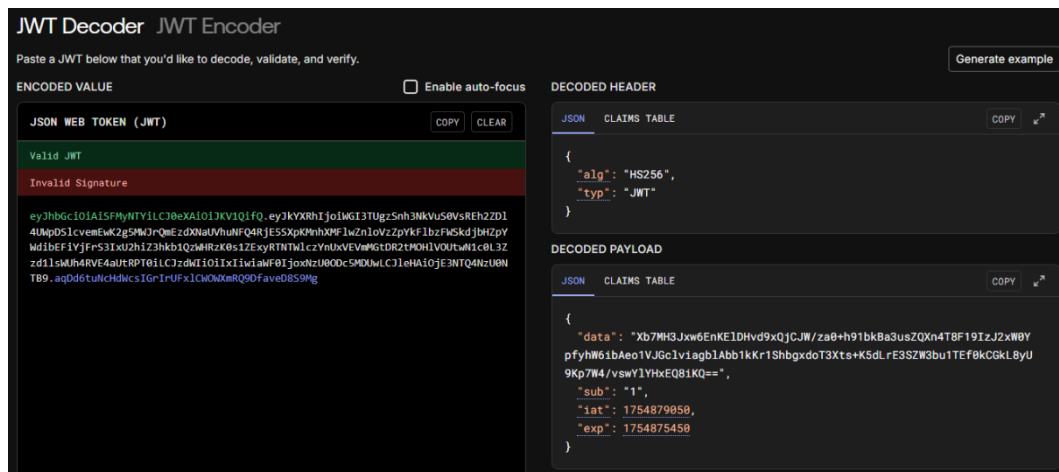
Gambar 4.34 Payload HTTPS pada Aplikasi Wireshark

(Sumber: Dokumentasi pribadi)

Sebaliknya, pada Gambar 4.34 yang menunjukkan pengiriman data melalui protokol HTTPS, terlihat bahwa seluruh *payload* aplikasi telah terenkripsi menggunakan TLS. Data yang sebelumnya dapat dibaca dengan jelas pada HTTP, kini tampil sebagai rangkaian karakter terenkripsi yang tidak dapat dipahami tanpa kunci dekripsi yang sesuai. Enkripsi ini memastikan bahwa token JWT dan data sensitif lainnya terlindungi selama proses transmisi, sehingga meskipun paket data berhasil dicegat oleh pihak yang tidak berwenang, informasi di dalamnya tetap tidak dapat diakses atau digunakan untuk tujuan yang merugikan.

Gambar 4.35 menunjukkan *log* dari *server* yang berisi tentang bagaimana data sensor dari *node* diproses oleh *server*. *Server* mengawali dengan melakukan pemeriksaan atribut pada "Authorization" pada *header* permintaan, yang harus memuat token JWT dengan format standar, yaitu *Bearer* <eyJ...>. Setelah memastikan bahwa format token JWT telah sesuai, *server* melanjutkan proses dengan melakukan verifikasi terhadap token tersebut.

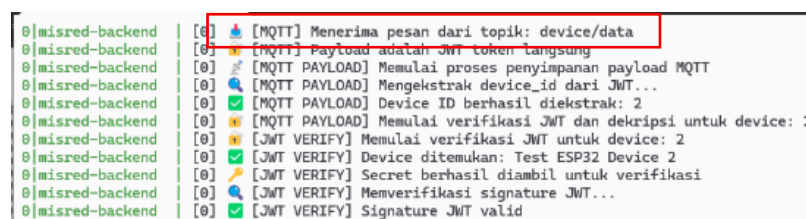
(Sumber: Dokumentasi pribadi)



(Sumber: Dokumentasi pribadi)

Proses verifikasi token diawali dengan melakukan *decoding* terhadap token JWT untuk memperoleh informasi yang terdapat pada bagian *payload*, khususnya nilai *sub*, yang merepresentasikan ID unik perangkat yang telah terdaftar pada sistem.

Gambar 4.6 menunjukkan tampilan visual dari token saat berhasil di-*decode*. Berdasarkan ID tersebut, *server* akan mengambil nilai *secret key* yang terasosiasi dengan perangkat dari *database*. Selanjutnya, *server* menggunakan nilai *secret key* tersebut untuk memverifikasi tanda tangan digital (*signature*) dari token JWT yang dikirim oleh perangkat ESP32. Apabila proses verifikasi tanda tangan berhasil, *server* akan melanjutkan pemrosesan data lebih lanjut agar dapat menyimpan data *sensor* dengan benar pada tabel *payloads*, termasuk melakukan dekripsi AES pada data sensor, hingga pemeriksaan alarm. Hal ini juga berlaku sama pada pengiriman menggunakan protokol MQTT, *server* akan memproses data yang dikirimkan oleh *node* pada topik yang sudah ditentukan. Gambar 4.7 menunjukkan *log* dari *server* saat menerima data dari perangkat MQTT yang melakukan *publish* ke topik "*device/data*".



```

0|misred-backend | [0] [MQTT] Menerima pesan dari topik: device/data
0|misred-backend | [0] [MQTT] Payload adalah JWT token langsung
0|misred-backend | [0] [MQTT PAYLOAD] Memulai proses penyimpanan payload MQTT
0|misred-backend | [0] [MQTT PAYLOAD] Mengekstrak device_id dari JWT...
0|misred-backend | [0] [MQTT PAYLOAD] Device ID berhasil diekstrak: 2
0|misred-backend | [0] [MQTT PAYLOAD] Memulai verifikasi JWT dan dekripsi untuk device: 2
0|misred-backend | [0] [JWT VERIFY] Memulai verifikasi JWT untuk device: 2
0|misred-backend | [0] [JWT VERIFY] Device ditemukan: Test ESP32 Device 2
0|misred-backend | [0] [JWT VERIFY] Secret berhasil diambil untuk verifikasi
0|misred-backend | [0] [JWT VERIFY] Memverifikasi signature JWT...
0|misred-backend | [0] [JWT VERIFY] Signature JWT valid

```

Gambar 4.37 Proses pemeriksaan *payload* MQTT

(Sumber: Dokumentasi Pribadi)

Proses dekripsi data terenkripsi yang dikirimkan oleh perangkat IoT dilakukan untuk mengubah *ciphertext* menjadi *plaintext* yang dapat dibaca dan diproses oleh sistem. Data yang diterima berada dalam format *Base64*, sehingga langkah pertama yang dilakukan adalah melakukan *decoding* Base64 untuk memperoleh data biner asli. Data biner tersebut terdiri dari dua bagian utama, yaitu:

1. *Initialization Vector* (IV) – diambil dari 16 *byte* pertama hasil *decoding*. IV diperlukan pada algoritma AES-CBC untuk proses pembalikan enkripsi pada blok pertama.
2. *Ciphertext* (C_i) – sisa data setelah *byte* ke-16 hingga akhir, yang merupakan hasil enkripsi data sensor.

Secara konseptual, proses ini dapat digambarkan sebagai berikut:

Base64Decode(

Xb7MH3Jxw6EnKEIDHvd9xQjCJW/za0+h91bkBa3usZQXn4T8F19IzJ2xW0Yp

fyhW6ibAeo1VJGclviagblAbb1kKr1ShbgxdoT3Xts+K5dLrE3SZW3bu1TEf0kC
GkL8yU9Kp7W4/vswYIYHxEQ8iKQ==
) = [IV] [C₁] [C₂] [C₃] [C₄] [C₅] [C₆]

Terdapat enam blok *ciphertext* dikarenakan *node* mengirimkan enam data. Selanjutnya, *server* menyiapkan JWT *secret* milik perangkat untuk proses dekripsi. Dalam analisis ini, penulis hanya akan menganalisis blok *ciphertext* pertama. Nilai *secret* (K) dan IV, serta blok *ciphertext* pertama yang digunakan pada pengujian ini adalah:

$$K = 23050c3dcef3c669690aab113a21c3b2$$

$$IV = 5dbecc1f7271c3a1272849431ef77dc5$$

$$C_1 = 08c2256ff36b4fa1f756e405adeeb194$$

Proses dekripsi kemudian dilakukan menggunakan algoritma AES-128 dalam mode operasi CBC (D) menggunakan fungsi "*createDecipheriv*" dari *library* bernama *crypto*, dengan nilai IV dan K yang telah ditentukan. Algoritma AES-CBC kemudian memproses blok-blok data secara berantai, mengikuti Persamaan (2.2). Pada blok pertama, hasil dekripsi di-XOR dengan blok IV, sedangkan pada blok-blok berikutnya, hasil dekripsi di-XOR dengan blok *ciphertext* sebelumnya. Contoh perhitungan pada blok pertama adalah sebagai berikut:

$$P_1 = D(C_1, K) \oplus C_{1-1} \dots\dots\dots (2.2)$$

$$P_1 = \text{createDecipheriv}(\text{"aes-128-cbc"}, \\ 08c2256ff36b4fa1f756e405adeeb194, 23050c3dcef3c669690aab113a21c3b2 \\) \oplus 5dbecc1f7271c3a1272849431ef77dc5$$

$$P_1 = 7b0a225630223a20372e30382c0a225631223a200a$$

Hasil P_1 kemudian diubah ke format ASCII menjadi: " { "V0":7.08,"V1":. ". Untuk membaca keseluruhan data secara lengkap, dilanjutkanlah proses dekripsi hingga blok keenam (P_6). Setelah semua blok selesai diproses, *padding* dihapus untuk mendapatkan *plaintext* asli. Hasil akhir dekripsi berupa *plaintext* dalam format JSON, yang berisi data pembacaan sensor dari perangkat IoT sebagai berikut:

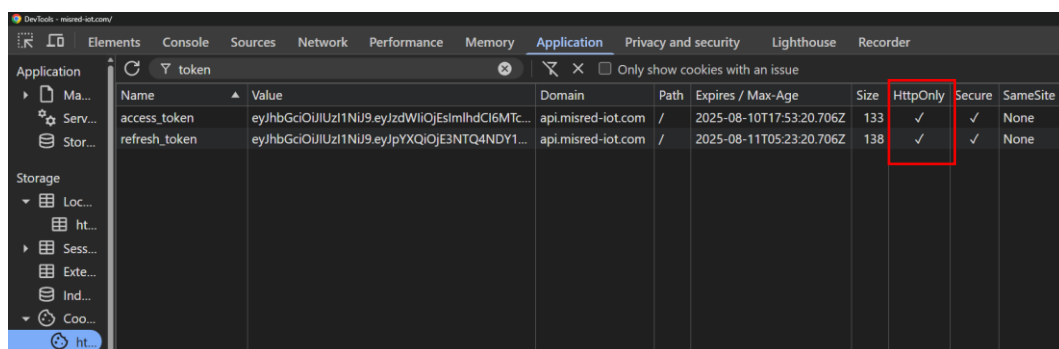

```

{
  "V0": 7.08,
  "V1": 31.37,
  "V2": 61.19,
  "V3": 21.62,
  "V4": 2.69,
  "V5": 15.78,
  "timestamp": 1754239266
}

```

Data "V0" hingga "V5" pada hasil dekripsi merupakan data hasil pengukuran dari masing-masing sensor yang telah didaftarkan pada halaman *datastream*, sedangkan "timestamp" menunjukkan waktu pengambilan data dalam format *epoch time*. Data ini merupakan data yang dibentuk oleh *node* sebelum melakukan proses enkripsi. Dengan mekanisme ini, data yang dikirim secara terenkripsi dapat dipastikan keasliannya serta tetap terjaga kerahasiaannya selama proses transmisi.

Pada kasus verifikasi pengguna, *server* juga akan melakukan autentikasi dan otorisasi melalui token JWT yang digunakan oleh pengguna saat mengakses aplikasi. Mekanisme ini memastikan bahwa hanya pengguna yang telah terverifikasi yang dapat mengakses sumber daya sistem.



Gambar 4.38 Tampilan Menu *Cookie* pada *Browser* Pengguna

(Sumber: Dokumentasi pribadi)

Setelah pengguna berhasil melakukan proses *login*, sistem akan memberikan dua jenis token JWT, yaitu *access token* dan *refresh token*. Kedua token tersebut disimpan dalam *cookie browser* dengan konfigurasi *HttpOnly = true*. Konfigurasi

ini memiliki peran yang sangat krusial dalam konteks keamanan, karena memastikan bahwa token JWT tidak dapat diakses secara langsung melalui *JavaScript* di sisi klien. Dengan demikian, token hanya dapat digunakan melalui permintaan HTTP yang dikirim ke *endpoint* API pada *api.misred-iot.com*, sehingga mengurangi risiko penyalahgunaan token oleh pihak yang tidak berwenang.

Sebagai bagian dari upaya untuk memastikan keamanan sistem secara menyeluruh, dilakukan pula pengujian terhadap kekuatan *secret key* yang digunakan dalam penandatanganan token JWT. Pengujian ini mencakup dua aspek utama, yaitu (1) evaluasi tingkat *entropy* sebagai indikator kerandoman *secret*, dan (2) estimasi ketahanan terhadap serangan *brute force*, yang menggambarkan kemampuan kunci untuk bertahan terhadap upaya pemecahan secara sistematis.

Pada sistem yang dikembangkan, *secret key* untuk JWT dibuat secara otomatis oleh *server* dalam bentuk string acak heksadesimal sepanjang 32 karakter. Contoh *secret key* yang diuji adalah sebagai berikut:

4b3d6e7c94148b64022cece554fe79a1

Karakteristik teknis dari *secret* tersebut dapat dijabarkan sebagai berikut:

- a. Panjang (L) = 32 karakter
- b. Karakter set = hexadecimal (0–9, a–f)
- c. Jumlah karakter unik (N) = 16

Tingkat *entropy* dari *secret* dihitung menggunakan rumus *entropy* Shannon seperti yang disajikan dalam Persamaan (2.3):

$$H = L \cdot \log_2(N) \quad \dots\dots\dots (2.3)$$

$$H = 32 \cdot \log_2(16) = 32 \cdot 4 = 128 \text{ bit}$$

Nilai *entropy* sebesar 128 bit ini telah memenuhi rekomendasi minimal untuk kekuatan kunci kriptografi yang aman, sebagaimana diusulkan oleh Grassi dkk. (2017) dan Barker (2020), serta sesuai dengan standar industri untuk sistem autentikasi dan komunikasi data. Berdasarkan hasil perhitungan ini, dapat disimpulkan bahwa *secret key* memiliki tingkat kerandoman yang tinggi dan tidak

rentan terhadap pendekatan statistik maupun prediktif. Selain pengukuran *entropy*, dilakukan pula simulasi terhadap ketahanan *secret* terhadap serangan *brute force* yakni skenario di mana penyerang mencoba melakukan enumerasi untuk menebak nilai *secret* dengan asumsi telah memiliki akses terhadap token JWT. Jumlah kemungkinan kombinasi *secret* dihitung dengan rumus seperti pada Persamaan (2.4):

$$C = N^L \dots\dots\dots (2.4)$$

$$C = N^L = 16^{32} = 2^{128} \approx 3.4 \times 10^{38} \text{kemungkinan}$$

Jika diasumsikan penyerang menggunakan perangkat keras yang mampu melakukan satu miliar atau 10^9 tebakan per detik (R), maka estimasi waktu untuk menjangkau seluruh kemungkinan kombinasi *secret* dapat dihitung dengan Persamaan (2.5):

$$T = \frac{C}{R} \dots\dots\dots (2.5)$$

$$T = \frac{2^{128}}{10^9} \approx 10^{29} \text{ detik} \approx 10^{21} \text{ tahun}$$

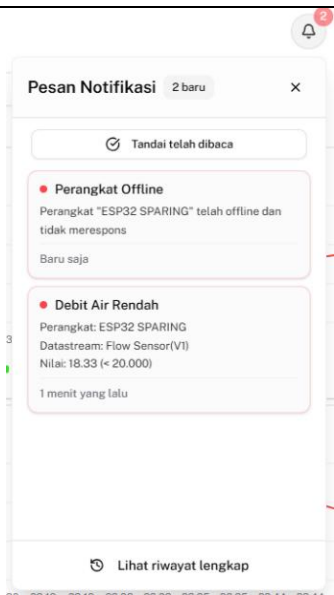
Hasil estimasi tersebut menunjukkan bahwa pendekatan *brute force* terhadap *secret* dengan panjang 32 karakter heksadesimal berada di luar jangkauan kemampuan komputasi konvensional maupun superkomputer dalam waktu yang realistis.

Berdasarkan kedua metode evaluasi yang dilakukan, yaitu analisis *entropy* dan simulasi *brute force*, dapat disimpulkan bahwa penggunaan *secret key* JWT sepanjang 32 karakter heksadesimal yang dihasilkan secara acak oleh *server* telah memenuhi aspek dasar keamanan sistem autentikasi. Nilai *entropy* sebesar 128 bit memberikan perlindungan yang memadai terhadap serangan tebakan acak (*guessing attack*), sementara simulasi *brute force* memperlihatkan bahwa kompleksitas kunci sangat tinggi dan tidak mudah dieksploitasi. Pengujian ini memperkuat keyakinan bahwa sistem yang dibangun memiliki mekanisme keamanan yang solid dalam menangani autentikasi perangkat maupun pengguna, serta mampu menjaga integritas data yang dikirimkan melalui jaringan.

4.3 Hasil Pengujian Notifikasi

Pengujian notifikasi dilakukan untuk menilai kemampuan sistem dalam memberikan respons secara *real-time* melalui tampilan notifikasi pada *browser* dan perangkat Android ketika parameter limbah cair berada di luar batas normal. Tujuan dari pengujian ini adalah untuk memastikan bahwa sistem dapat merespons kondisi parameter lingkungan yang berada di bawah ambang batas yang ditetapkan, serta menyampaikan peringatan secara akurat dan tepat waktu. Hasil pengujian terhadap masing-masing skenario kondisi parameter ditampilkan pada Tabel 4.2.

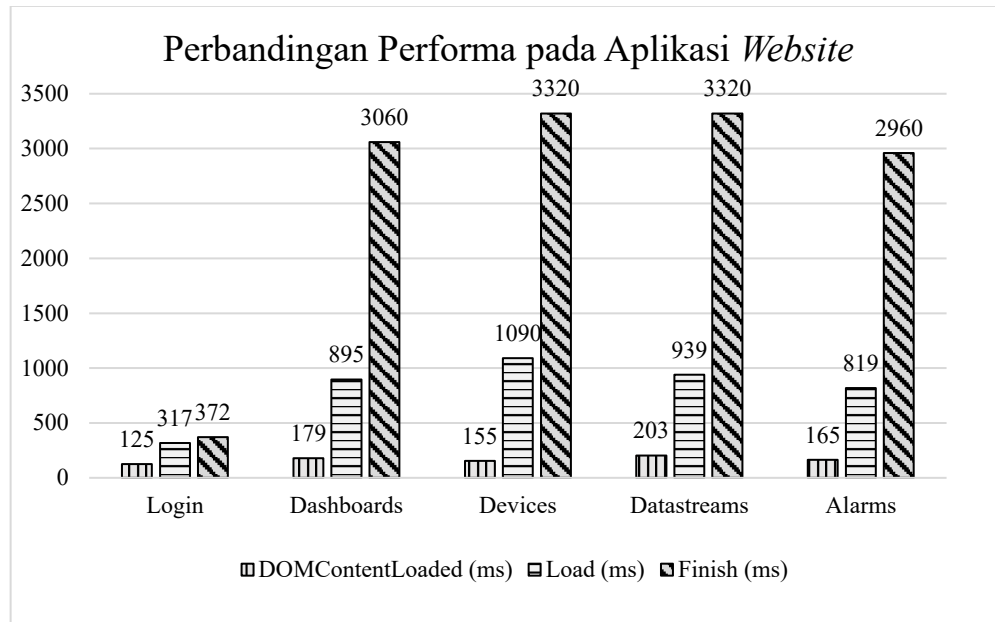
Tabel 4.2 Hasil pengujian notifikasi

No.	Deskripsi Pengujian	Gambar
1.	Menampilkan pesan notifikasi pada <i>browser</i> saat ada alarm yang terpicu atau ada peringatan perangkat yang <i>offline</i>	 The screenshot shows a notification box titled "Pesan Notifikasi" with a close button (X) and a "Tandai telah dibaca" (Mark as read) checkbox. It contains two notifications: 1. "Perangkat Offline" (Device Offline) stating that the "ESP32 SPARING" device is offline and unresponsive, with a "Baru saja" (Just now) timestamp. 2. "Debit Air Rendah" (Low Water Discharge) stating the device is "ESP32 SPARING", the datastream is "Flow Sensor(VI)", the value is "18.33 (< 20.000)", and it was received "1 menit yang lalu" (1 minute ago). At the bottom is a "Lihat riwayat lengkap" (View full history) link.
2.	Menampilkan pesan notifikasi pada nomor WhatsApp saat ada alarm yang terpicu atau ada peringatan perangkat yang <i>offline</i>	 The screenshot shows a WhatsApp chat interface with a contact named "Aku (Anda)". It displays two automated messages: 1. "PERINGATAN SENSOR ALARM" (Alarm Sensor Warning) with details: Alarm: Debit Air Rendah, Perangkat: ESP32 SPARING, Sensor: Flow Sensor(VI), Nilai Saat Ini: 18.33, Kondisi: VI < 20.000, Akun: admin@misred.com, Waktu: 11/8/2025, 00.04.37 WIB. It ends with "Mohon segera melakukan pengecekan!" (Please check immediately!). 2. "PERINGATAN STATUS PERANGKAT" (Device Status Warning) with details: Perangkat: ESP32 SPARING, Status: OFFLINE, Akun: Admin MiSREd, Email: admin@misred.com, Waktu: 11/8/2025, 00.05.49 WIB. It ends with "Mohon periksa koneksi perangkat Anda!" (Please check your device connection!). Both messages have a timestamp and a double-checkmark icon.

Pengguna akan menerima notifikasi apabila nilai suatu parameter melebihi atau berada di bawah ambang batas yang telah ditentukan sebelumnya melalui halaman *Alarms*. Notifikasi ini dapat disampaikan melalui dua media, yaitu melalui tampilan pada *browser* maupun melalui aplikasi WhatsApp. Pada tampilan *browser*, notifikasi akan muncul di pojok kanan atas halaman dalam bentuk ikon lonceng. Sementara itu, notifikasi melalui WhatsApp hanya akan dikirim apabila pengguna telah menambahkan nomor telepon yang terhubung dengan akun WhatsApp pada bagian *pengaturan profil akun* serta mengaktifkan fitur notifikasi tersebut.

4.4 Hasil Pengujian Performa

Pengujian performa dilakukan untuk mengidentifikasi sejauh mana sistem yang dikembangkan mampu beroperasi secara efisien dalam memanfaatkan sumber daya perangkat, baik pada platform *website* maupun aplikasi Android. Tujuan dari pengujian ini adalah untuk mengukur tingkat penggunaan memori, CPU, serta jaringan, guna memastikan bahwa sistem dapat berjalan secara optimal dalam berbagai kondisi operasional. Pada aplikasi berbasis *website*, pengujian dilakukan dengan memanfaatkan fitur *Network Monitor* pada *browser* Google Chrome untuk memperoleh data waktu pemuatan halaman, meliputi waktu pemuatan struktur HTML (*DOMContentLoaded*), waktu pemuatan halaman utama tanpa ikon (*load*), serta waktu pemuatan keseluruhan (*finish*). Sementara itu, pengujian performa pada aplikasi Android menggunakan fitur *Android Profiler* pada Android Studio, yang memberikan informasi terkait penggunaan CPU dan memori, selama aplikasi dijalankan. Hasil pengujian tersebut disajikan dengan grafik pada Gambar 4.39. Pada saat pengujian ini dilakukan, nilai dari *bandwidth* jaringan adalah 43,7 Mbps.



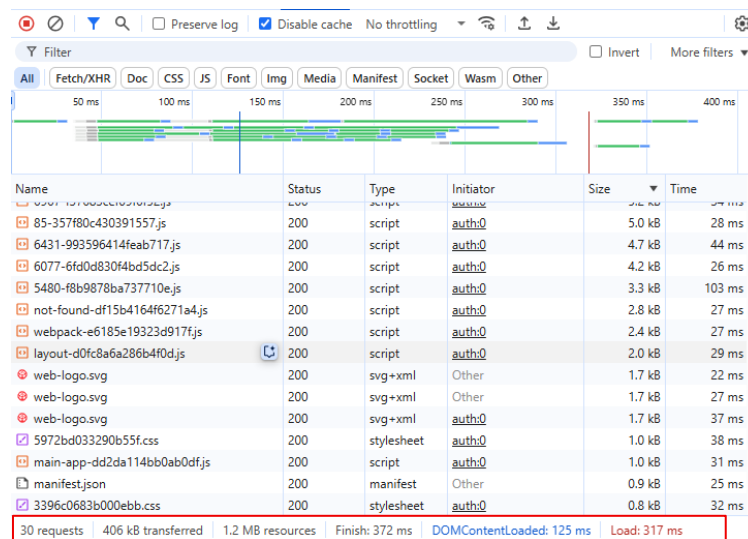
Gambar 4.39 Hasil pengujian performa *website*

(Sumber: Dokumentasi pribadi)

Berdasarkan hasil pengujian performa aplikasi *website* yang ditampilkan pada grafik, sebagaimana ditunjukkan pada Gambar 4.39. Pengujian dilakukan terhadap lima halaman utama, yaitu:

1. Halaman *Login*

Halaman ini berfungsi untuk melakukan autentikasi pengguna melalui input kredensial berupa *email* dan kata sandi. Hasil dari “*Network Monitor*” ditunjukkan pada Gambar 4.40.



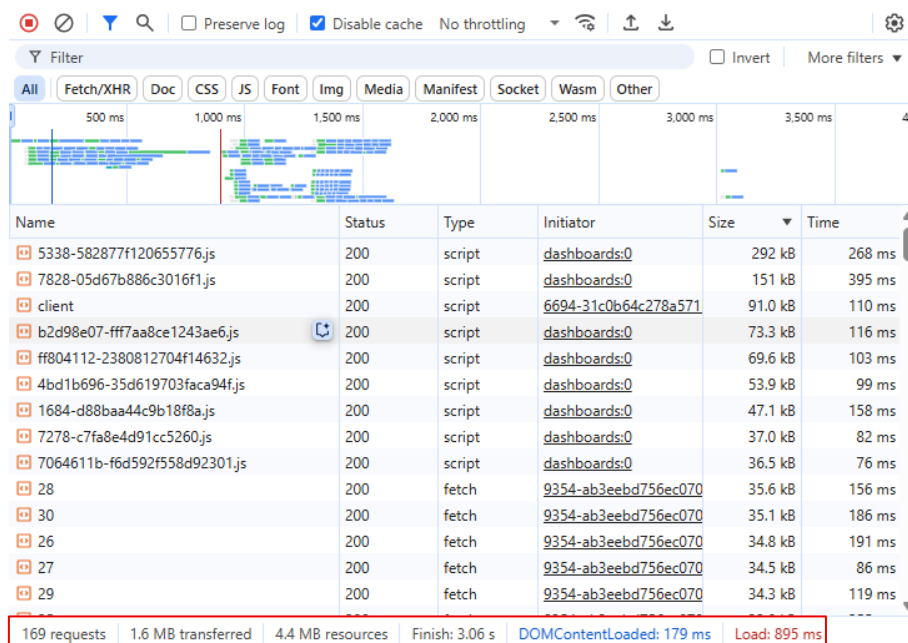
Gambar 4.40 Hasil dari “*Network Monitor*” halaman *login*

(Sumber: Dokumentasi pribadi)

Berdasarkan hasil di atas, terdapat 30 permintaan yang dibutuhkan untuk memuat halaman, diantaranya yaitu beberapa *file* JavaScript, CSS, *font*, logo, dan satu dokumen untuk halaman *auth*. Keseluruhan halaman berukuran 1,2 MB dengan *file* terbesarnya adalah dokumen *auth* yang berukuran 0,5 kB. Berdasarkan pengujian waktu muat, halaman *Login* memiliki performa paling optimal dengan waktu *DOMContentLoaded* sebesar 125 ms, *Load* sebesar 317 ms, dan *Finish* sebesar 372 ms. Hal ini menunjukkan bahwa halaman *Login* relatif ringan dan tidak memiliki proses dinamis tambahan setelah halaman dimuat secara visual.

2. Halaman *Dashboards*

Halaman ini menampilkan grafik data dari parameter-parameter yang telah ditambahkan oleh pengguna. Hasil dari “*Network Monitor*” ditunjukkan pada Gambar 4.41.



Gambar 4.41 Hasil dari “*Network Monitor*” halaman *dashboards*

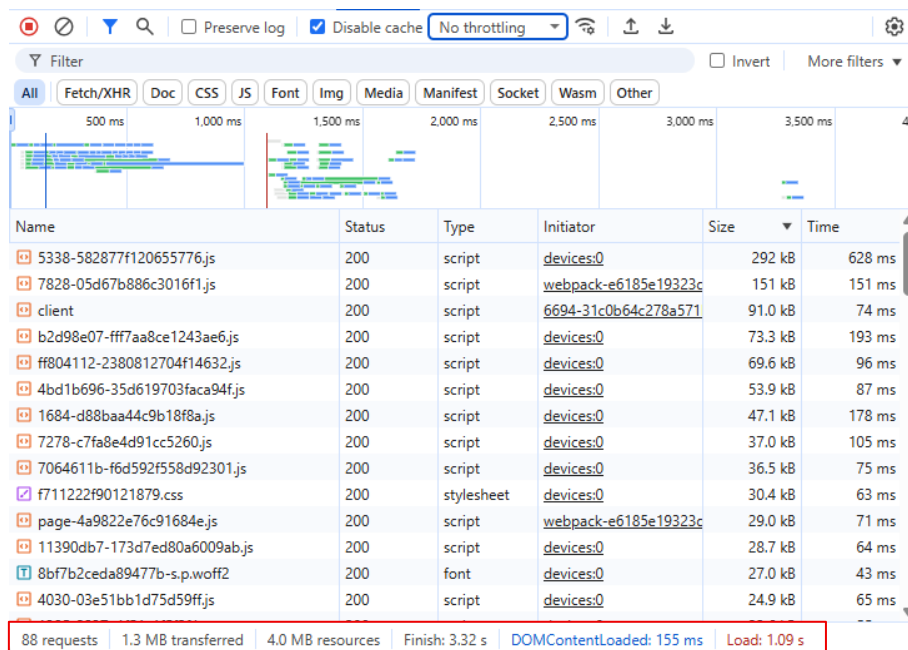
(Sumber: Dokumentasi pribadi)

Berdasarkan hasil di atas, terdapat 169 permintaan yang dibutuhkan untuk memuat halaman, diantaranya yaitu beberapa *file* JavaScript, CSS, *font*, logo, permintaan data, dan satu dokumen untuk halaman *dashboard*. Permintaan data dilakukan untuk mengambil ringkasan parameter suhu, debit air, kekeruhan air, pH air, NH3-N, dan COD. Keseluruhan halaman berukuran 4,4 MB dengan *file*

terbesarnya adalah *file* JavaScript yang berukuran 292 kB dengan total waktu yang dibutuhkan browser untuk menyelesaikan permintaan adalah 268 ms. Namun demikian, pengujian performa menunjukkan bahwa waktu *DOMContentLoaded* adalah 179 ms, *Load* sebesar 895 ms, dan *Finish* mencapai 3060 ms. Perbedaan signifikan antara waktu *load* dan *finish* menunjukkan adanya proses pemuatan data dinamis yang intensif setelah halaman dimuat secara visual, terutama dari permintaan API yang memuat grafik parameter.

3. Halaman *Devices*

Halaman ini menyajikan data dalam bentuk tabel mengenai perangkat IoT yang telah ditambahkan oleh pengguna. Hasil dari “*Network Monitor*” ditunjukkan pada Gambar 4.42.



Gambar 4.42 Hasil dari “*Network Monitor*” halaman *devices*

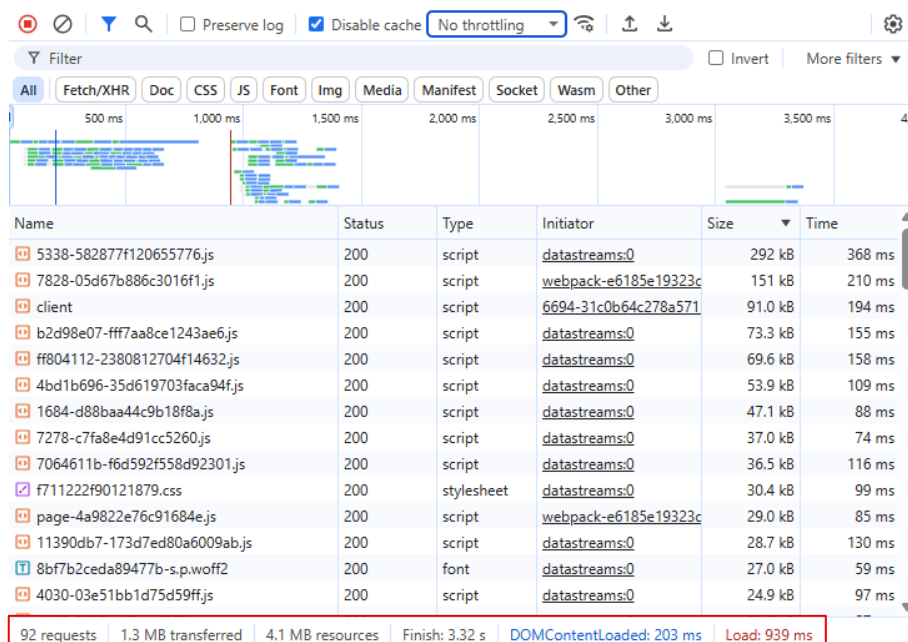
(Sumber: Dokumentasi pribadi)

Berdasarkan hasil di atas, terdapat 88 permintaan yang dibutuhkan untuk memuat halaman, diantaranya yaitu beberapa *file* JavaScript, CSS, *font*, logo, permintaan data, dan satu dokumen untuk halaman *devices*. Permintaan data dilakukan untuk memverifikasi token otorisasi dan mengambil data perangkat. Keseluruhan halaman berukuran 4 MB dengan *file* terbesarnya adalah *file* JavaScript yang berukuran 292 kB dengan total waktu yang dibutuhkan

browser untuk menyelesaikan permintaan adalah 628 ms. Sementara itu, hasil pengujian performa menunjukkan bahwa waktu *DOMContentLoaded* adalah 155 ms, *Load* sebesar 1090 ms, dan *Finish* sebesar 3320 ms. Seperti pada halaman *Dashboards*, waktu *finish* yang tinggi mengindikasikan adanya aktivitas jaringan tambahan, yang kemungkinan disebabkan oleh pemrosesan data tabel perangkat secara dinamis.

4. Halaman *Datastreams*

Halaman ini menampilkan data tabel dari parameter-parameter (*datastream*) perangkat IoT yang telah didaftarkan. Hasil dari “*Network Monitor*” ditunjukkan pada Gambar 4.43.



Gambar 4.43 Hasil dari “*Network Monitor*” halaman *datastreams*

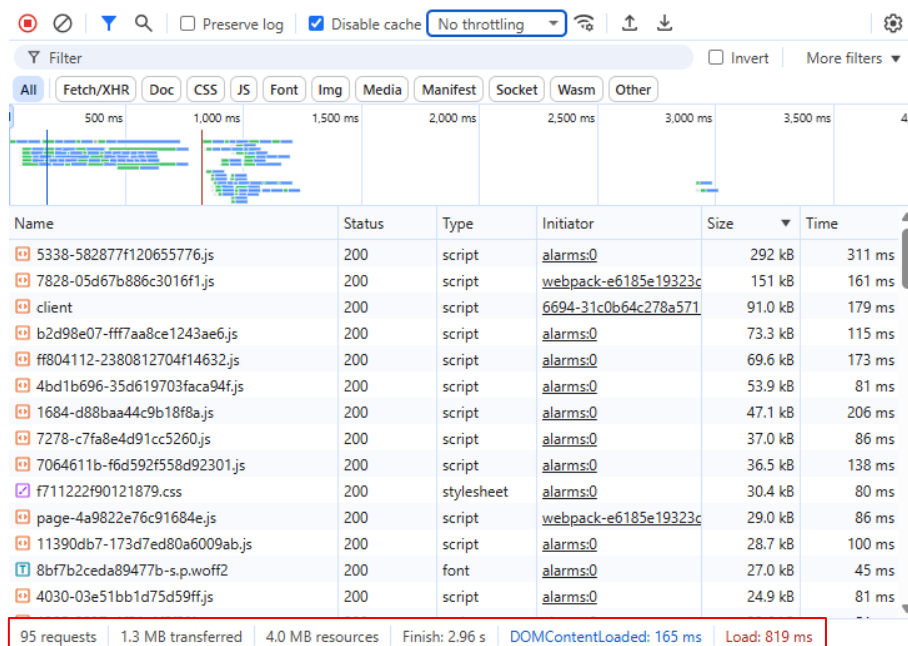
(Sumber: Dokumentasi pribadi)

Berdasarkan hasil di atas, terdapat 92 permintaan yang dibutuhkan untuk memuat halaman, diantaranya yaitu beberapa *file* JavaScript, CSS, *font*, logo, permintaan data, dan satu dokumen untuk halaman *datastreams*. Permintaan data dilakukan untuk memverifikasi token otorisasi dan mengambil data perangkat beserta parameteranya. Keseluruhan halaman berukuran 4,1 MB dengan *file* terbesarnya adalah *file* JavaScript yang berukuran 292 kB dengan total waktu yang dibutuhkan *browser* untuk menyelesaikan permintaan adalah 368 ms. Namun, hasil pengujian performa menunjukkan bahwa waktu

DOMContentLoaded adalah 203 ms, *Load* sebesar 939 ms, dan *Finish* mencapai 3320 ms. Hal ini mengindikasikan bahwa meskipun halaman dimuat secara visual dalam waktu yang wajar, proses pengambilan data dan *rendering* komponen *datastream* memerlukan waktu tambahan yang signifikan.

5. Halaman *Alarms*

Halaman ini menampilkan data tabel mengenai alarm, yaitu pengaturan ambang batas terhadap parameter dari perangkat IoT. Hasil dari “*Network Monitor*” ditunjukkan pada Gambar 4.44.



Gambar 4.44 Hasil dari “*Network Monitor*” halaman *alarms*

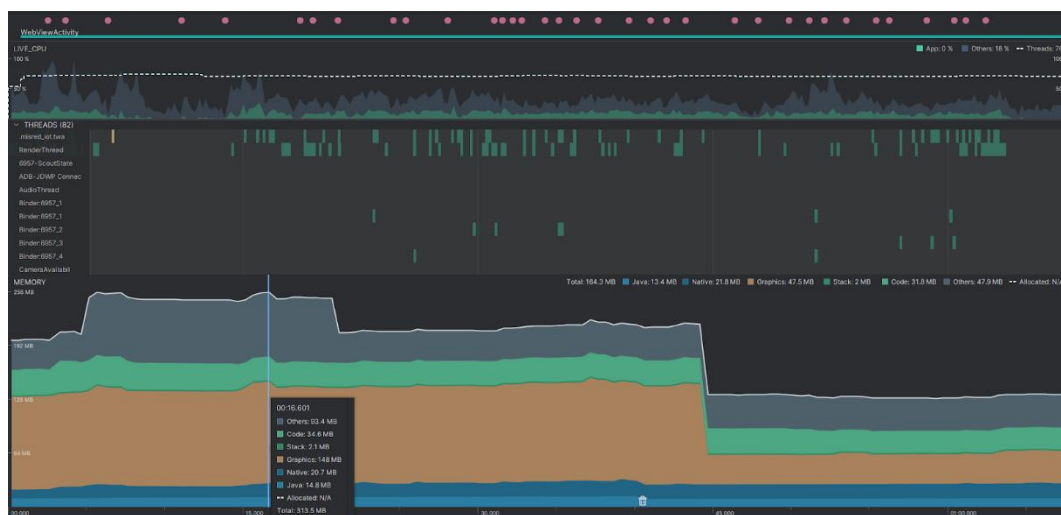
(Sumber: Dokumentasi pribadi)

Berdasarkan hasil di atas, terdapat 85 permintaan yang dibutuhkan untuk memuat halaman, diantaranya yaitu beberapa *file* JavaScript, CSS, *font*, logo, permintaan data, dan satu dokumen untuk halaman *alarms*. Permintaan data dilakukan untuk memverifikasi token otorisasi dan mengambil data alarm dari perangkat beserta parameternya. Keseluruhan halaman berukuran 4 MB dengan *file* JavaScript yang berukuran 292 kB dengan total waktu yang dibutuhkan browser untuk menyelesaikan permintaan adalah 311 ms. Pada sisi performa, halaman ini memiliki waktu *DOMContentLoaded* sebesar 165 ms, *Load* sebesar 819 ms, dan *Finish* sebesar 2960 ms. Meski waktu awal pemuatan tergolong cepat, lamanya waktu *finish* menandakan bahwa proses pengambilan

dan pemrosesan data alarm berjalan secara bertahap dan tidak langsung ditampilkan pada saat awal.

Secara keseluruhan berdasarkan Tabel 2.2, hasil pengujian menunjukkan bahwa waktu *DOMContentLoaded* di seluruh halaman tergolong cepat (<250 ms), yang berarti struktur HTML dapat dimuat dengan baik oleh *browser* dan termasuk dalam kategori “Sangat baik”. Namun, waktu *Finish* pada beberapa halaman utama seperti *Dashboards*, *Devices*, dan *Datastreams* cukup tinggi (>3000 ms) dan termasuk dalam kategori “Cukup”. Hal ini menunjukkan adanya beban proses dinamis seperti pemanggilan API, pemrosesan JavaScript, dan visualisasi data grafik atau tabel.

Pengujian pada perangkat Android dilakukan menggunakan perangkat lunak Android Studio, khususnya melalui fitur *Android Profiler* dengan memanfaatkan *task View Live Telemetry*. Namun, hasil yang diperoleh dari *Android Profiler* bersifat terbatas karena aplikasi Android yang dikembangkan menggunakan pendekatan *Trusted Web Activity* (TWA), yang pada dasarnya hanya membungkus situs *web* pada domain *https://misred-iot.com*. Akibatnya, seluruh aktivitas permintaan berlangsung di dalam lingkungan *WebView*, sehingga *Android Profiler* hanya dapat merekam dan menampilkan informasi terkait aktivitas CPU, *threads*, dan penggunaan memori. Hasil pengujian tersebut disajikan pada Gambar 4.45.



Gambar 4.45 Hasil pengujian *Android Profiler* dari Android Studio
(Sumber: Dokumentasi pribadi)

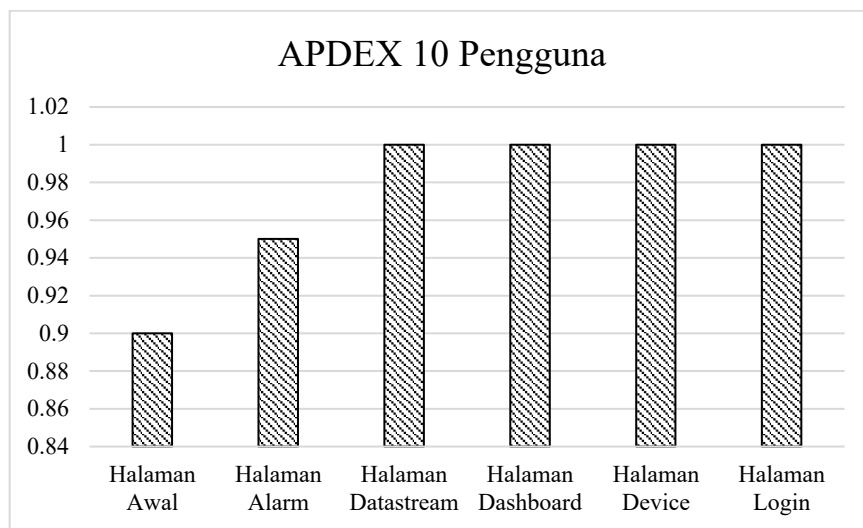
Berdasarkan hasil pengujian menggunakan fitur *View Live Telemetry* pada *Android Profiler*, diperoleh gambaran umum mengenai performa aplikasi selama dijalankan dalam mode TWA. Grafik menunjukkan bahwa aktivitas utama yang terekam meliputi penggunaan CPU, aktivitas *thread*, dan konsumsi memori. Penggunaan CPU terlihat relatif stabil dengan rata-rata di bawah 10%, menandakan bahwa aplikasi berjalan dengan beban kerja yang ringan tanpa adanya lonjakan proses yang signifikan. Pada bagian *threads*, tercatat aktivitas dari 82 *thread* sistem yang aktif, di antaranya adalah *RenderThread*, *Binder*, GCM (*Google Cloud Messaging*), *WebView*, serta komponen *CamerashellActivity* yang menunjukkan bahwa proses pemuatan antarmuka, komunikasi antarproses, dan pemrosesan *WebView* berjalan selama periode pengujian. Sementara itu, alokasi memori menunjukkan penurunan signifikan sekitar detik ke 45, dari total penggunaan sebesar ± 164 MB menjadi sekitar ± 120 MB. Rincian memori pada titik awal sebelum penurunan mencakup: Java sebesar 13,4 MB, *Native* sebesar 21.8 MB, *Graphics* sebesar 4,5 MB, *Stack* sebesar 2 MB, *Code* sebesar 3,1 MB, dan *Others* sebesar 47 MB. Setelah penurunan, total memori yang digunakan hanya berkisar di bawah 20 MB, mengindikasikan adanya proses *garbage collection* atau pembersihan memori oleh sistem.

Secara keseluruhan, performa aplikasi dalam lingkungan TWA dapat dikategorikan efisien dan stabil dari sisi sistem Android. Namun demikian, informasi yang ditampilkan terbatas pada lapisan Android karena seluruh aktivitas permintaan terjadi di dalam lingkungan *WebView*. Hal ini menyebabkan *profiler* tidak dapat merekam trafik jaringan atau aktivitas internal dari situs *web* yang dibungkus, sehingga tidak memberikan representasi menyeluruh terhadap perilaku aplikasi secara *end-to-end*.

4.5 Load Testing

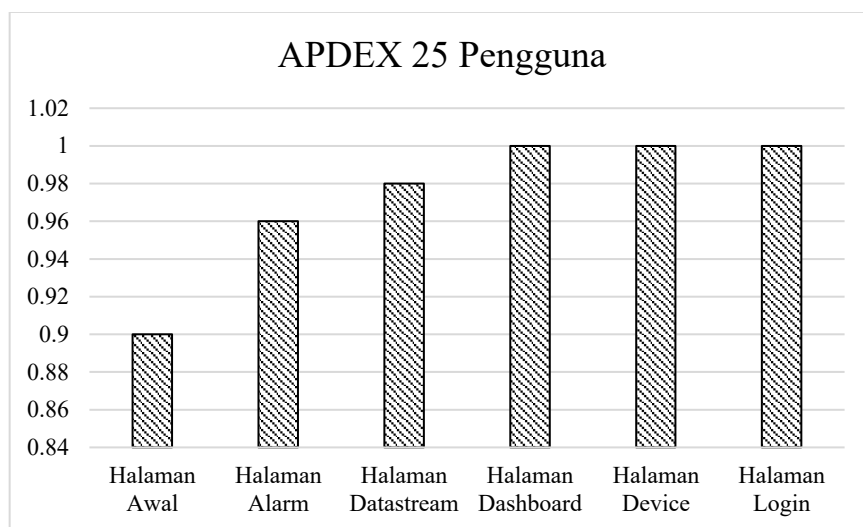
Pengujian *load testing* dilakukan pada halaman-halaman utama sistem, yaitu *landing page* (halaman awal), *login*, *dashboards*, *datastreams*, *devices*, dan *alarms*, dengan menggunakan tiga parameter yaitu *error rate*, *response time*, *throughput*, serta ditambahkan perhitungan APDEX sebagai indikator kepuasan pengguna. Analisis dilakukan dengan membandingkan hasil pengujian pada berbagai variasi

jumlah pengguna, yaitu 10, 25, 50, 100, dan 150 pengguna, sehingga dapat diketahui stabilitas dan performa sistem dalam menangani beban yang meningkat. Hasil pengujian juga divisualisasikan dalam bentuk grafik APDEX. Grafik ini memberikan representasi visual terhadap perubahan kualitas layanan pada setiap jumlah pengguna, sehingga memudahkan dalam melihat tren penurunan performa dan batas optimal kapasitas sistem. Gambar 4.46 hingga Gambar 4.50 secara berurutan menunjukkan hasil nilai APDEX terhadap 10, 25, 50, 100, dan 150 pengguna.



Gambar 4.46 Hasil APDEX terhadap 10 pengguna

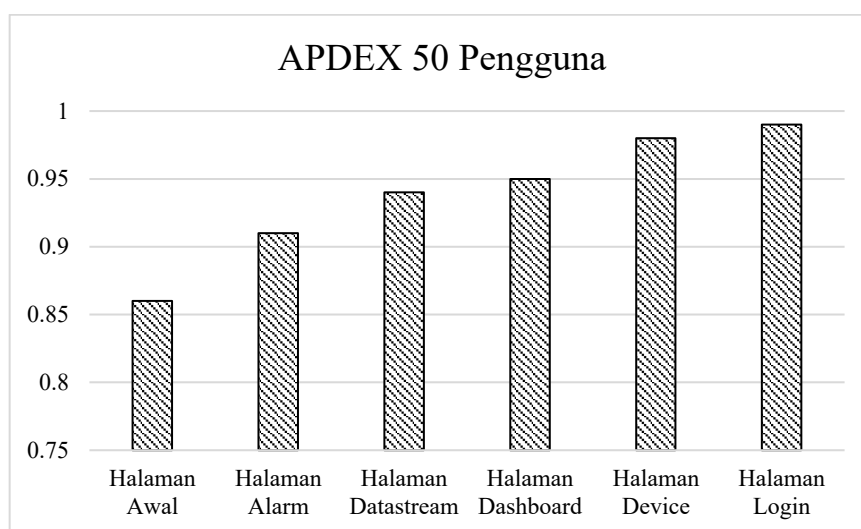
(Sumber: Dokumentasi pribadi)



Gambar 4.47 Hasil APDEX terhadap 25 pengguna

(Sumber: Dokumentasi pribadi)

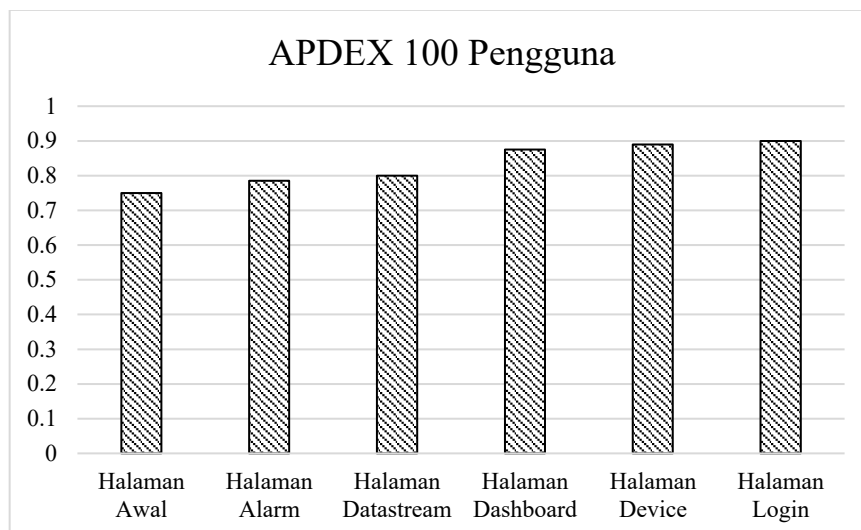
Hasil pengujian *load testing* dengan menggunakan parameter APDEX, *error rate*, *response time*, dan *throughput* menunjukkan bahwa sistem mampu melayani pengguna dengan performa yang cukup baik hingga jumlah tertentu. Pada saat diuji dengan 10 pengguna, nilai APDEX hampir mencapai 1.0 pada semua halaman, menandakan kualitas layanan yang sangat baik (*excellent*). Tidak terdapat *error* pada tahap ini, dan waktu respon rata-rata masih sangat rendah yaitu sekitar 177 milidetik (ms), dengan *throughput* sebesar 25,09 transaksi per detik. Halaman Awal tercatat memiliki waktu respon relatif lebih tinggi dibanding halaman lain, namun tetap berada dalam kategori baik. Ketika jumlah pengguna ditingkatkan menjadi 25, sistem masih menunjukkan kestabilan yang baik. APDEX tetap tinggi mendekati 1.0, *error rate* tetap 0%, dan rata-rata *response time* meningkat menjadi 269 ms. *Throughput* naik signifikan menjadi 46,71 transaksi per detik, menunjukkan bahwa sistem mampu menyesuaikan diri terhadap kenaikan beban akses.



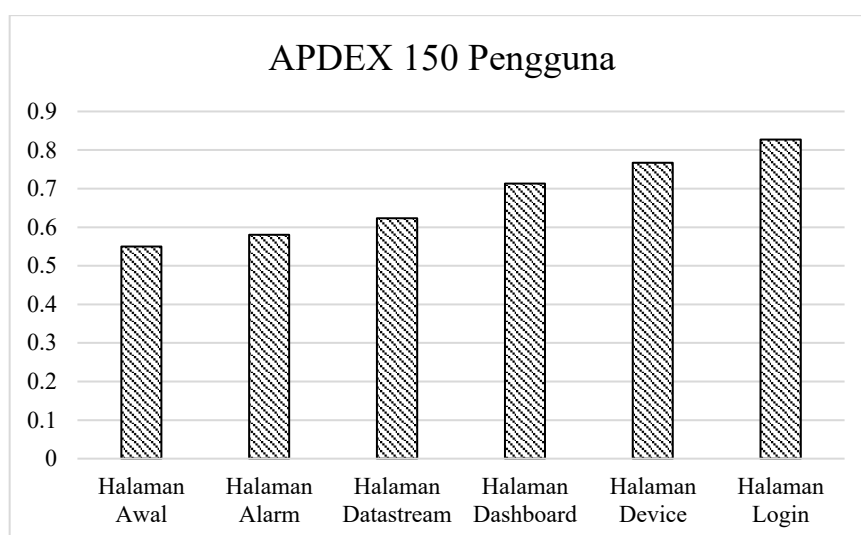
Gambar 4.48 Hasil APDEX terhadap 50 pengguna

(Sumber: Dokumentasi pribadi)

Pada pengujian dengan 50 pengguna, performa mulai menunjukkan tanda-tanda penurunan meskipun masih berada dalam kategori baik. Nilai APDEX pada beberapa halaman seperti Halaman Awal turun menjadi 0.86 dan Halaman *Alarms* 0.91, sementara *error rate* tetap 0%. Rata-rata *response time* meningkat menjadi 316 ms dengan *throughput* 85,71 transaksi per detik. Kondisi ini menunjukkan bahwa sistem masih dapat menampung beban hingga 50 pengguna dengan kualitas layanan yang memuaskan.



Gambar 4.49 Hasil APDEX terhadap 100 pengguna
(Sumber: Dokumentasi pribadi)



Gambar 4.50 Hasil APDEX terhadap 150 pengguna
(Sumber: Dokumentasi pribadi)

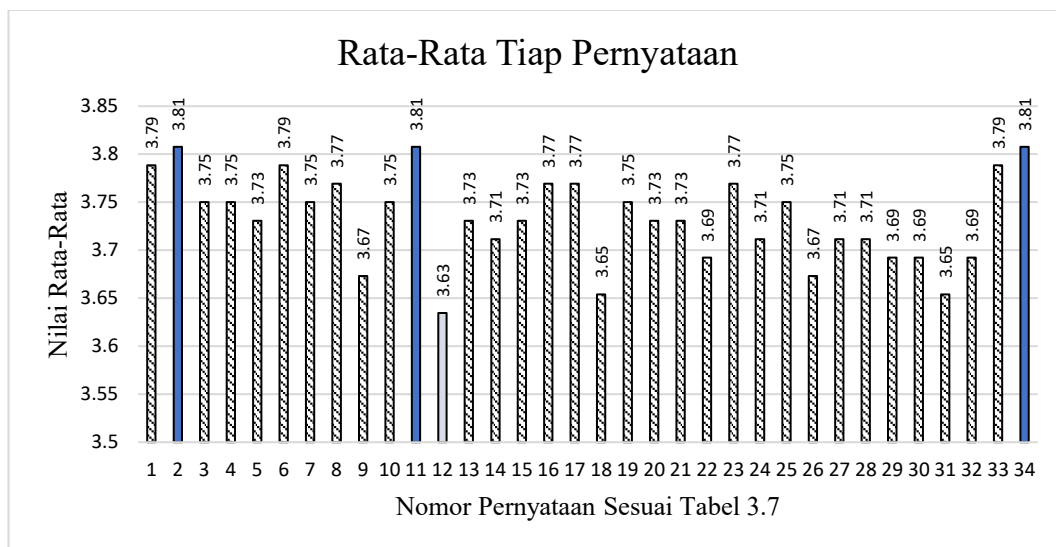
Perubahan yang lebih signifikan terlihat saat jumlah pengguna mencapai 100. Nilai APDEX mengalami penurunan lebih lanjut, dengan Halaman Awal hanya mencapai 0.76 (kategori *fair*) dan Halaman *Alarms* 0.79. Rata-rata *response time* meningkat menjadi 467 ms, dengan beberapa halaman seperti *Dashboards* mencapai 548 ms. Meskipun *throughput* meningkat menjadi 131,15 transaksi per detik, *response time* menjadi semakin lama, sehingga kualitas layanan secara keseluruhan mulai menurun. Pengujian dengan 150 pengguna menunjukkan keterbatasan kapasitas sistem secara lebih jelas. Nilai APDEX turun drastis, dengan Halaman Awal hanya

0.55 (kategori *poor*) dan halaman lainnya sebagian besar berada pada kategori *fair*. Rata-rata *response time* melonjak hingga 665 ms, bahkan Halaman *Devices* mencapai 822 ms dan *Dashboards* 788 ms. *Throughput* justru sedikit menurun menjadi 127,55 transaksi per detik, menandakan bahwa sistem telah mencapai batas kemampuan pemrosesan.

Secara keseluruhan, hasil ini menunjukkan bahwa sistem memiliki performa optimal pada rentang 50 hingga 100 pengguna. Di bawah 50 pengguna, kualitas layanan berada pada kategori *excellent* dengan nilai APDEX mendekati 1.0, *error rate* 0%, dan *response time* yang rendah. Pada jumlah 100 pengguna, performa masih dapat ditoleransi meskipun kualitas layanan mulai menurun. Namun, pada 150 pengguna, degradasi performa terjadi cukup signifikan dengan penurunan nilai APDEX hingga kategori *poor*, peningkatan *response time* yang tajam, serta *throughput* yang tidak lagi meningkat. Hal ini menandakan bahwa kapasitas optimal sistem dalam memberikan layanan berada pada kisaran maksimal 100 pengguna secara bersamaan, dengan Halaman Awal teridentifikasi sebagai titik lemah (*bottleneck*) karena konsisten memiliki *response time* tertinggi dan nilai APDEX terendah dibanding halaman lainnya.

4.6 User Acceptance Testing (UAT)

UAT dilakukan untuk mengevaluasi sejauh mana pengguna menerima dan merasa puas terhadap antarmuka serta fungsionalitas sistem berbasis web yang telah dikembangkan. Metode pengujian ini dilakukan dengan menyebarkan kuisioner kepada sejumlah responden yang telah menggunakan sistem, menggunakan skala Likert 4 poin, yaitu: Sangat Tidak Setuju (1), Tidak Setuju (2), Setuju (3), dan Sangat Setuju (4). Kuisioner terdiri atas 34 pernyataan yang dikelompokkan ke dalam aspek-aspek UI/UX seperti tampilan antarmuka, kemudahan navigasi, kejelasan informasi, performa, responsivitas, kenyamanan penggunaan, dan fungsionalitas fitur. Data hasil kuisioner kemudian dianalisis dengan menghitung nilai rata-rata dari setiap pernyataan, serta mengkonversinya ke dalam bentuk persentase sesuai dengan Persamaan (2.1).



Gambar 4.51 Hasil rata-rata tiap pernyataan

(Sumber: Dokumentasi pribadi)

Berdasarkan hasil rekapitulasi nilai rata-rata pada Gambar 4.51, diperoleh bahwa sebagian besar pernyataan memperoleh skor di atas 3, yang menunjukkan bahwa mayoritas pengguna menyatakan “Setuju” hingga “Sangat Setuju” terhadap kualitas antarmuka dan fungsionalitas sistem. Nilai tertinggi diperoleh pada pernyataan nomor 2 dalam aspek Visualisasi dan Antarmuka, nomor 11 dalam aspek Fungsionalitas dan Interaksi, serta nomor 34 dalam aspek Notifikasi dan Respon Sistem, sebagaimana tercantum dalam Tabel 3.7. Ketiga pernyataan tersebut memiliki rata-rata skor sebesar 3,81 atau setara dengan 95,25%, yang menurut interpretasi pada Tabel 2.3 termasuk dalam kategori “Sangat Setuju”. Sementara itu, pernyataan dengan skor terendah adalah pernyataan nomor 12, yaitu “Sistem memberikan umpan balik ketika terjadi kesalahan,” yang juga termasuk dalam aspek Fungsionalitas dan Interaksi sebagaimana dijelaskan pada Tabel 3.7. Pernyataan ini memperoleh rata-rata skor sebesar 3,63 atau setara dengan 90,75%, yang tetap berada dalam kategori “Setuju” berdasarkan Tabel 2.3.

Jika dihitung secara keseluruhan, nilai rata-rata dari seluruh pernyataan adalah 3,73, yang setara dengan 93,25%. Hasil ini mengindikasikan bahwa sistem telah secara umum diterima dengan sangat baik oleh pengguna dari sisi kenyamanan, kemudahan penggunaan, serta kepuasan terhadap fitur dan antarmuka sistem. Dengan demikian, dapat disimpulkan bahwa sistem yang telah dikembangkan telah

berhasil memenuhi ekspektasi pengguna dan layak untuk digunakan dalam konteks operasional.

4.7 Evaluasi

Berdasarkan hasil pengujian yang telah dilakukan, tugas akhir yang dikembangkan menunjukkan keunggulan signifikan dibandingkan dengan penelitian sebelumnya. Hasil dari evaluasi dapat dilihat pada Tabel 4.3. Evaluasi ini mengacu pada perbandingan hasil dari beberapa penelitian yang telah dirangkum dalam Tabel 2.1.

Tabel 4.3 Komparasi hasil penelitian

Aspek Evaluasi	Tugas Akhir	Febriana & Farros (2024)	Salem dkk. (2022)	Bogdan dkk. (2023)	Raman & Martin (2024)	Shodiq dkk. (2021)
Protokol Komunikasi	HTTP, MQTT *	HTTP	HTTP	HTTP	MQTT	-
Keamanan Data	JWT, AES-128, HTTPS *	-	-	-	-	JWT, XXTEA
<i>Entropy JWT Secret</i>	128 bit *	-	-	-	-	~64 bit
Resistensi <i>Brute Force</i>	~10 ²¹ tahun *	-	-	-	-	~10 ¹⁰ tahun
Platform Akses	<i>Website, Android</i> *	<i>Website</i>	<i>Website</i>	Android	<i>Website</i>	<i>Website</i>
Sistem Notifikasi	<i>Browser (Bawaan), WhatsApp</i>	-	SMS	Android (Bawaan)	-	-
Fungsionalitas	<i>Black Box (100%)</i>	-	<i>Black Box (100%)</i>	<i>Black Box (100%)</i>	<i>Black Box (100%)</i>	-
Performa <i>Website</i>	<i>DOMContent Loaded < 250ms</i> *	-	-	-	-	-
Performa Android	CPU < 10%, RAM 20 – 31MB *	-	-	-	-	-
<i>Load Testing</i>	APDEX ≥ 0.76 saat ≤ 100 pengguna *	-	-	-	-	-
<i>User Acceptance Testing</i>	93,25% *	-	-	-	-	-

(* lebih unggul)

Berdasarkan pengujian fungsionalitas, metode *black-box testing* menunjukkan tingkat keberhasilan 100% pada seluruh 16 skenario uji. Hasil ini menegaskan bahwa sistem dapat beroperasi sesuai dengan kebutuhan fungsional yang telah ditetapkan. Pada aspek performa, hasil pengujian menunjukkan bahwa aplikasi

website memiliki waktu *DOMContentLoaded* di bawah 250 ms (sangat baik) untuk seluruh halaman, meskipun waktu *finish* pada halaman dinamis mencapai 3000 ms (cukup). Sementara itu, aplikasi Android mempertahankan penggunaan CPU di bawah 10% dengan konsumsi memori yang relatif efisien di kisaran ± 120 MB.

Dari sisi aksesibilitas *platform*, sistem yang dikembangkan menyediakan antarmuka ganda berupa *website* dan aplikasi Android. Keberadaan dua *platform* ini memperluas pilihan akses pengguna, berbeda dengan penelitian Febriana & Farros (2024), Raman & Martin (2024), Salem dkk. (2022), dan Shodiq dkk. (2021) yang hanya mendukung antarmuka berbasis *website*. Walaupun Bogdan dkk. (2023) telah menyediakan aplikasi Android, penelitian tersebut belum mengintegrasikan dukungan *website* sehingga akses lintas perangkat masih terbatas.

Dari aspek interoperabilitas protokol komunikasi, sistem yang dikembangkan menunjukkan kemampuan mendukung dua protokol secara simultan, yaitu HTTP dan MQTT. Dukungan terhadap lebih dari satu protokol ini memberikan fleksibilitas integrasi yang lebih tinggi dibandingkan penelitian sebelumnya. Studi oleh Febriana & Farros (2024), Salem dkk. (2022), dan Bogdan dkk. (2023) hanya mengimplementasikan protokol HTTP, sedangkan penelitian Raman & Martin (2024) terbatas pada protokol MQTT. Dengan demikian, rancangan sistem yang diajukan lebih adaptif dalam mengakomodasi berbagai perangkat IoT dengan perbedaan protokol komunikasi dalam satu ekosistem pemantauan.

Pada aspek keamanan data, sistem mengimplementasikan mekanisme berlapis melalui penggunaan JWT untuk autentikasi dan otorisasi serta HTTPS untuk enkripsi komunikasi. Pendekatan ini memberikan tingkat keamanan yang lebih komprehensif dibandingkan sebagian besar penelitian sebelumnya yang tidak mengintegrasikan mekanisme keamanan sama sekali, seperti pada penelitian Febriana & Farros (2024), Raman & Martin (2024), Salem dkk. (2022), Bogdan dkk. (2023), dan Shakhdher dkk. (2019). Walaupun Shodiq dkk. (2021) telah memanfaatkan JWT dan algoritma enkripsi XXTEA, rancangan pada sistem ini lebih unggul karena menambahkan lapisan perlindungan melalui HTTPS sehingga komunikasi data memperoleh jaminan keamanan ujung ke ujung. Dari perspektif ketahanan terhadap serangan *brute force*, *entropy* JWT *secret* yang digunakan

adalah sebesar 128 bit. Secara teoretis, tingkat kompleksitas ini memerlukan waktu sekitar 10^{21} tahun untuk ditembus dengan asumsi satu juta percobaan per detik. Ketahanan ini jauh lebih tinggi dibandingkan dengan penelitian Shodiq dkk. (2021) yang hanya memperkirakan daya tahan sekitar 10^{10} tahun dengan *entropy* ~64 bit. Perbedaan tersebut menunjukkan peningkatan signifikan dalam perancangan *secret key* sehingga sistem lebih terlindungi dari serangan enumerasi kunci.

Dalam hal fitur notifikasi, sistem menawarkan dua mekanisme, yaitu notifikasi bawaan pada *browser* dan notifikasi melalui WhatsApp. Konsep ini memberikan alternatif yang lebih variatif dibandingkan dengan penelitian Bogdan dkk. (2023) yang hanya memanfaatkan notifikasi bawaan aplikasi. Namun, sistem tetap bergantung pada ketersediaan akses internet, berbeda dengan pendekatan Salem dkk. (2022) yang menggunakan SMS sehingga notifikasi dapat diterima meskipun perangkat tidak terkoneksi internet.

Hasil pengujian *load testing* menunjukkan batas kapasitas sistem dalam melayani pengguna secara bersamaan. Pada rentang 10 hingga 50 pengguna, nilai APDEX mendekati 1.0 (*excellent*) dengan nilai *error rate* 0% dan *response time* yang rendah (177–316 ms). *Throughput* meningkat secara proporsional hingga 85,71 transaksi/detik. Saat 100 pengguna, kualitas layanan mulai menurun ke kategori fair (APDEX 0.76–0.79) dengan *response time* 467 ms dan beberapa halaman mencapai 548 ms. Pada 150 pengguna, performa turun signifikan dengan nilai APDEX 0.55 (*poor*) dan *response time* hingga 822 ms serta *throughput* sedikit menurun menjadi 127,55 transaksi/detik. Halaman awal atau *Landing Page* menjadi titik lemah (*bottleneck*) karena konsisten memiliki *response time* tertinggi dan APDEX terendah. Dengan demikian, kapasitas optimal sistem berada pada kisaran maksimal 100 pengguna aktif secara bersamaan.

Terakhir, dari hasil UAT, tingkat penerimaan pengguna mencapai 93,25%. Angka ini mengindikasikan bahwa sistem telah memenuhi ekspektasi pengguna baik dari segi antarmuka, fungsionalitas, maupun pengalaman penggunaan. Hasil UAT ini juga memberikan nilai tambah karena sebagian besar penelitian terdahulu belum melakukan pengujian serupa, sehingga temuan ini memperkuat validitas kualitas sistem yang dikembangkan.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem yang telah dilakukan, maka dapat disimpulkan bahwa:

1. Antarmuka pemantauan berbasis *website* dan Android berhasil dikembangkan dan telah diuji secara komprehensif untuk memantau parameter limbah cair industri, meliputi debit, kekeruhan, pH, NH₃-N, dan COD secara *real-time*. Antarmuka ini dilengkapi dengan fitur *dashboard* pemantauan, pengelolaan perangkat (*device*) dan parameter (*datastream*), serta sistem alarm, sehingga memudahkan pengguna dalam mengakses dan mengontrol data melalui berbagai perangkat.
2. Aspek performa aplikasi menunjukkan hasil yang memadai untuk mendukung kebutuhan pemantauan secara *real-time*. Pada pengujian *website*, waktu *DOMContentLoaded* tercatat di bawah 250 ms untuk seluruh halaman dan tergolong sangat baik, meskipun waktu *finish* pada halaman dinamis dapat mencapai ± 3000 ms dan tergolong cukup. Sementara itu, penggunaan CPU aplikasi Android di bawah 10% dengan konsumsi memori yang relatif efisien di kisaran ± 120 MB, sehingga dapat berjalan dengan ringan pada perangkat seluler dan memberikan pengalaman pengguna yang stabil.
3. Integrasi multi-protokol komunikasi, yaitu HTTP dan MQTT telah berhasil diterapkan pada sistem, sehingga meningkatkan interoperabilitas dan fleksibilitas sistem dibandingkan sistem terdahulu yang hanya mengandalkan satu protokol komunikasi.
4. Mekanisme keamanan sistem telah diimplementasikan secara berlapis, mencakup penggunaan JWT untuk autentikasi dan otorisasi, AES-128 untuk enkripsi data, serta HTTPS untuk menjamin kerahasiaan komunikasi dari ujung ke ujung. *Entropy secret* JWT sebesar 128 bit meningkatkan ketahanan sistem terhadap serangan *brute force* dengan estimasi daya tahan mencapai sekitar 10^{21} tahun. Pendekatan ini menghasilkan perlindungan data yang lebih komprehensif

dan andal dibandingkan sebagian besar penelitian sebelumnya yang belum menerapkan mekanisme keamanan secara menyeluruh.

5. Fitur notifikasi telah diimplementasikan secara terintegrasi, baik melalui notifikasi bawaan pada browser dan aplikasi, maupun melalui platform pihak ketiga seperti WhatsApp. Fitur ini memungkinkan pengguna menerima peringatan ketika parameter pemantauan melampaui ambang batas yang telah ditentukan, sehingga meningkatkan kecepatan respons dan efektivitas sistem dalam mengantisipasi kondisi kritis.
6. Pengujian kinerja sistem melalui *load testing* menunjukkan bahwa kapasitas optimal sistem berada pada kisaran maksimal 100 pengguna aktif secara bersamaan. Pada rentang 10–50 pengguna, nilai APDEX mendekati 1,00 (*excellent*), dengan nilai *error rate* 0% dan rata-rata *response time* 177–316 ms. Pada 100 pengguna, kualitas layanan masih dapat diterima dengan nilai APDEX 0,76–0,79 (*fair*) meskipun terjadi peningkatan rata-rata *response time* menjadi 467 ms. Halaman landing page teridentifikasi sebagai titik lemah (*bottleneck*) karena memiliki waktu tanggap tertinggi dibandingkan halaman lainnya.
7. Hasil *User Acceptance Testing* menunjukkan tingkat penerimaan pengguna sebesar 93,25%, yang mengindikasikan bahwa sistem yang dikembangkan telah memenuhi ekspektasi pengguna dari segi antarmuka, fungsionalitas, kinerja, dan pengalaman penggunaan secara keseluruhan.

5.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan, maka disarankan beberapa hal untuk pengembangan selanjutnya, antara lain:

1. Menambahkan fitur analitik *real-time* berbasis *machine learning* untuk mendukung pengambilan keputusan berbasis data.
2. Mengintegrasikan autentikasi dua faktor (*Two-Factor Authentication/2FA*) untuk peningkatan keamanan akun pengguna.
3. Mengembangkan versi aplikasi iOS agar sistem dapat digunakan oleh lebih banyak pengguna lintas *platform*.
4. Menyediakan sistem manajemen pengguna berbasis peran (*Role-Based Access Control*) untuk meningkatkan fleksibilitas pengelolaan hak akses.

DAFTAR PUSTAKA

- Ahmad, S., Jeon, G., & Park, M. (2022). Assessing the linguistic quality of REST APIs for IoT applications. *Journal of Systems and Software*, 189, 111294. <https://doi.org/10.1016/j.jss.2022.111294>
- Ahmed, S., & Mahmood, Q. (2019). An authentication based scheme for applications using JSON web token. *2019 22nd International Multitopic Conference (INMIC)*, 1-6. <https://doi.org/10.1109/INMIC48123.2019.9022766>
- Akasiadis, C., Tryferidis, A., & Tzovaras, D. (2019). Internet of Things architecture for combined applications. *IEEE Internet of Things Journal*, 6(2), 2055-2070. <https://doi.org/10.1109/JIOT.2018.2877660>
- Aladwani, A. M. (2020). Website performance and bounce rate: An empirical investigation. *International Journal of Information Management*, 54, 102154. <https://doi.org/10.1016/j.ijinfomgt.2020.102154>
- Avancha, S., Goel, O., & Pandian, P. K. G. (2024). Agile project planning and execution in large-scale IT projects. *Deleted Journal*, 12(3), 239–252. <https://doi.org/10.36676/dira.v12.i3.80>
- Ayuningtyas, R., Putri, D. A., & Sari, M. (2023). Performance and functional testing with the black box testing method. *International Journal of Progressive Sciences and Technologies*, 39(2), 234-241. <https://doi.org/10.52155/ijpsat.v39.2.5471>
- Barker, E. (2020). *Recommendation for key management: Part 1 – General* (NIST Special Publication 800-57 Part 1 Rev. 5). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- Biznetgio. (2024). Mengenal agile development, metode yang cocok diterapkan developer. *Biznetgio*. <https://www.biznetgio.com/news/apa-itu-agile-development>
- Bogdan, R., Paliuc, C., Crisan-Vida, M., Nimara, S., & Barmayoun, D. (2023). Low-cost Internet-of-Things water-quality monitoring system for rural areas. *Sensors*, 23(8), 3919. <https://doi.org/10.3390/s23083919>
- Bolanowski, M., Ćmil, M., & Starzec, A. (2024). New model for defining and implementing performance tests. *Future Internet*, 16(10), 366. <https://doi.org/10.3390/fi16100366>
- Bourhis, P., Reutter, J. L., Suárez, F., & Vrgoč, D. (2020). JSON: Data model and query languages. *Information Systems*, 89, 101478. <https://doi.org/10.1016/j.is.2019.101478>

- Brahmbhatt, M., & Vora, H. (2022). Building high-performance web applications using Next.js. *International Journal of Scientific Research in Engineering and Management*, 6(3), 1–5.
- Camilli, M., Janes, A., & Russo, B. (2022). Automated test-based learning and verification of performance models for microservices systems. *Journal of Systems and Software*, 187, 111225. <https://doi.org/10.1016/j.jss.2022.111225>
- Centenaro, M., Vangelista, L., Zanella, A., & Zorzi, M. (2016). Long-range communications in unlicensed bands: The rising stars in the IoT and smart city scenarios. *IEEE Wireless Communications*, 23(5), 60-67. <https://doi.org/10.1109/MWC.2016.7721743>
- Darmawan, I., Mansyur, M. U., Imam, K. Z., Syahdan, M., & Fawaid, A. (2023). Evaluasi keamanan privilege terintegrasi JSON Web Token pada sistem informasi akademik. *Jurnal Informasi dan Teknologi*, 120-128.
- Eismann, S., Scheuner, J., Grohmann, J., Okanović, D., & Herbst, N. (2022). More than just cloud functions: A systematic literature review of serverless performance. *Journal of Systems and Software*, 190, 111305. <https://doi.org/10.1016/j.jss.2022.111305>
- Febriana, A., & Farros, R. A. (2024). Rancang bangun industrial internet of things (IIoT) board multiguna untuk monitoring limbah cair. *Tugas Akhir*, Politeknik Negeri Semarang, 1-91.
- Gonzalez, L., Perez, M., & Romero, E. (2021). Effectiveness of real-time alert systems in IoT monitoring: A review. *Sensors*, 21(15), 5189. <https://doi.org/10.3390/s21155189>
- Google. (2023). Core Web Vitals: What developers should know. *Google Developers*. <https://web.dev/vitals/>
- Gortázar, F., Gallego, M., Maes-Bermejo, M., Chicano, I., & Santos, C. (2022). Cost-effective load testing of WebRTC applications. *Journal of Systems and Software*, 193, 111439. <https://doi.org/10.1016/j.jss.2022.111439>
- Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63b>
- Hosenkhan, M. R., & Pattanayak, B. K. (2021). A framework for secure communication on Internet of Things (IoT). In *Advances in Intelligent Systems and Computing* (pp. 599–605). Springer. https://doi.org/10.1007/978-981-33-4299-6_49
- ISO/IEC. (2016). *ISO/IEC 25023:2016—Systems and software engineering—SQuaRE—Measurement of system and software product quality*. International Organization for Standardization. <https://www.iso.org/standard/35747.html>

- ISO/IEC. (2023). *ISO/IEC 25010:2023—Systems and software engineering—SQuaRE—Product quality model*. International Organization for Standardization. <https://www.iso.org/standard/78176.html>
- Jartarghar, H. A., Salanke, G. R., A.R, A. K., G.S, S., & Dalali, S. (2022). React apps with server-side rendering: Next.js. *Journal of Telecommunication, Electronic and Computer Engineering*, 14(4). <https://doi.org/10.54554/jtec.2022.14.04.005>
- Joshi, A., Kale, S., Chandel, S., & Pal, D. K. (2015). Likert scale: Explored and explained. *British Journal of Applied Science & Technology*, 7(4), 396–403. <https://doi.org/10.9734/BJAST/2015/14975>
- JSON. (2006). JSON: The fat-free alternative to XML. *JSON.org*. <https://json.org>
- JWT.io. (2024). JSON web token. *JWT.io*. <https://jwt.io/>
- Kim, J., Park, S., & Choi, Y. (2021). Design of a RESTful API system for an IoT platform. *Sensors*, 21(14), 4642. <https://doi.org/10.3390/s21144642>
- Lee, J. E.-Y. (2022). *Structured Query Language (SQL)* (pp. 889–893). https://doi.org/10.1007/978-3-319-32010-6_460
- Loring, M. C., Marron, M., & Leijen, D. (2015). "Static Analysis of Event-Driven Node.js JavaScript Applications." *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '15)*. ACM. <https://doi.org/10.1145/2858965.2814272>
- Machuca Yaguana, J. (2023). Tratamiento y representación de datos provenientes de escalas tipo Likert. *Ciencia Latina*, 7(4). https://doi.org/10.37811/cl_rcm.v7i4.6905
- Marnewick, C. (2016). Benefits of information system projects: The tale of two countries. *International Journal of Project Management*, 34(4), 748–760. <https://doi.org/10.1016/j.ijproman.2016.03.010>
- Microsoft. (2024). Visual Studio Code: Code editing. Redefined. *Visual Studio Code*. <https://code.visualstudio.com/>
- Mirani, A. A., Velasco-Hernandez, G., Awasthi, A., & Walsh, J. (2022). Key challenges and emerging technologies in industrial IoT architectures: A review. *Sensors*, 22(15), 5836. <https://doi.org/10.3390/s22155836>
- Mudassir, M., & Mushtaq, M. (2024). The role of APIs in modern software development. *World Journal of Advanced Engineering Technology and Sciences*, 13(1), 1045–1047. <https://doi.org/10.30574/wjaets.2024.13.1.0515>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3rd ed.). John Wiley & Sons.

- Next.js. (2024). The React framework for the web. *Next.js*. <https://nextjs.org/>
- Nurbaya, S. (2019). *Peraturan Menteri Lingkungan Hidup dan Kehutanan. Kementerian Lingkungan Hidup dan Kehutanan*. <https://ppkl.menlhk.go.id/website/filebox/885/200306144608%20Permen%20LHK%20tentang%20SPARING.pdf>
- Nurul, F., Hidayat, H., & Eniati, E. (2020). Analysis of COD, BOD, and DO levels at the wastewater treatment at the Balai PIALAM DPUP-ESDM Special Region of Yogyakarta. *IJCR-Indonesian Journal of Chemical Research*, 5(2), 80–89. <https://doi.org/10.20885/ijcer.vol5.iss2.art5>
- Pavlović, A., Zdravković, M., Misic, V., & Sladic, G. (2021). Performance impact of optimization methods on MySQL document-based and relational databases. *Applied Sciences*, 11(15), 6794. <https://doi.org/10.3390/app11156794>
- Pramukantoro, E. K. (2020). *Internet of Things dengan Python, ESP32, dan Raspberry PI: Teori dan praktik*. Universitas Brawijaya Press.
- Raman, R., & Martin, N. (2024). IoT-enabled water pollution detection for real-time monitoring and pollution source identification with MQTT protocol. *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, 1–6. <https://doi.org/10.1109/ADICS58448.2024.10533607>
- Sabherwal, R., Chan, Y. E., & Hirschheim, R. (2019). Recent developments in research on the management of information technology: An overview of leading journals. *Journal of Information Technology*, 34(2), 97–112. <https://doi.org/10.1177/0268396219831986>
- Salem, R. M. M., Saraya, M. S., & Ali-Eldin, A. M. T. (2022). An industrial cloud-based IoT system for real-time monitoring and controlling of wastewater. *IEEE Access*, 10, 6528–6540. <https://doi.org/10.1109/ACCESS.2022.3141977>
- Saputra, W. Y., Sugiarti, S., Junianto, H., & Suhartono, D. (2025). Password strength study using the zxcvbn algorithm and brute-force time estimation to strengthen cybersecurity. *Journal of Computing and Information System*, 21(1), 52–59. <https://doi.org/10.33480/pilar.v21i1.6119>
- Sari, D. R. (2024). Analisis keamanan sistem informasi dalam era Internet of Things (IoT). *Technologia Journal: Jurnal Informatika*, 1(2), 1-10.
- Sawitri, D. (2023). Internet of Things memasuki era society 5.0. *KITEKTRO: Jurnal Komputer, Informasi Teknologi, dan Elektro*, 8(1), 31-35.
- Shah, R., & Correia, S. (2021). *Encryption of Data over AES (Advanced Encryption Standard) and HTTPS (Hypertext Transfer Protocol Secure) Requests for Secure Data transfers over the Internet*. <https://doi.org/10.1109/RTEICT52294.2021.9573978>

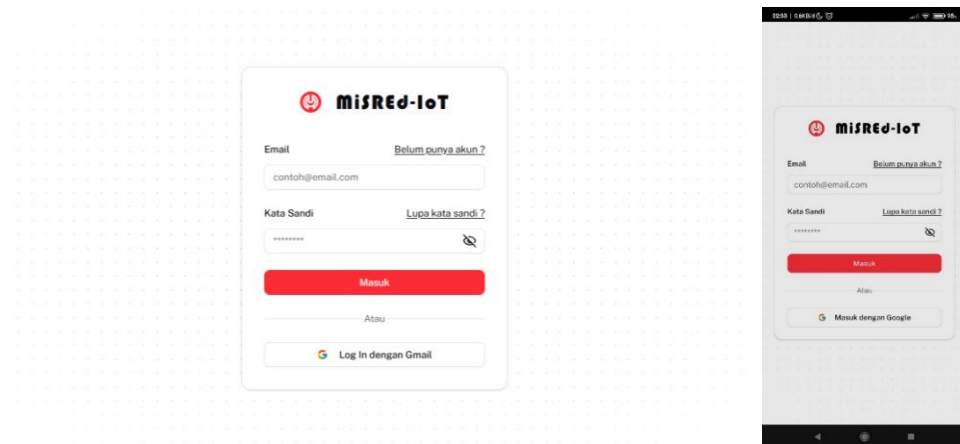
- Shakdher, A., Agrawal, S., & Yang, B. (2019). Security vulnerabilities in consumer IoT applications. *2019 IEEE 5th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing (HPSC), and IEEE International Conference on Intelligent Data and Security (IDS)*, 1-6. <https://doi.org/10.1109/BigDataSecurity-HPSC-IDS.2019.00012>
- Shodiq, F. A., Pahlevi, R. R., & Sukarno, P. (2021). Secure MQTT authentication and message exchange methods for IoT constrained device. *2021 International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 70–74. <https://doi.org/10.1109/ICICyTA53712.2021.9689126>
- Vainer, M. (2023). Multi-purpose password dataset generation and its application in decision making for password cracking through machine learning. *New Trends in Computer Sciences*, 1(1), 1-18. <https://doi.org/10.3846/ntcs.2023.17639>
- Vercel. (2023). Introduction to Next.js. *Vercel*. <https://vercel.com/solutions/nextjs>
- Xia, X., Wang, C., Hoffmann, H., Zhang, L., & Jia, C. (2024). Reducing the length of field-replay-based load testing. *IEEE Transactions on Software Engineering*, 50(9), 1–17. <https://doi.org/10.1109/TSE.2024.3408079>

LAMPIRAN

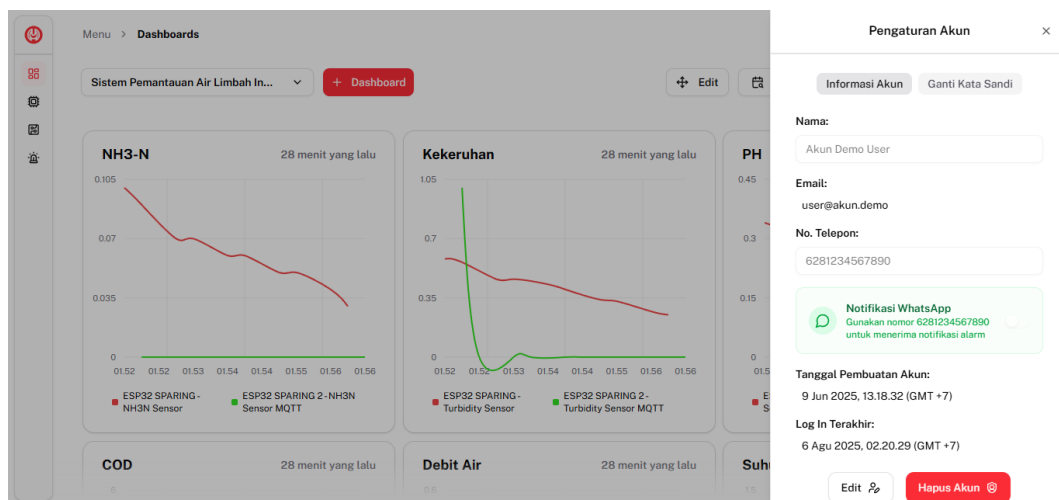
Lampiran 1 Halaman *Landing Page* Aplikasi *Website*



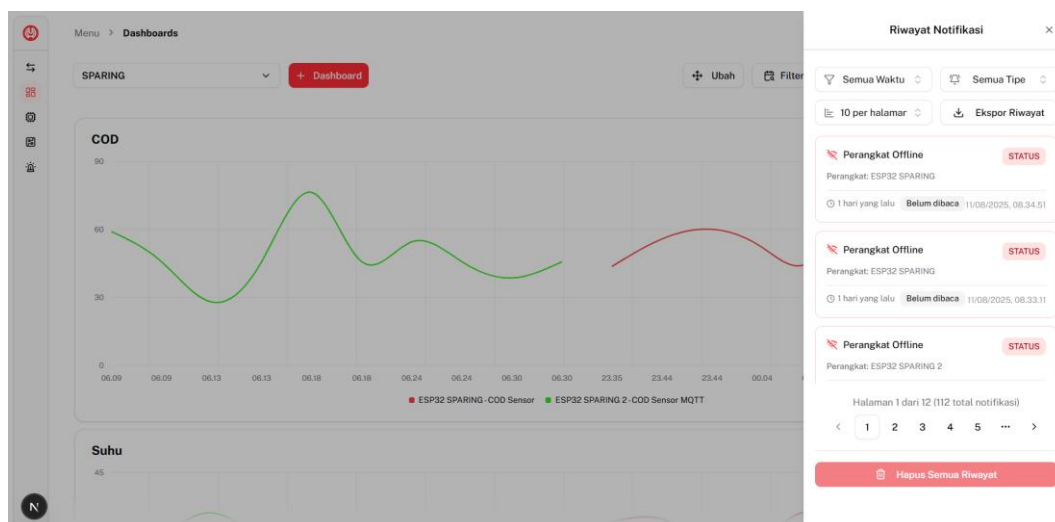
Lampiran 2 Halaman *Login* Aplikasi *Website* dan *Android*



Lampiran 3 Pengaturan Akun Aplikasi *Website*



Lampiran 4 Riwayat Notifikasi Aplikasi *Website*



Lampiran 5 Fitur Ekspor

The screenshot shows a modal dialog titled 'Ekspor 10 data terakhir' (Export Last 10 Data). It allows users to select a datastream for export. The 'Pilih Datastream untuk Diekspor' (Select Datastream to Export) section includes a search bar and a list of sensors: pH Sensor (V0), Flow Sensor (V1), COD Sensor (V2), Temperature Sensor (V3), and NH3N Sensor (V4). The 'Pilih Format Ekspor' (Select Export Format) section offers two options: 'CSV (Kompatibel dengan Microsoft Excel)' (checked) and 'PDF (Siap Cetak)' (unchecked). At the bottom, there are buttons for 'Ekspor Data' (Export Data) and 'Batal' (Cancel).

Lampiran 6 Hasil Ekspor CSV Data Perangkat

	Tanggal dan Waktu	UID Perangkat	Nama Perangkat	Datastream	Virtual Pin	Nilai
1	10/08/2025, 16.35.46	1	ESP32 SPARING	pH Sensor	V0	7.300
2	10/08/2025, 16.35.46	1	ESP32 SPARING	Flow Sensor	V1	37.010
3	10/08/2025, 16.44.38	1	ESP32 SPARING	pH Sensor	V0	6.900
4	10/08/2025, 16.44.38	1	ESP32 SPARING	Flow Sensor	V1	45.150
5	10/08/2025, 16.44.43	1	ESP32 SPARING	pH Sensor	V0	6.600
6	10/08/2025, 16.44.43	1	ESP32 SPARING	Flow Sensor	V1	37.030
7	10/08/2025, 17.04.35	1	ESP32 SPARING	pH Sensor	V0	7.600
8	10/08/2025, 17.04.35	1	ESP32 SPARING	Flow Sensor	V1	18.330
9	10/08/2025, 17.04.40	1	ESP32 SPARING	pH Sensor	V0	7.900
10	10/08/2025, 17.04.40	1	ESP32 SPARING	Flow Sensor	V1	40.530
11	10/08/2025, 19.29.50	1	ESP32 SPARING	pH Sensor	V0	8.100
12	10/08/2025, 19.29.50	1	ESP32 SPARING	Flow Sensor	V1	10.240
13	10/08/2025, 19.29.55	1	ESP32 SPARING	pH Sensor	V0	6.900
14	10/08/2025, 19.29.55	1	ESP32 SPARING	Flow Sensor	V1	32.490
15	11/08/2025, 01.31.58	1	ESP32 SPARING	pH Sensor	V0	7.100

Lampiran 7 Hasil Ekspor PDF Data Perangkat

Laporan Data Perangkat																																															
Laporan ini dibuat secara otomatis pada 06/08/2025, 02.28.52																																															
Informasi Laporan Jumlah Perangkat 2 perangkat Jumlah Datastream 2 sensor Jumlah Data 20 titik data Rentang Tanggal 05/08/2025, 18.52.14 - 05/08/2025, 18.56.50																																															
Datastream yang Dipilih ESP32 SPARING: V3 Temperature Sensor ESP32 SPARING 2: V3 Temperature Sensor MQTT																																															
Data ESP32 SPARING (UID: 1) <table> <tr> <th>TANGGAL DAN WAKTU</th><th>DATASREAM</th><th>VIRTUAL PIN</th><th>NILAI</th></tr> <tr><td>05/08/2025, 18.52.14</td><td>Temperature Sensor</td><td>V3</td><td>1.420</td></tr> <tr><td>05/08/2025, 18.52.44</td><td>Temperature Sensor</td><td>V3</td><td>1.370</td></tr> <tr><td>05/08/2025, 18.53.14</td><td>Temperature Sensor</td><td>V3</td><td>1.170</td></tr> <tr><td>05/08/2025, 18.53.44</td><td>Temperature Sensor</td><td>V3</td><td>1.100</td></tr> <tr><td>05/08/2025, 18.54.14</td><td>Temperature Sensor</td><td>V3</td><td>0.940</td></tr> <tr><td>05/08/2025, 18.54.45</td><td>Temperature Sensor</td><td>V3</td><td>0.910</td></tr> <tr><td>05/08/2025, 18.55.14</td><td>Temperature Sensor</td><td>V3</td><td>0.760</td></tr> <tr><td>05/08/2025, 18.55.44</td><td>Temperature Sensor</td><td>V3</td><td>0.710</td></tr> <tr><td>05/08/2025, 18.56.14</td><td>Temperature Sensor</td><td>V3</td><td>0.690</td></tr> <tr><td>05/08/2025, 18.56.44</td><td>Temperature Sensor</td><td>V3</td><td>0.610</td></tr> </table>				TANGGAL DAN WAKTU	DATASREAM	VIRTUAL PIN	NILAI	05/08/2025, 18.52.14	Temperature Sensor	V3	1.420	05/08/2025, 18.52.44	Temperature Sensor	V3	1.370	05/08/2025, 18.53.14	Temperature Sensor	V3	1.170	05/08/2025, 18.53.44	Temperature Sensor	V3	1.100	05/08/2025, 18.54.14	Temperature Sensor	V3	0.940	05/08/2025, 18.54.45	Temperature Sensor	V3	0.910	05/08/2025, 18.55.14	Temperature Sensor	V3	0.760	05/08/2025, 18.55.44	Temperature Sensor	V3	0.710	05/08/2025, 18.56.14	Temperature Sensor	V3	0.690	05/08/2025, 18.56.44	Temperature Sensor	V3	0.610
TANGGAL DAN WAKTU	DATASREAM	VIRTUAL PIN	NILAI																																												
05/08/2025, 18.52.14	Temperature Sensor	V3	1.420																																												
05/08/2025, 18.52.44	Temperature Sensor	V3	1.370																																												
05/08/2025, 18.53.14	Temperature Sensor	V3	1.170																																												
05/08/2025, 18.53.44	Temperature Sensor	V3	1.100																																												
05/08/2025, 18.54.14	Temperature Sensor	V3	0.940																																												
05/08/2025, 18.54.45	Temperature Sensor	V3	0.910																																												
05/08/2025, 18.55.14	Temperature Sensor	V3	0.760																																												
05/08/2025, 18.55.44	Temperature Sensor	V3	0.710																																												
05/08/2025, 18.56.14	Temperature Sensor	V3	0.690																																												
05/08/2025, 18.56.44	Temperature Sensor	V3	0.610																																												
Data ESP32 SPARING 2 (UID: 2) <table> <tr> <th>TANGGAL DAN WAKTU</th><th>DATASREAM</th><th>VIRTUAL PIN</th><th>NILAI</th></tr> <tr><td>05/08/2025, 18.52.44</td><td>Temperature Sensor MQTT</td><td>V3</td><td>1.000</td></tr> <tr><td>05/08/2025, 18.52.50</td><td>Temperature Sensor MQTT</td><td>V3</td><td>1.000</td></tr> <tr><td>05/08/2025, 18.53.44</td><td>Temperature Sensor MQTT</td><td>V3</td><td>1.000</td></tr> </table>				TANGGAL DAN WAKTU	DATASREAM	VIRTUAL PIN	NILAI	05/08/2025, 18.52.44	Temperature Sensor MQTT	V3	1.000	05/08/2025, 18.52.50	Temperature Sensor MQTT	V3	1.000	05/08/2025, 18.53.44	Temperature Sensor MQTT	V3	1.000																												
TANGGAL DAN WAKTU	DATASREAM	VIRTUAL PIN	NILAI																																												
05/08/2025, 18.52.44	Temperature Sensor MQTT	V3	1.000																																												
05/08/2025, 18.52.50	Temperature Sensor MQTT	V3	1.000																																												
05/08/2025, 18.53.44	Temperature Sensor MQTT	V3	1.000																																												

Lampiran 8 Hasil Ekspor CSV Riwayat Notifikasi

	Tanggal/Waktu	Tipe	Judul	Pesan	Device	Sensor	Nilai	Kondisi	Status
55	06/08/2025, 14.48.23	Status Perangkat	Status Perangkat	Perangkat "ESP32 SPARING" telah offline dan tidak merespons	ESP32 SPARING	N/A	N/A	N/A	Belum dibaca
56	06/08/2025, 14.48.24	Status Perangkat	Status Perangkat	Perangkat "ESP32 SPARING" telah offline dan tidak merespons	ESP32 SPARING	N/A	N/A	N/A	Belum dibaca
57	06/08/2025, 14.48.29	Status Perangkat	Status Perangkat	Perangkat "ESP32 SPARING 2" telah offline dan tidak merespons	ESP32 SPARING 2	N/A	N/A	N/A	Belum dibaca
58	06/08/2025, 14.48.37	Status Perangkat	Status Perangkat	Perangkat "ESP32 SPARING 2" telah offline dan tidak merespons	ESP32 SPARING 2	N/A	N/A	N/A	Belum dibaca
59	06/08/2025, 20.39.45	Alarm	Debit air terlalu rendah	Perangkat: ESP32 SPARING Datastream: Flow Sensor (V1) Nilai pemicu: 5.26 Kondisi: V1 < 15.000 Waktu: 6/8/2025, 20.39.46 WIB	ESP32 SPARING	Flow Sensor	5.26	V1 < 15.000	Belum dibaca
60	06/08/2025, 20.40.45	Alarm	Debit air terlalu rendah	Perangkat: ESP32 SPARING Datastream: Flow Sensor (V1) Nilai pemicu: 0.39 Kondisi: V1 < 15.000 Waktu: 6/8/2025, 20.40.46 WIB	ESP32 SPARING	Flow Sensor	0.39	V1 < 15.000	Belum dibaca
61	06/08/2025, 20.41.45	Alarm	Debit air terlalu rendah	Perangkat: ESP32 SPARING Datastream: Flow Sensor (V1) Nilai pemicu: 0.04 Kondisi: V1 < 15.000 Waktu: 6/8/2025, 20.41.46 WIB	ESP32 SPARING	Flow Sensor	0.04	V1 < 15.000	Belum dibaca

Lampiran 9 Hasil Ekspor PDF Riwayat Notifikasi

Laporan Riwayat Notifikasi

Laporan ini dibuat secara otomatis pada 12/08/2025, 13.34.03

Ringkasan Laporan

JUMLAH PERANGKAT
2 perangkat

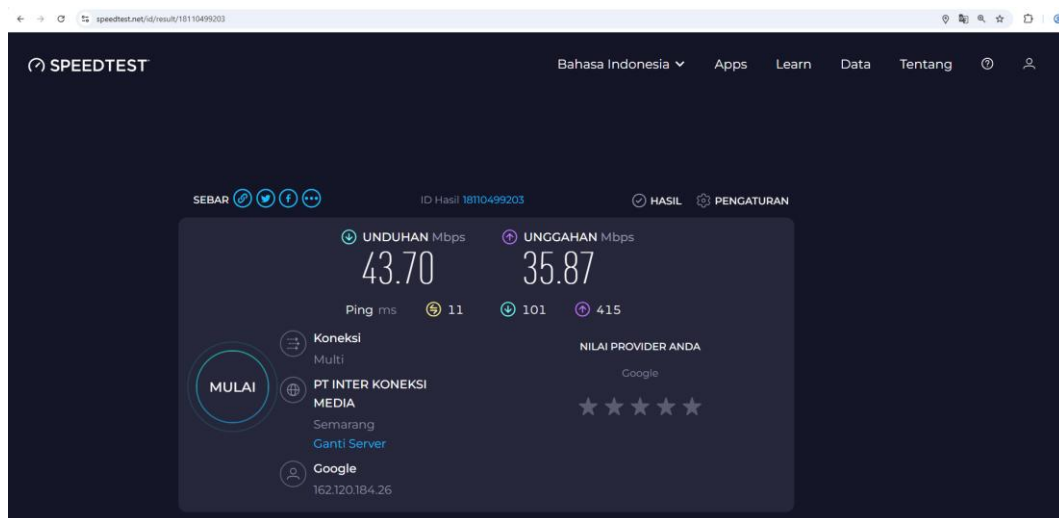
DIBUAT
12/08/2025, 13.34.03

TOTAL NOTIFIKASI
100 peringatan

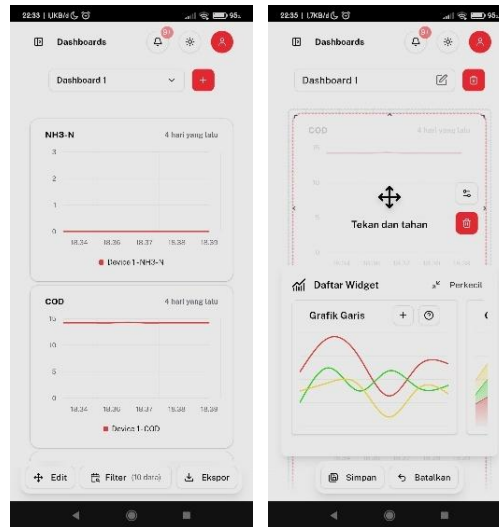
Notifikasi ESP32 SPARING (UID: 1)

TANGGAL & WAKTU	TIPE	JUDUL/ALARM	SENSOR	NILAI	KONDISI
11/08/2025, 20.05.16	Status	Perangkat Offline	-	-	-
11/08/2025, 19.16.45	Status	Perangkat Offline	-	-	-
11/08/2025, 19.08.15	Status	Perangkat Offline	-	-	-
11/08/2025, 18.09.15	Status	Perangkat Offline	-	-	-
11/08/2025, 16.24.15	Status	Perangkat Offline	-	-	-
06/08/2025, 21.24.56	Status	Status Perangkat	-	-	-
06/08/2025, 21.23.52	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.53.20	Status	Status Perangkat	-	-	-
06/08/2025, 20.51.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.50.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.49.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.48.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.47.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.46.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.45.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.44.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.43.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.42.45	Alarm	Debit air terlalu rendah	Flow Sensor	0	V1 < 15.000
06/08/2025, 20.41.45	Alarm	Debit air terlalu rendah	Flow Sensor	0.04	V1 < 15.000

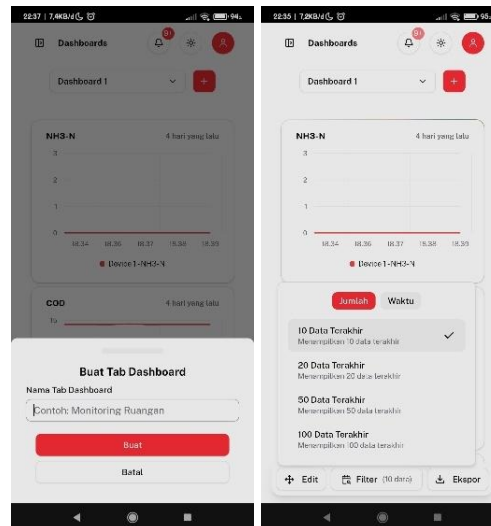
Lampiran 10 Hasil Speedtest Jaringan



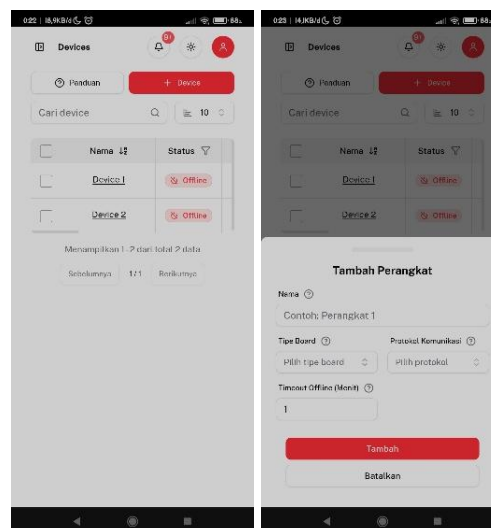
Lampiran 11 Halaman dan Mode *Edit Dashboard* Aplikasi Android



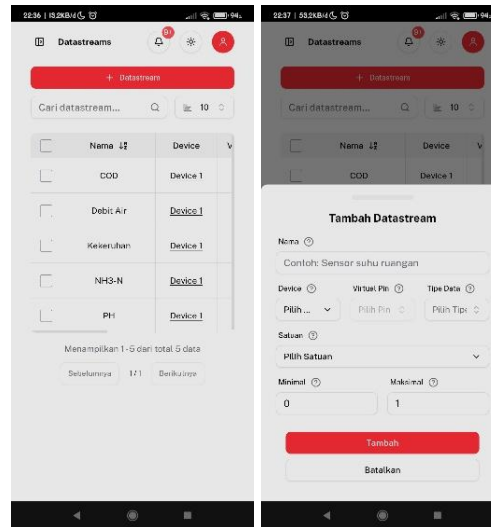
Lampiran 12 Fitur Tambah dan *Filter Dashboard* Aplikasi Android



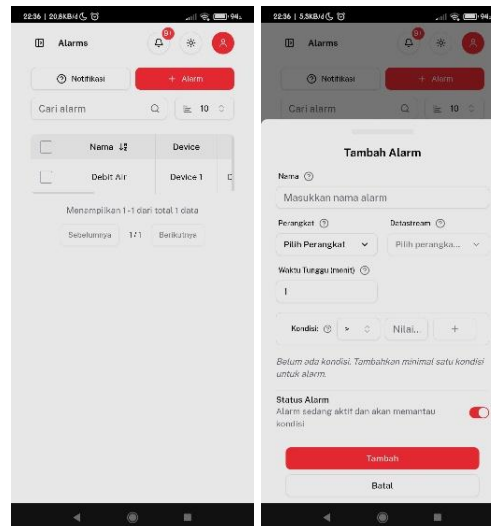
Lampiran 13 Halaman dan Fitur Tambah *Device* Aplikasi Android



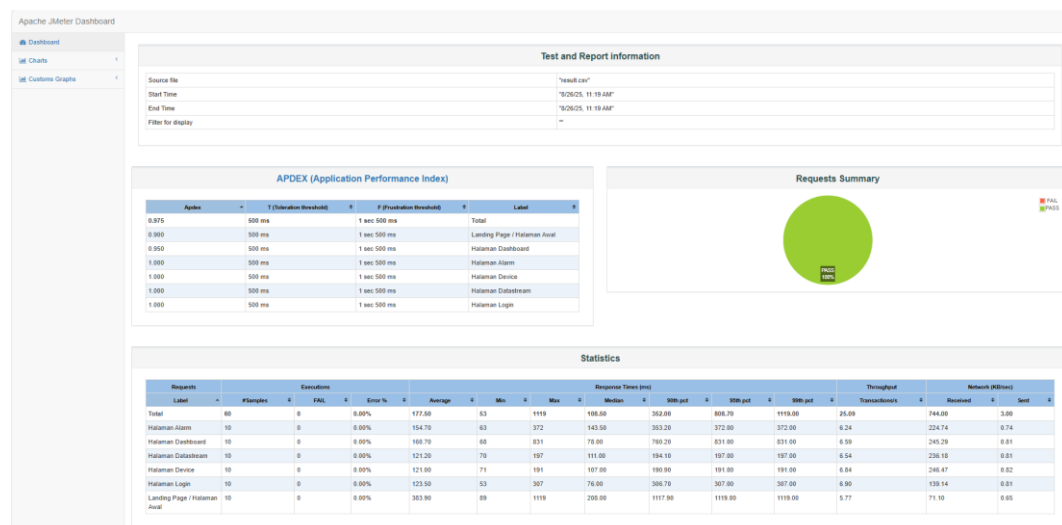
Lampiran 14 Halaman dan Fitur Tambah *Datastream* Aplikasi Android



Lampiran 15 Halaman dan Fitur Tambah Alarm Aplikasi Android



Lampiran 16 Hasil *Load Test* Aplikasi JMeter



Lampiran 17 Sensor PH Meter Kit V2 SKU SEN0161-V2



Lampiran 18 Datasheet Sensor PH Meter Kit V2 SKU SEN0161-V2

DFRobot Gravity: Analog pH meter pro V2 is specifically designed to measure the pH of the solution and reflect the acidity or alkalinity. It is commonly used in various applications such as aquaponics, aquaculture, and environmental water testing.

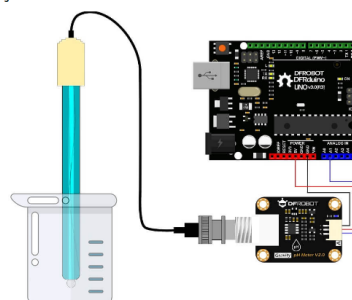
As an upgraded version of pH meter pro V1, the secondary generation pH meter pro greatly improves the precision and user experience. The onboard voltage regulator chip supports the wide voltage supply of 3.3~5.5V, which is compatible with 5V and 3.3V main control boards, such as Arduino and LattePanda. The output signal filtered by hardware has low jitter. The software library adopts the two-point calibration method, and can automatically identify two standard buffer solutions (4.0 and 7.0), so simple and convenient. It uses an industry electrode and has a built-in simple, convenient, practical connection and long life, very suitable for online monitoring.

This industry pH combination electrode is made of a sensitive glass membrane with low impedance. It can be used in a variety of PH measurements with fast response, and good thermal stability. It has good reproducibility, difficult to hydrolysis, and basically eliminates the alkali error. Between 0 to 14 pH range, the output voltage of the electrode is linear. The reference system which consists of the Ag/AgCl gel electrolyte salt bridge has a stable half-cell potential and excellent anti-pollution performance. The ring PTFE membrane is not easy to be clogged, so the electrode is suitable for long-term online detection.

With this product, the main control board (such as Arduino), and the software library, you can quickly build the pH meter, plug, and play, with no welding. DFRobot provides a variety of water quality sensor products, uniform size, and interface, not only meeting the needs of various water quality testing but are also suitable for the DIY multi-parameter water quality tester. You may also check Liquid Sensor Selection Guide to get better familiar with our liquid sensor series.

The pH is a value that measures the acidity or alkalinity of the solution. It is also called the hydrogen ion concentration index. The pH is a scale of hydrogen ion activity in the solution. The pH has a wide range of uses in medicine, chemistry, and agriculture. Usually, the pH is a number between 0 to 14. Under the thermodynamic standard conditions, pH=7, which means the solution is neutral; pH<7, which means the solution is acidic; pH>7, which means the solution is alkaline.

Arduino Connection Diagram



NOTES

- The BNC connector and the signal conversion board must be kept dry and clean, otherwise, it will affect the input impedance, resulting in an inaccurate measurement. If it is damp, it needs to be dried.
- The signal conversion board cannot be directly placed on a wet or semiconductor surface, otherwise, it will affect the input impedance, resulting in an inaccurate measurement. It is recommended to use the nylon pillar to fix the signal conversion board, and allow a certain distance between the signal conversion board and the surface.
- The sensitive glass bubble in the head of the pH probe should avoid touching the hard material. Any damage or scratches will cause the electrode to fail.
- After completing the measurement, disconnect the pH probe from the signal conversion board. The pH probe should not be connected to the signal conversion board without the power supply for a long time.

THE PACKAGE INCLUDES

- 1x pH Probe (Industrial Grade)
- 1x pH Signal Conversion Board V2
- 1x Gravity Analog Sensor Cable
- 2x Waterproof Gasket
- 1x Screw Cap for BNC Connector
- 4x M3 x10 nylon pillar
- 8x M3 x5 screws

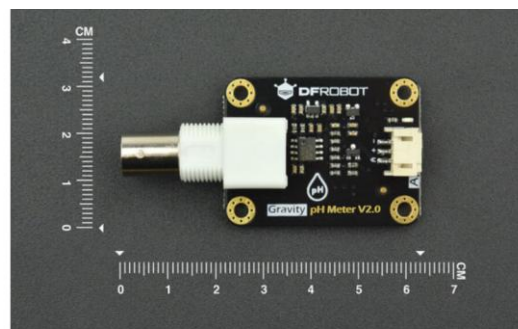
Signal Conversion Board (Transmitter) V2

- Supply Voltage: 3.3~5.5V
- Output Voltage: 0~3.0V
- Probe Connector: BNC
- Signal Connector: PH2.0-3P
- Measurement Accuracy: ± 0.1 @25°C
- Dimension: 42mm*52mm/1.66*1.26in

pH Probe

- Probe Type: Industrial Grade
- Detection Range: 0~14
- Temperature Range: 0~60°C
- Accuracy: ± 0.1 pH (25 °C)
- Response Time: <1min
- Probe Life: 7~24hours >0.5 years (depending on the water quality)
- Cable Length: 500cm

- 3.3~5.5V wide voltage input
- Hardware filtered output signal, low jitter
- Gravity connector and BNC connector, plug-and-play, no soldering required
- The software library supports two-point calibration and automatically identifies standard buffer solution



- Uniform size and connector, convenient for the design of mechanical structures
- Industrial Probe - Support 7~24 Measurement
- Upgraded Transmitter - More Accurate and Stable

Lampiran 19 Sensor UV254 COD



Lampiran 20 Datasheet UV254 COD

Item	Technical Specification
Measurement Method	Absorption measurement at 254 nm, DIN 38402 C2, DIN 38404, HJ/T191 standard
Range	6mm cuvette gap 0.15~300mg/L equiv. KHP(COD solution) 2mm cuvette gap 0.5~1000mg/L equiv. KHP(COD solution) 2mm cuvette gap 1.5~1500mg/L equiv. KHP(COD solution)
Resolution	0.1mg/L or 0.01mg/L COD
Repeatability	+/-1% FS equiv. KHP
Operating temperature	0~45°C
Storage Temperature	-10~50°C
Clean system	Auto wiper system is optional
Warranty	1 year
Depth	IP68, 10m Max
Power	12V 20mA (normal), 200mA (Max)
Output	RS485 and Modbus protocol
Materials	Titanium, SS316, POM, Nylon, PUR, quartz glass
Dimensions	Length 280mm, diameter 142x157mm
Flow rate	< 3 m/s
Response time	Minimum 10s T90
Field life*	Sensor 3 years or greater, wiper sub-system 1.5 years or greater
Recommended Calibration Frequency*	Sensor 3 months, wiper sub-system inspection 6 months.

Lampiran 21 Sensor DSN260 *Electronic Nitrate*



Lampiran 22 Datasheet Sensor DSN260 *Electronic Nitrate*

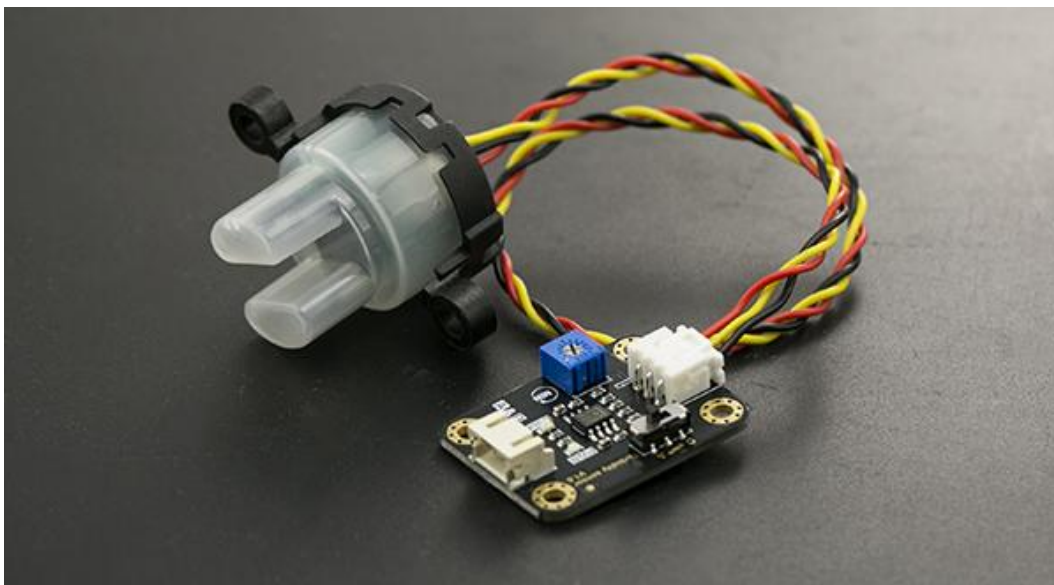
Technical parameter

Product name	Nitrate sensor
Model	DSN260
Detection principle	Electrode method
Measurement range	0-100mg/L
Measurement accuracy	< 5%
Resolution	0.1mg/L
Temperature range	0 ~ 50°C
Output signal	RS-485, MODBUS protocol

Specifications

Waterproof level	IP68
Under pressure	1bar
Product material	POM
Product Size	Φ48 X 209mm
Power information	DC 6-12V, current <50mA
Cable length	Standard 5 meters, longer can be customized

Lampiran 23 Sensor *Turbidity Meter* SKU SEN0189

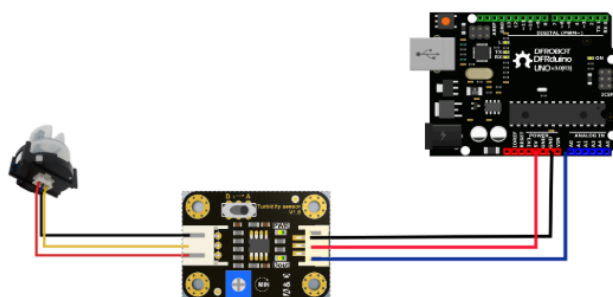


Lampiran 24 Datasheet Sensor *Turbidity Meter* SKU SEN0189

Specification

- Operating Voltage: 5V DC
- Operating Current: 40mA (MAX)
- Response Time : <500ms
- Insulation Resistance: 100M (Min)
- Output Method:
 - Analog output: 0-4.5V
 - Digital Output: High/Low level signal (you can adjust the threshold value by adjusting the potentiometer)
- Operating Temperature: 5°C~90°C
- Storage Temperature: -10°C~90°C
- Weight: 30g
- Adapter Dimensions: 38mm*28mm*10mm/1.5inches *1.1inches*0.4inches

Connection Diagram



Interface Description:

1. "D/A" Output Signal Switch
 - i. "A": Analog Signal Output, the output value will decrease when in liquids with a high turbidity
 - ii. "D": Digital Signal Output, high and low levels, which can be adjusted by the threshold potentiometer
2. Threshold Potentiometer: you can change the trigger condition by adjusting the threshold potentiometer in digital signal mode.

Lampiran 25 Sensor OF05ZAT Liquid Flow Sensor



Lampiran 26 Datasheet Sensor OF05ZAT Liquid Flow Sensor

Application examples

1. For flow check of lubrication oil

Combination use of a flowsensor and PLC can detect clogging in a lubrication oil pipe and avoid machine breakdown.

2. For monitoring of dryer's kerosene

Combination use of a flowsensor, a pump, a solenoid valve, PLC can control dried condition inside a dryer by controlling fan speed in accordance with kerosene supply amount.

Wiring instruction

【Voltage pulse output】(Z□T-AR)

Connect under load resistance (impedance) between White-Black as 10kΩ or more

【NPN open collector pulse output】(Z□T-MR)

Is (Output sink current: mA) = $\frac{V}{R}$ (Power supply voltage: Volt) ≤ 8 mA
Current limiting resistor

Applied voltage of sensor power supply (Red-Black) and pulse output (Blue-White-Black) shall be the same.

External dimensions

Model	OF05	OF10
A	80	90
B	46.9	46.9
C	46.9	46.9
D	8	8.5
E	27.3	40.3

Unit: mm

*About the drawing direction of lead wire (1) for open collector output type and (2) for voltage pulse output type

Connection method

+ pole (power supply) is to be connected to red and - pole is to be connected to black. At the time of power voltage 12V DC, High is not less than 10V DC and Low is not more than 1V DC.

External dimensions

Unit: mm

Model	OF05	OF10
A	80	90
B	46.9	46.9
C	46.9	46.9
D	8	8.5
E	27.3	40.3

Lampiran 27 *Hardware* atau Alat Pengujian Sistem Pemantauan Limbah



Lampiran 28 Hasil Pembacaan Sensor

No.	Waktu	pH	COD (mg/L)	TSS (NTU)	NH3-N (mg/L)	Debit (L/s)
1	11:24:26	6.75	1.35	5.12	0.06	1.20
2	11:26:36	6.73	1.34	5.10	0.06	1.21
3	11:28:28	6.71	1.36	5.14	0.06	1.19
4	11:30:33	6.75	1.33	5.18	0.05	1.22
5	11:32:30	6.74	1.35	5.11	0.06	1.20
6	11:34:52	6.70	1.37	5.15	0.06	1.18
7	11:36:41	6.76	1.34	5.13	0.06	1.23
8	11:38:55	6.72	1.36	5.12	0.06	1.20
9	11:40:43	6.78	1.33	5.19	0.05	1.21
10	11:42:55	6.71	1.35	5.14	0.06	1.19

Lampiran 29 Hasil Pembacaan Sensor (Setelah penambahan HCL)

No.	Waktu	pH	COD (mg/L)	TSS (NTU)	NH3-N (mg/L)	Debit (L/s)
1	12:13:27	5.18	1.34	5.12	0.03	1.74

2	12:15:35	5.12	1.33	5.14	0.02	1.73
3	12:17:31	5.09	1.36	5.11	0.03	1.69
4	12:19:42	5.25	1.35	5.15	0.03	1.72
5	12:21:41	5.14	1.34	5.13	0.02	1.70
6	12:23:44	5.22	1.33	5.18	0.03	1.66
7	12:25:54	5.08	1.35	5.12	0.02	1.71
8	12:27:45	5.19	1.34	5.10	0.03	1.70
9	12:30:04	5.11	1.36	5.16	0.03	1.71
10	12:32:01	5.16	1.35	5.14	0.03	1.68

Lampiran 30 *Source Code* Aplikasi

Instalasi dan Setup

1. Kloning Repositori

```
git clone https://github.com/username/misred-iot.git
cd misred-iot
```

2. Instalasi Dependensi Global

Pilih salah satu pengelola paket:

Menggunakan Bun (Direkomendasikan)

```
# Install Bun jika belum ada
curl -fsSL https://bun.sh/install | bash

# Instalasi dependensi
bun install
```

Menggunakan NPM

```
npm install
```

Alamat *Source Code* Aplikasi : <https://github.com/muh-adrian76/misred-iot>